

CIS 6270

Module 3: Score Matching

Lecture 3.3: Diffusion Paths and Probability-Flow ODE

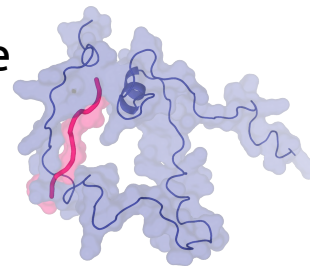
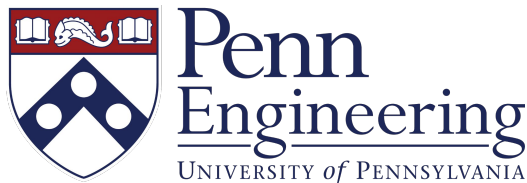
Tue, Sep 22, 2026

Pranam Chatterjee, PhD

Assistant Professor of Bioengineering

Assistant Professor of Computer and Information Science

University of Pennsylvania



We can now put the DDPM training and sampler together

We can now put the DDPM training and sampler together

From the schedule, calculate α_k and cumulative retention $\bar{\alpha}_k$.

We can now put the DDPM training and sampler together

From the schedule, calculate α_k and cumulative retention $\bar{\alpha}_k$. During training, predict the sampled corruption noise.

We can now put the DDPM training and sampler together

From the schedule, calculate α_k and cumulative retention $\bar{\alpha}_k$. During training, predict the sampled corruption noise. During sampling, use that prediction in the reverse Gaussian mean.

We can now put the DDPM training and sampler together

From the schedule, calculate α_k and cumulative retention $\bar{\alpha}_k$. During training, predict the sampled corruption noise. During sampling, use that prediction in the reverse Gaussian mean.

The image and embedding walkthroughs below keep the same training and sampling structure.

Schedule and forward corruption

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)
alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
) # At index k, store alpha_bar_{k-1}.

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *[1] * (len(x_shape) - 1))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bars), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.
    return xk, noise # Return x_k and the exact noise target.
```

Schedule and forward corruption

Creates $\beta_0 = 0$ followed by K linearly increasing noise variances β_1 through β_K .

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
) # At index k, store alpha_bar_{k-1}.

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *[1] * (len(x_shape) - 1))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bars), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.
    return xk, noise # Return x_k and the exact noise target.
```

Schedule and forward corruption

Compute the one-step signal retention

$$\alpha_k = 1 - \beta_k$$

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
# alpha_bar_k = torch.cumsum(alphas, dim=0) # alpha_bar_k = product of alpha_1 through alpha_k
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
) # At index k, store alpha_bar_{k-1}.

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *[1] * (len(x_shape) - 1))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bars), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.
    return xk, noise # Return x_k and the exact noise target.
```

Schedule and forward corruption

Compute the one-step signal retention

$$\alpha_k = 1 - \beta_k$$

The one-step signal retention α_k equals one minus the noise variance β_k added at diffusion step k .

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
# alpha_bar_k = sum_{t=0}^k alpha_t * beta_{t+1} * ... * beta_K
alpha_bar_prev = torch.cat(
    [torch.ones(1, device=DEVICE), alphas[:-1]]
) # At index k, store alpha_bar_{k-1}.

posterior_variances = (
    betas
    * (1.0 - alpha_bar_prev)
    / (1.0 - alpha_bar_prev.clamp(min=1e-20))
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *[1] * (len(x_shape) - 1))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bar), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bar), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.
    return xk, noise # Return x_k and the exact noise target.
```

Schedule and forward corruption

Compute the one-step signal retention

$$\alpha_k = 1 - \beta_k$$

Then, compute cumulative signal retention:

$$\bar{\alpha}_k = \prod_{j=1}^k \alpha_j$$

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)
alphas = 1.0 - betas # alpha_k = 1 - beta_k
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
# alpha_bar_0 = 1.0, alpha_bar_K = product of all alphas

# At index k, store alpha_bar_{k-1}.
(torch.ones(1, device=DEVICE), alpha_bars[:-1])

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *[1] * (len(x_shape) - 1))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bars), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.
    return xk, noise # Return x_k and the exact noise target.
```

Schedule and forward corruption

Compute the one-step signal retention

$$\alpha_k = 1 - \beta_k$$

Then, compute cumulative signal retention:

$$\bar{\alpha}_k = \prod_{j=1}^k \alpha_j$$

The cumulative signal retention $\bar{\alpha}_k$ equals the product of the one-step retentions α_j , with j running from the first step through the current step k .

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)
alphas = 1.0 - betas # alpha_k = 1 - beta_k
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
# alpha_bar_0 = 1.0, alpha_bar_K = product of all alphas

# At index k, store alpha_bar_{k-1}.
(torch.ones(1, device=DEVICE), alpha_bars[:-1])

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *[1] * (len(x_shape) - 1))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bars), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.
    return xk, noise # Return x_k and the exact noise target.
```

Schedule and forward corruption

Compute the one-step signal retention

$$\alpha_k = 1 - \beta_k$$

Then, compute cumulative signal retention:

$$\bar{\alpha}_k = \prod_{j=1}^k \alpha_j$$

Shift the cumulative schedule so index k stores $\bar{\alpha}_{k-1}$.

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)
alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.

alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
) # At index k, store alpha_bar_{k-1}.

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *[1] * (len(x_shape) - 1))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bars), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.
    return xk, noise # Return x_k and the exact noise target.
```

Schedule and forward corruption

Compute the reverse posterior variance

$$\tilde{\beta}_k = \frac{\beta_k(1 - \bar{\alpha}_{k-1})}{1 - \bar{\alpha}_k}$$

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
) # At index k, store alpha_bar_{k-1}.

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *(1] * (len(x_shape) - 1)))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bars), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.
    return xk, noise # Return x_k and the exact noise target.
```

Schedule and forward corruption

Compute the reverse posterior variance

$$\tilde{\beta}_k = \frac{\beta_k(1 - \bar{\alpha}_{k-1})}{1 - \bar{\alpha}_k}$$

The reverse variance $\tilde{\beta}_k$ equals step noise variance β_k times one minus previous cumulative retention $\bar{\alpha}_{k-1}$, divided by one minus current cumulative retention $\bar{\alpha}_k$.

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)
alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
) # At index k, store alpha_bar_{k-1}.

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *(1] * (len(x_shape) - 1)))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bars), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.
    return xk, noise # Return x_k and the exact noise target.
```

Schedule and forward corruption

Select the correct schedule value for each image in the batch and reshape it for broadcasting across image channels, height, and width.

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
) # At index k, store alpha_bar_{k-1}.

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *(1] * (len(x_shape) - 1)))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bars), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.
    return xk, noise # Return x_k and the exact noise target.
```

Schedule and forward corruption

Sample ε with the same shape as x_0 :

$$\varepsilon \sim \mathcal{N}(0, I)$$

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
) # At index k, store alpha_bar_{k-1}.

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *(1] * (len(x_shape) - 1)))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bars), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.
    return xk, noise # Return x_k and the exact noise target.
```

Schedule and forward corruption

Sample ε with the same shape as x_0 :

$$\varepsilon \sim \mathcal{N}(0, I)$$

The noise tensor ε is sampled from a Gaussian with zero mean and identity covariance I , giving independent unit-variance noise with the same shape as the clean image batch x_0 .

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)
alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
) # At index k, store alpha_bar_{k-1}.

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *[1] * (len(x_shape) - 1))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bars), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.
    return xk, noise # Return x_k and the exact noise target.
```

Schedule and forward corruption

Implement direct forward corruption:

$$X_k = \sqrt{\bar{\alpha}_k} X_0 + \sqrt{1 - \bar{\alpha}_k} \varepsilon.$$

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
) # At index k, store alpha_bar_{k-1}.

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *[1] * (len(x_shape) - 1))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bars), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.

    return xk, noise # Return x_k and the exact noise target.
```

Schedule and forward corruption

Implement direct forward corruption:

$$X_k = \sqrt{\bar{\alpha}_k} X_0 + \sqrt{1 - \bar{\alpha}_k} \epsilon.$$

The noisy image X_k equals the clean image X_0 scaled by the square root of cumulative retention $\bar{\alpha}_k$, plus standard Gaussian noise ϵ scaled by the square root of one minus that retention.

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)
alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
) # At index k, store alpha_bar_{k-1}.

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *[1] * (len(x_shape) - 1))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bars), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.

    return xk, noise # Return x_k and the exact noise target.
```

Schedule and forward corruption

Implement direct forward corruption:

$$X_k = \sqrt{\bar{\alpha}_k} X_0 + \sqrt{1 - \bar{\alpha}_k} \varepsilon.$$

This lets us jump directly to any noise level during training without simulating the earlier steps.

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)
alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
) # At index k, store alpha_bar_{k-1}.

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *[1] * (len(x_shape) - 1))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bars), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.

    return xk, noise # Return x_k and the exact noise target.
```

Schedule and forward corruption

Implement direct forward corruption:

$$X_k = \sqrt{\bar{\alpha}_k} X_0 + \sqrt{1 - \bar{\alpha}_k} \varepsilon.$$

And return both x_k and the exact ε used to construct it. The training loss needs both.

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
) # At index k, store alpha_bar_{k-1}.

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *[1] * (len(x_shape) - 1))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bars), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.
    return xk, noise # Return x_k and the exact noise target.
```

Schedule and forward corruption

The schedule determines how much signal and noise appear at every step, and `q_sample()` constructs x_k directly from x_0 .

```
import math
from pathlib import Path
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

# Configuration
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 2e-4
BASE_CHANNELS = 64
TIME_DIM = 128
OUTPUT_DIR = Path("ddpm_outputs")

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)

# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)
alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
) # At index k, store alpha_bar_{k-1}.

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, x_shape):
    # Select the schedule value for each example and add broadcast dimensions.
    return values[k].view(k.shape[0], *[1] * (len(x_shape) - 1))

def q_sample(x0, k, noise=None):
    if noise is None:
        noise = torch.randn_like(x0) # Sample epsilon ~ N(0, I).

    signal_scale = extract(torch.sqrt(alpha_bars), k, x0.shape) # sqrt(alpha_bar_k).
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, x0.shape) # sqrt(1-alpha_bar_k).
    xk = signal_scale * x0 + noise_scale * noise # Directly sample x_k from x_0.
    return xk, noise # Return x_k and the exact noise target.
```

Sinusoidal time embedding

```
class SinusoidalTimeEmbedding(nn.Module):
    def __init__(self, embedding_dim):
        super().__init__()
        if embedding_dim % 2 != 0:
            raise ValueError("embedding_dim must be even")
        self.embedding_dim = embedding_dim

    def forward(self, k):
        half_dim = self.embedding_dim // 2 # Use half sine and half cosine features.
        scale = math.log(10000.0) / max(half_dim - 1, 1) # Space frequencies geometrically.
        frequencies = torch.exp(
            -scale
            * torch.arange(half_dim, device=k.device, dtype=torch.float32)
        )
        angles = k.float().unsqueeze(1) * frequencies.unsqueeze(0) # One angle vector per k.
        return torch.cat([torch.sin(angles), torch.cos(angles)], dim=1)

def group_count(channels):
    for groups in (8, 4, 2, 1):
        if channels % groups == 0:
            return groups
    return 1
```

Sinusoidal time embedding

We will convert the scalar diffusion step k into a vector that the U-Net can use.

```
class SinusoidalTimeEmbedding(nn.Module):
    def __init__(self, embedding_dim):
        super().__init__()
        if embedding_dim % 2 != 0:
            raise ValueError("embedding_dim must be even")
        self.embedding_dim = embedding_dim

    def forward(self, k):
        half_dim = self.embedding_dim // 2 # Use half sine and half cosine features.
        scale = math.log(10000.0) / max(half_dim - 1, 1) # Space frequencies geometrically.
        frequencies = torch.exp(
            -scale
            * torch.arange(half_dim, device=k.device, dtype=torch.float32)
        )
        angles = k.float().unsqueeze(1) * frequencies.unsqueeze(0) # One angle vector per k.
        return torch.cat([torch.sin(angles), torch.cos(angles)], dim=1)

def group_count(channels):
    for groups in (8, 4, 2, 1):
        if channels % groups == 0:
            return groups
    return 1
```

Sinusoidal time embedding

Use half of the embedding dimensions for sine and half for cosine.

```
class SinusoidalTimeEmbedding(nn.Module):
    def __init__(self, embedding_dim):
        super().__init__()
        if embedding_dim % 2 != 0:
            raise ValueError("embedding_dim must be even")
        self.embedding_dim = embedding_dim

    def forward(self, k):
        half_dim = self.embedding_dim // 2 # Use half sine and half cosine features.
        scale = math.log(10000.0) / max(half_dim - 1, 1) # Space frequencies geometrically.
        frequencies = torch.exp(
            -scale
            * torch.arange(half_dim, device=k.device, dtype=torch.float32)
        )
        angles = k.float().unsqueeze(1) * frequencies.unsqueeze(0) # One angle vector per k.
        return torch.cat([torch.sin(angles), torch.cos(angles)], dim=1)

def group_count(channels):
    for groups in (8, 4, 2, 1):
        if channels % groups == 0:
            return groups
    return 1
```

Sinusoidal time embedding

Select the largest group count among 8, 4, 2, and 1 that divides the number of channels. This keeps every GroupNorm operation valid.

```
class SinusoidalTimeEmbedding(nn.Module):
    def __init__(self, embedding_dim):
        super().__init__()
        if embedding_dim % 2 != 0:
            raise ValueError("embedding_dim must be even")
        self.embedding_dim = embedding_dim

    def forward(self, k):
        half_dim = self.embedding_dim // 2 # Use half sine and half cosine features.
        scale = math.log(10000.0) / max(half_dim - 1, 1) # Space frequencies geometrically.
        frequencies = torch.exp(
            -scale
            * torch.arange(half_dim, device=k.device, dtype=torch.float32)
        )
        angles = k.float().unsqueeze(1) * frequencies.unsqueeze(0) # One angle vector per k.
        return torch.cat([torch.sin(angles), torch.cos(angles)], dim=1)

def group_count(channels):
    for groups in (8, 4, 2, 1):
        if channels % groups == 0:
            return groups
    return 1
```

Sinusoidal time embedding

So overall, low-frequency components distinguish widely separated noise levels, while high-frequency components distinguish nearby steps.

```
class SinusoidalTimeEmbedding(nn.Module):
    def __init__(self, embedding_dim):
        super().__init__()
        if embedding_dim % 2 != 0:
            raise ValueError("embedding_dim must be even")
        self.embedding_dim = embedding_dim

    def forward(self, k):
        half_dim = self.embedding_dim // 2 # Use half sine and half cosine features.
        scale = math.log(10000.0) / max(half_dim - 1, 1) # Space frequencies geometrically.
        frequencies = torch.exp(
            -scale
            * torch.arange(half_dim, device=k.device, dtype=torch.float32)
        )
        angles = k.float().unsqueeze(1) * frequencies.unsqueeze(0) # One angle vector per k.
        return torch.cat([torch.sin(angles), torch.cos(angles)], dim=1)

def group_count(channels):
    for groups in (8, 4, 2, 1):
        if channels % groups == 0:
            return groups
    return 1
```

Time-conditioned residual block

Next, we create the basic U-Net block that processes image features while incorporating the current diffusion step.

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(group_count(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.time_projection = nn.Linear(time_dim, out_channels)
        self.norm2 = nn.GroupNorm(group_count(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.skip = (
            nn.Identity()
            if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, 1)
        )

    def forward(self, x, time_embedding):
        residual = self.skip(x) # Match the residual channels when needed.
        h = self.conv1(F.silu(self.norm1(x))) # Normalize, activate, then convolve.
        h = h + self.time_projection(time_embedding).unsqueeze(-1).unsqueeze(-1)
        h = self.conv2(F.silu(self.norm2(h))) # Refine the time-conditioned features.
        return h + residual # Preserve information through the residual connection.
```

Time-conditioned residual block

Normalize the incoming feature channels with GroupNorm.

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(group_count(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.time_projection = nn.Linear(time_dim, out_channels)
        self.norm2 = nn.GroupNorm(group_count(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.skip = (
            nn.Identity()
            if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, 1)
        )

    def forward(self, x, time_embedding):
        residual = self.skip(x) # Match the residual channels when needed.
        h = self.conv1(F.silu(self.norm1(x))) # Normalize, activate, then convolve.
        h = h + self.time_projection(time_embedding).unsqueeze(-1).unsqueeze(-1)
        h = self.conv2(F.silu(self.norm2(h))) # Refine the time-conditioned features.
        return h + residual # Preserve information through the residual connection.
```

Time-conditioned residual block

Map `in_channels` to `out_channels` using a 3×3 convolution.

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(group_count(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.time_projection = nn.Linear(time_dim, out_channels)
        self.norm2 = nn.GroupNorm(group_count(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.skip = (
            nn.Identity()
            if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, 1)
        )

    def forward(self, x, time_embedding):
        residual = self.skip(x) # Match the residual channels when needed.
        h = self.conv1(F.silu(self.norm1(x))) # Normalize, activate, then convolve.
        h = h + self.time_projection(time_embedding).unsqueeze(-1).unsqueeze(-1)
        h = self.conv2(F.silu(self.norm2(h))) # Refine the time-conditioned features.
        return h + residual # Preserve information through the residual connection.
```

Time-conditioned residual block

Map the time embedding e_k into one value for each output channel:

$$W_t e_k \in \mathbb{R}^{C_{\text{out}}}$$

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(group_count(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.time_projection = nn.Linear(time_dim, out_channels)
        self.norm2 = nn.GroupNorm(group_count(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.skip = (
            nn.Identity()
            if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, 1)
        )

    def forward(self, x, time_embedding):
        residual = self.skip(x) # Match the residual channels when needed.
        h = self.conv1(F.silu(self.norm1(x))) # Normalize, activate, then convolve.
        h = h + self.time_projection(time_embedding).unsqueeze(-1).unsqueeze(-1)
        h = self.conv2(F.silu(self.norm2(h))) # Refine the time-conditioned features.
        return h + residual # Preserve information through the residual connection.
```

Time-conditioned residual block

Map the time embedding e_k into one value for each output channel:

$$W_t e_k \in \mathbb{R}^{C_{\text{out}}}$$

Multiply the diffusion-step embedding e_k by the learned projection W_t to obtain one real value for each of the C_{out} output channels.

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(group_count(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.time_projection = nn.Linear(time_dim, out_channels)
        self.norm2 = nn.GroupNorm(group_count(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.skip = (
            nn.Identity()
            if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, 1)
        )

    def forward(self, x, time_embedding):
        residual = self.skip(x) # Match the residual channels when needed.
        h = self.conv1(F.silu(self.norm1(x))) # Normalize, activate, then convolve.
        h = h + self.time_projection(time_embedding).unsqueeze(-1).unsqueeze(-1)
        h = self.conv2(F.silu(self.norm2(h))) # Refine the time-conditioned features.
        return h + residual # Preserve information through the residual connection.
```

Time-conditioned residual block

Apply a second normalization, activation, and convolution.

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(group_count(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.time_projection = nn.Linear(time_dim, out_channels)
        self.norm2 = nn.GroupNorm(group_count(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.skip = (
            nn.Identity()
            if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, 1)
        )

    def forward(self, x, time_embedding):
        residual = self.skip(x) # Match the residual channels when needed.
        h = self.conv1(F.silu(self.norm1(x))) # Normalize, activate, then convolve.
        h = h + self.time_projection(time_embedding).unsqueeze(-1).unsqueeze(-1)
        h = self.conv2(F.silu(self.norm2(h))) # Refine the time-conditioned features.
        return h + residual # Preserve information through the residual connection.
```

Time-conditioned residual block

Use the identity when the channel counts match. A 1×1 convolution aligns the channels when their counts differ.

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(group_count(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.time_projection = nn.Linear(time_dim, out_channels)
        self.norm2 = nn.GroupNorm(group_count(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.skip = (
            nn.Identity()
            if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, 1)
        )

    def forward(self, x, time_embedding):
        residual = self.skip(x) # Match the residual channels when needed.
        h = self.conv1(F.silu(self.norm1(x))) # Normalize, activate, then convolve.
        h = h + self.time_projection(time_embedding).unsqueeze(-1).unsqueeze(-1)
        h = self.conv2(F.silu(self.norm2(h))) # Refine the time-conditioned features.
        return h + residual # Preserve information through the residual connection.
```

Time-conditioned residual block

Preserve a transformed copy of the original features.

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(group_count(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.time_projection = nn.Linear(time_dim, out_channels)
        self.norm2 = nn.GroupNorm(group_count(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.skip = (
            nn.Identity()
            if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, 1)
        )

    def forward(self, x, time_embedding):
        residual = self.skip(x) # Match the residual channels when needed.
        h = self.conv1(F.silu(self.norm1(x))) # Normalize, activate, then convolve.
        h = h + self.time_projection(time_embedding).unsqueeze(-1).unsqueeze(-1)
        h = self.conv2(F.silu(self.norm2(h))) # Refine the time-conditioned features.
        return h + residual # Preserve information through the residual connection.
```

Time-conditioned residual block

Compute the first feature transformation:

$$h = \text{Conv}_1(\text{SiLU}(\text{GN}(x)))$$

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(group_count(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.time_projection = nn.Linear(time_dim, out_channels)
        self.norm2 = nn.GroupNorm(group_count(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.skip = (
            nn.Identity()
            if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, 1)
        )

    def forward(self, x, time_embedding):
        residual = self.skip(x) # Match the residual channels when needed.
        h = self.conv1(F.silu(self.norm1(x))) # Normalize, activate, then convolve.
        h = h + self.time_projection(time_embedding).unsqueeze(-1).unsqueeze(-1)
        h = self.conv2(F.silu(self.norm2(h))) # Refine the time-conditioned features.
        return h + residual # Preserve information through the residual connection.
```

Time-conditioned residual block

Compute the first feature transformation:

$$h = \text{Conv}_1(\text{SiLU}(\text{GN}(x)))$$

To obtain features h from input features x , apply group normalization GN, the SiLU activation, and the first convolution Conv_1 , in that order.

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(group_count(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.time_projection = nn.Linear(time_dim, out_channels)
        self.norm2 = nn.GroupNorm(group_count(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.skip = (
            nn.Identity()
            if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, 1)
        )

    def forward(self, x, time_embedding):
        residual = self.skip(x) # Match the residual channels when needed.
        h = self.conv1(F.silu(self.norm1(x))) # Normalize, activate, then convolve.
        h = h + self.time_projection(time_embedding).unsqueeze(-1).unsqueeze(-1)
        h = self.conv2(F.silu(self.norm2(h))) # Refine the time-conditioned features.
        return h + residual # Preserve information through the residual connection.
```

Time-conditioned residual block

Add the time information to every spatial location:

$$h \leftarrow h + W_t e_k$$

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(group_count(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.time_projection = nn.Linear(time_dim, out_channels)
        self.norm2 = nn.GroupNorm(group_count(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.skip = (
            nn.Identity()
            if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, 1)
        )

    def forward(self, x, time_embedding):
        residual = self.skip(x) # Match the residual channels when needed.
        h = self.conv1(F.silu(self.norm1(x))) # Normalize, activate, then convolve
        h = h + self.time_projection(time_embedding).unsqueeze(-1).unsqueeze(-1)
        h = self.conv2(F.silu(self.norm2(h))) # Refine the time-conditioned features.
        return h + residual # Preserve information through the residual connection.
```

Time-conditioned residual block

Add the time information to every spatial location:

$$h \leftarrow h + W_t e_k$$

Add the learned projection W_t of step embedding e_k to features h at every spatial location, so each channel receives an offset for the current noise step.

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(group_count(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.time_projection = nn.Linear(time_dim, out_channels)
        self.norm2 = nn.GroupNorm(group_count(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.skip = (
            nn.Identity()
            if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, 1)
        )

    def forward(self, x, time_embedding):
        residual = self.skip(x) # Match the residual channels when needed.
        h = self.conv1(F.silu(self.norm1(x))) # Normalize, activate, then convolve
        h = h + self.time_projection(time_embedding).unsqueeze(-1).unsqueeze(-1)
        h = self.conv2(F.silu(self.norm2(h))) # Refine the time-conditioned features.
        return h + residual # Preserve information through the residual connection.
```

Time-conditioned residual block

Add the time information to every spatial location:

$$h \leftarrow h + W_t e_k$$

The same feature pattern can require a different denoising update at different noise levels.

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(group_count(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.time_projection = nn.Linear(time_dim, out_channels)
        self.norm2 = nn.GroupNorm(group_count(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.skip = (
            nn.Identity()
            if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, 1)
        )

    def forward(self, x, time_embedding):
        residual = self.skip(x) # Match the residual channels when needed.
        h = self.conv1(F.silu(self.norm1(x))) # Normalize, activate, then convolve
        h = h + self.time_projection(time_embedding).unsqueeze(-1).unsqueeze(-1)
        h = self.conv2(F.silu(self.norm2(h))) # Refine the time-conditioned features.
        return h + residual # Preserve information through the residual connection.
```

Time-conditioned residual block

Add the residual connection:

$$\text{Block}(x, e_k) = h + \text{skip}(x)$$

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(group_count(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.time_projection = nn.Linear(time_dim, out_channels)
        self.norm2 = nn.GroupNorm(group_count(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.skip = (
            nn.Identity()
            if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, 1)
        )

    def forward(self, x, time_embedding):
        residual = self.skip(x) # Match the residual channels when needed.
        h = self.conv1(F.silu(self.norm1(x))) # Normalize, activate, then convolve.
        h = h + self.time_projection(time_embedding).unsqueeze(-1).unsqueeze(-1)
        h = self.conv2(F.silu(self.norm2(h))) # Refine the time-conditioned features.
        return h + residual # Preserve information through the residual connection.
```

Time-conditioned residual block

Add the residual connection:

$$\text{Block}(x, e_k) = h + \text{skip}(x)$$

Given input features x and step embedding e_k , the block adds its transformed features h to $\text{skip}(x)$, the input preserved directly or projected to matching channels.

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(group_count(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.time_projection = nn.Linear(time_dim, out_channels)
        self.norm2 = nn.GroupNorm(group_count(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.skip = (
            nn.Identity()
            if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, 1)
        )

    def forward(self, x, time_embedding):
        residual = self.skip(x) # Match the residual channels when needed.
        h = self.conv1(F.silu(self.norm1(x))) # Normalize, activate, then convolve.
        h = h + self.time_projection(time_embedding).unsqueeze(-1).unsqueeze(-1)
        h = self.conv2(F.silu(self.norm2(h))) # Refine the time-conditioned features.
        return h + residual # Preserve information through the residual connection.
```

Time-conditioned residual block

The convolution processes image structure. We add the projected time embedding so the block can use the current noise level.

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, time_dim):
        super().__init__()
        self.norm1 = nn.GroupNorm(group_count(in_channels), in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.time_projection = nn.Linear(time_dim, out_channels)
        self.norm2 = nn.GroupNorm(group_count(out_channels), out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
        self.skip = (
            nn.Identity()
            if in_channels == out_channels
            else nn.Conv2d(in_channels, out_channels, 1)
        )

    def forward(self, x, time_embedding):
        residual = self.skip(x) # Match the residual channels when needed.
        h = self.conv1(F.silu(self.norm1(x))) # Normalize, activate, then convolve.
        h = h + self.time_projection(time_embedding).unsqueeze(-1).unsqueeze(-1)
        h = self.conv2(F.silu(self.norm2(h))) # Refine the time-conditioned features.
        return h + residual # Preserve information through the residual connection.
```

Now for the DDPM U-Net

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Now for the DDPM U-Net

Combine time-conditioned residual blocks into an encoder, bottleneck, and decoder with skip connections.

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Now for the DDPM U-Net

Transform the sinusoidal embedding into a learned time representation.

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Now for the DDPM U-Net

Process full-resolution features and reduce 28×28 to 14×14 .

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Now for the DDPM U-Net

Increase the channels to 128 and reduce 14 x 14 to 7 x 7.

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Now for the DDPM U-Net

Process the lowest-resolution representation using 256 channels.

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Now for the DDPM U-Net

Increase 7 x 7 back to 14 x 14 and combine the result with the second encoder skip.

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Now for the DDPM U-Net

Increase 14 x 14 back to 28 x 28 and combine the result with the first encoder skip.

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Now for the DDPM U-Net

Map the final 64 feature channels back to one image channel.

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Forward Pass

Compute the learned representation of diffusion step k .

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Forward Pass

Store the 28 x 28 encoder features.

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Forward Pass

Store the 28 x 28 encoder features and reduce the spatial resolution:

$$28 \times 28 \longrightarrow 14 \times 14$$

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Forward Pass

Store the 14 x 14 encoder features and reduce the spatial resolution again:

$$14 \times 14 \longrightarrow 7 \times 7$$

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Forward Pass

Process the compressed representation.

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Forward Pass

Concatenate decoder features with the matching encoder features.

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Forward Pass

Produce the predicted noise image:

$$\varepsilon_{\theta}(x_k, k).$$

Its shape matches x_k :

$$B \times 1 \times 28 \times 28$$

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Forward Pass

Produce the predicted noise image:

$$\varepsilon_{\theta}(x_k, k).$$

Its shape matches x_k :

$$B \times 1 \times 28 \times 28$$

The noise prediction $\varepsilon_{\theta}(x_k, k)$, from a U-Net with parameters θ and noisy input x_k at step k , contains B images, each with one channel and 28×28 pixels.

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

Forward Pass

Produce the predicted noise image:

$$\varepsilon_{\theta}(x_k, k).$$

Its shape matches x_k :

$$B \times 1 \times 28 \times 28$$

$$28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7 \rightarrow 14 \times 14 \rightarrow 28 \times 28$$

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k)  # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk)  # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding)  # Full-resolution encoder features.
        x = self.downsample1(skip1)  # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding)  # Half-resolution encoder features.
        x = self.downsample2(skip2)  # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding)  # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x)  # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding)  # Add the first skip.

        x = self.upsample1(x)  # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding)  # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x)))  # Predict epsilon with image shape.
```

Forward Pass

Produce the predicted noise image:

$$\varepsilon_{\theta}(x_k, k).$$

Its shape matches x_k :

$$B \times 1 \times 28 \times 28$$

$$28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7 \rightarrow 14 \times 14 \rightarrow 28 \times 28$$

The encoder learns broad image structure, the decoder restores spatial detail, and the skip connections preserve information that would otherwise be lost during downsampling.

```
class DDPMUNet(nn.Module):
    def __init__(self, in_channels=1, base_channels=64, time_dim=128):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            nn.Linear(time_dim, time_dim * 4),
            nn.SiLU(),
            nn.Linear(time_dim * 4, time_dim),
        )

        self.input_conv = nn.Conv2d(in_channels, base_channels, 3, padding=1)

        self.down1 = ResidualBlock(base_channels, base_channels, time_dim)
        self.downsample1 = nn.Conv2d(base_channels, base_channels, 4, 2, 1)

        self.down2 = ResidualBlock(base_channels, base_channels * 2, time_dim)
        self.downsample2 = nn.Conv2d(base_channels * 2, base_channels * 2, 4, 2, 1)

        self.middle1 = ResidualBlock(base_channels * 2, base_channels * 4, time_dim)
        self.middle2 = ResidualBlock(base_channels * 4, base_channels * 4, time_dim)

        self.upsample2 = nn.ConvTranspose2d(base_channels * 4, base_channels * 2, 4, 2, 1)
        self.up2 = ResidualBlock(base_channels * 4, base_channels * 2, time_dim)

        self.upsample1 = nn.ConvTranspose2d(base_channels * 2, base_channels, 4, 2, 1)
        self.up1 = ResidualBlock(base_channels * 2, base_channels, time_dim)

        self.output_norm = nn.GroupNorm(group_count(base_channels), base_channels)
        self.output_conv = nn.Conv2d(base_channels, in_channels, 3, padding=1)

    def forward(self, xk, k):
        time_embedding = self.time_mlp(k) # Convert each discrete step k to a learned vector.

        x = self.input_conv(xk) # Lift the noisy image into feature channels.

        skip1 = self.down1(x, time_embedding) # Full-resolution encoder features.
        x = self.downsample1(skip1) # 28 x 28 becomes 14 x 14.

        skip2 = self.down2(x, time_embedding) # Half-resolution encoder features.
        x = self.downsample2(skip2) # 14 x 14 becomes 7 x 7.

        x = self.middle1(x, time_embedding) # Process the lowest-resolution representation.
        x = self.middle2(x, time_embedding)

        x = self.upsample2(x) # 7 x 7 becomes 14 x 14.
        x = self.up2(torch.cat([x, skip2], dim=1), time_embedding) # Add the first skip.

        x = self.upsample1(x) # 14 x 14 becomes 28 x 28.
        x = self.up1(torch.cat([x, skip1], dim=1), time_embedding) # Add the second skip.

        return self.output_conv(F.silu(self.output_norm(x))) # Predict epsilon with image shape.
```

One reverse step and the complete sampler

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation.
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

One reverse step and the complete sampler

Purpose: Use $\epsilon_{\theta}(x_k, k)$ to sample x_{k-1} , then repeat from $k = K$ through $k = 1$.

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation.
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

One reverse step and the complete sampler

Disable gradient storage because sampling does not update θ .

```
@torch.no_grad()
```

```
def p_sample(model, xk, k):  
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).  
  
    beta_k = extract(betas, k, xk.shape)  
    alpha_k = extract(alphas, k, xk.shape)  
    alpha_bar_k = extract(alpha_bars, k, xk.shape)  
  
    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).  
        xk  
        - beta_k  
        * predicted_noise  
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)  
    )  
  
    posterior_variance_k = extract(posterior_variances, k, xk.shape)  
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).  
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.  
  
    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise
```

```
@torch.no_grad()
```

```
def sample_images(model, num_samples=16):  
    model.eval() # Disable training-time behavior during generation.  
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.  
  
    for step in range(K, 0, -1):  
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)  
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.  
  
    return xk.clamp(-1.0, 1.0)
```

One reverse step and the complete sampler

Computes

$$\varepsilon_{\theta}(x_k, k)$$

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation.
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

One reverse step and the complete sampler

Computes

$$\varepsilon_{\theta}(x_k, k)$$

Given a noisy image x_k at step k , the trained U-Net with parameters θ predicts its added noise $\varepsilon_{\theta}(x_k, k)$.

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation.
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

One reverse step and the complete sampler

Select β_k , α_k , and $\bar{\alpha}_k$ for every image in the batch.

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation.
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

One reverse step and the complete sampler

Implement the DDPM reverse mean:

$$\mu_{\theta}(x_k, k) = \frac{1}{\sqrt{\alpha_k}} \left[x_k - \frac{\beta_k}{\sqrt{1 - \bar{\alpha}_k}} \varepsilon_{\theta}(x_k, k) \right].$$

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation.
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

One reverse step and the complete sampler

Implement the DDPM reverse mean:

$$\mu_{\theta}(x_k, k) = \frac{1}{\sqrt{\alpha_k}} \left[x_k - \frac{\beta_k}{\sqrt{1 - \bar{\alpha}_k}} \varepsilon_{\theta}(x_k, k) \right].$$

The reverse mean μ_{θ} equals noisy image x_k minus predicted noise ε_{θ} weighted by step variance β_k over the square root of one minus cumulative retention $\bar{\alpha}_k$, then divided by the square root of one-step retention α_k .

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation.
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

One reverse step and the complete sampler

Select $\tilde{\beta}_k$, the reverse variance at step k .

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, 1).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation.
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

One reverse step and the complete sampler

Sample the fresh Gaussian vector Z_k .

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation.
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

One reverse step and the complete sampler

Add stochastic noise for $k > 1$ and use no additional noise at the final step $k = 1$.

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation.
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

One reverse step and the complete sampler

Samples

$$X_{k-1} = \mu_{\theta}(X_k, k) + \sqrt{\tilde{\beta}_k} Z_k$$

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation.
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

One reverse step and the complete sampler

Samples

$$X_{k-1} = \mu_{\theta}(X_k, k) + \sqrt{\tilde{\beta}_k} Z_k$$

The preceding image X_{k-1} equals the predicted reverse mean $\mu_{\theta}(X_k, k)$ for current image X_k at step k , plus fresh standard Gaussian noise Z_k scaled by the square root of reverse variance $\tilde{\beta}_k$.

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation.
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

Complete sampling loop

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise
```

```
@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation.
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

Complete sampling loop

Start with standard Gaussian noise:

$$X_K \sim \mathcal{N}(0, I)$$

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

Complete sampling loop

Start with standard Gaussian noise:

$$X_K \sim \mathcal{N}(0, I)$$

The initial sampling state X_K , at the final noise index K , has zero mean and identity covariance I , so each image coordinate starts as independent unit-variance Gaussian noise.

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

Complete sampling loop

Run the reverse chain:

$$X_K \rightarrow X_{K-1} \rightarrow \dots \rightarrow X_1 \rightarrow X_0$$

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation.
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

Complete sampling loop

Run the reverse chain:

$$X_K \rightarrow X_{K-1} \rightarrow \dots \rightarrow X_1 \rightarrow X_0$$

Starting from the Gaussian image X_K at noise index K , sample each preceding state until reaching the generated clean image X_0 .

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation.
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

Complete sampling loop

Return the generated images in the same normalized range used during training.

```
@torch.no_grad()
def p_sample(model, xk, k):
    predicted_noise = model(xk, k) # Estimate epsilon_theta(x_k, k).

    beta_k = extract(betas, k, xk.shape)
    alpha_k = extract(alphas, k, xk.shape)
    alpha_bar_k = extract(alpha_bars, k, xk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute the DDPM mean mu_theta(x_k, k).
        xk
        - beta_k
        * predicted_noise
        / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, xk.shape)
    reverse_noise = torch.randn_like(xk) # Sample z_k ~ N(0, I).
    add_noise = (k > 1).float().view(k.shape[0], 1, 1, 1) # Use no noise at k = 1.

    return reverse_mean + add_noise * torch.sqrt(posterior_variance_k) * reverse_noise

@torch.no_grad()
def sample_images(model, num_samples=16):
    model.eval() # Disable training-time behavior during generation.
    xk = torch.randn(num_samples, 1, 28, 28, device=DEVICE) # Start from x_K.

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        xk = p_sample(model, xk, k) # Sample x_{k-1} from x_k.

    return xk.clamp(-1.0, 1.0)
```

Let's train the image denoiser on MNIST.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

        epoch_loss += loss.item()

    print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Make initialization and random sampling reproducible.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

        epoch_loss += loss.item()

    print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Convert each MNIST image into a PyTorch tensor.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

        epoch_loss += loss.item()

    print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Map pixel intensities from $[0,1]$ to approximately $[-1,1]$:

$$x_{\text{normalized}} = \frac{x - 0.5}{0.5} = 2x - 1$$

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

        epoch_loss += loss.item()

    print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Map pixel intensities from [0,1] to approximately [-1,1]:

$$x_{\text{normalized}} = \frac{x - 0.5}{0.5} = 2x - 1$$

The normalized pixel $x_{\text{normalized}}$ equals the original pixel x minus 0.5, divided by 0.5, mapping pixel values from zero to one into minus one to one.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

        epoch_loss += loss.item()

    print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Download and load the clean training images x0.

Create shuffled mini-batches of 128 images.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

        epoch_loss += loss.item()

    print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Instantiate $\varepsilon_{\theta}(x_k, k)$ with:

one image channel;

64 base feature channels;

a 128-dimensional time embedding.

And move the model parameters θ to the selected hardware.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)

    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

        epoch_loss += loss.item()

    print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Create the optimizer that updates θ using the noise-prediction loss.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)

    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

        epoch_loss += loss.item()

    print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Repeat training over the dataset for 20 epochs.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

        epoch_loss += loss.item()

    print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Load a mini-batch of clean images x_0 .
The digit labels are ignored because this is unconditional generation.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

            epoch_loss += loss.item()

        print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Sample one diffusion step for every image:

$$k \sim \text{Uniform}\{1, \dots, K\}$$

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)

            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

            epoch_loss += loss.item()

        print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Sample one diffusion step for every image:

$$k \sim \text{Uniform}\{1, \dots, K\}$$

Sample the noise index k uniformly from one through the total number of diffusion steps K , giving every noise level equal probability during training.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)

            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

            epoch_loss += loss.item()

        print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Construct

$$X_k = \sqrt{\bar{\alpha}_k} X_0 + \sqrt{1 - \bar{\alpha}_k} \varepsilon$$

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

            epoch_loss += loss.item()

        print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Construct

$$X_k = \sqrt{\bar{\alpha}_k} X_0 + \sqrt{1 - \bar{\alpha}_k} \varepsilon$$

To construct noisy training image X_k , scale clean image X_0 by the square root of cumulative retention $\bar{\alpha}_k$ and add standard Gaussian noise ε scaled by the square root of one minus that retention.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

            epoch_loss += loss.item()

        print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Compute $\varepsilon_{\theta}(X_k, k)$.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = g.sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

            epoch_loss += loss.item()

        print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Use the noise-prediction objective:

$$\mathcal{L}(\theta) = \mathbb{E} \left[\|\varepsilon - \varepsilon_{\theta}(X_k, k)\|^2 \right].$$

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

            epoch_loss += loss.item()

        print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Use the noise-prediction objective:

$$\mathcal{L}(\theta) = \mathbb{E} \left[\|\varepsilon - \varepsilon_{\theta}(X_k, k)\|^2 \right].$$

The loss $\mathcal{L}(\theta)$ averages squared differences between sampled noise ε and predicted noise $\varepsilon_{\theta}(X_k, k)$, over noisy images X_k , diffusion steps k , and noise draws, while training network parameters θ .

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

            epoch_loss += loss.item()

        print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Use the noise-prediction objective:

$$\mathcal{L}(\theta) = \mathbb{E} \left[\|\varepsilon - \varepsilon_{\theta}(X_k, k)\|^2 \right].$$

We know the training target because we sampled the noise ourselves.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

        epoch_loss += loss.item()

    print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Clear gradients from the previous mini-batch.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

            epoch_loss += loss.item()

        print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Compute the gradient and update θ to reduce the noise-prediction error.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

            epoch_loss += loss.item()

        print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Let's train the image denoiser on MNIST.

Save the trained U-Net parameters.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

        epoch_loss += loss.item()

    print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Generate 16 images with the trained model.

Start from 16 Gaussian noise images and reverse all K diffusion steps.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

            epoch_loss += loss.item()

        print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Generate 16 images with the trained model.

Map generated values from $[-1,1]$ back to $[0,1]$ and save a 4 x 4 image grid.

```
def main():
    torch.manual_seed(7)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.5,), (0.5,)),
        ]
    )

    dataset = datasets.MNIST("data", train=True, download=True, transform=transform)
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    model = DDPMUNet(
        in_channels=1,
        base_channels=BASE_CHANNELS,
        time_dim=TIME_DIM,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0

        for x0, _ in loader:
            x0 = x0.to(DEVICE)
            k = torch.randint(1, K + 1, (x0.shape[0],), device=DEVICE) # One k per image.

            xk, true_noise = q_sample(x0, k) # Corrupt x_0 directly at the selected k.
            predicted_noise = model(xk, k) # Predict the sampled Gaussian noise.
            loss = F.mse_loss(predicted_noise, true_noise) # Noise-prediction objective.

            optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
            loss.backward() # Differentiate the MSE through the U-Net.
            optimizer.step() # Update theta.

            epoch_loss += loss.item()

        print(f"epoch={epoch:03d} noise_mse={epoch_loss / len(loader):.6f}")

    torch.save(model.state_dict(), OUTPUT_DIR / "ddpm_unet_mnist.pt")

    samples = sample_images(model, num_samples=16)
    save_image((samples + 1.0) / 2.0, OUTPUT_DIR / "samples.png", nrow=4)
```

Now, return to AMP-Diffusion for the full implementation

Now, return to AMP-Diffusion for the full implementation

Now, connect the denoiser to the full training and sampling loop.

Now, return to AMP-Diffusion for the full implementation

Now, connect the denoiser to the full training and sampling loop.

Start with the embeddings that enter the diffusion model.

First, recall how BERT produces an embedding at each position

Use the output of the masked word's position to predict the masked word

Possible classes:
All English words

0.1%	Aardvark
...	...
10%	Improvisation
...	...
0%	Zyzyva

FFNN + Softmax



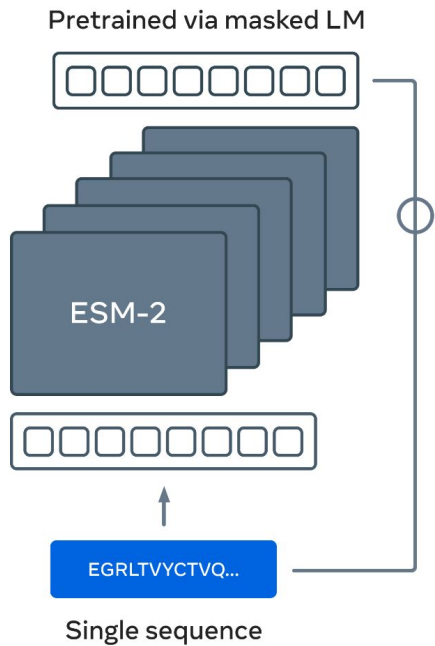
Randomly mask
15% of tokens

1 2 3 4 5 6 7 8 ... 512
[CLS] Let's stick to [MASK] in this skit

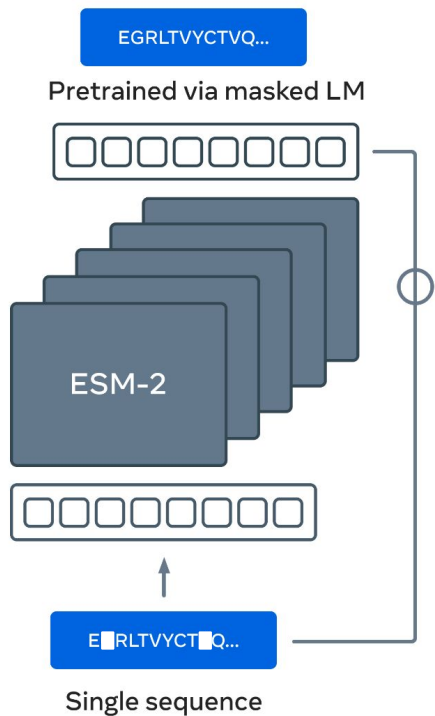
Input

[CLS] Let's stick to improvisation in this skit

The classic BERT pLM: ESM-2



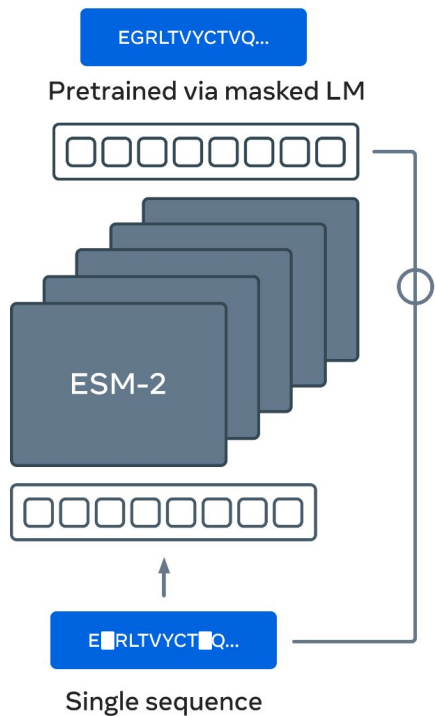
The classic BERT pLM: ESM



- Trained on ~70 million sequences to fill in the “masked”/hidden amino acids

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log P(x_i | x_{\setminus \mathcal{M}})$$

The classic BERT pLM: ESM

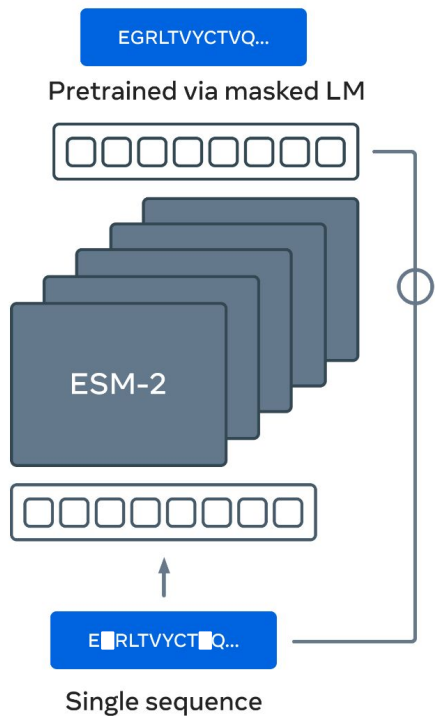


- Trained on ~70 million sequences to fill in the “masked”/hidden amino acids

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log P(x_i | x_{\setminus \mathcal{M}})$$

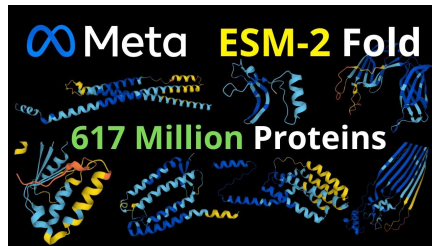
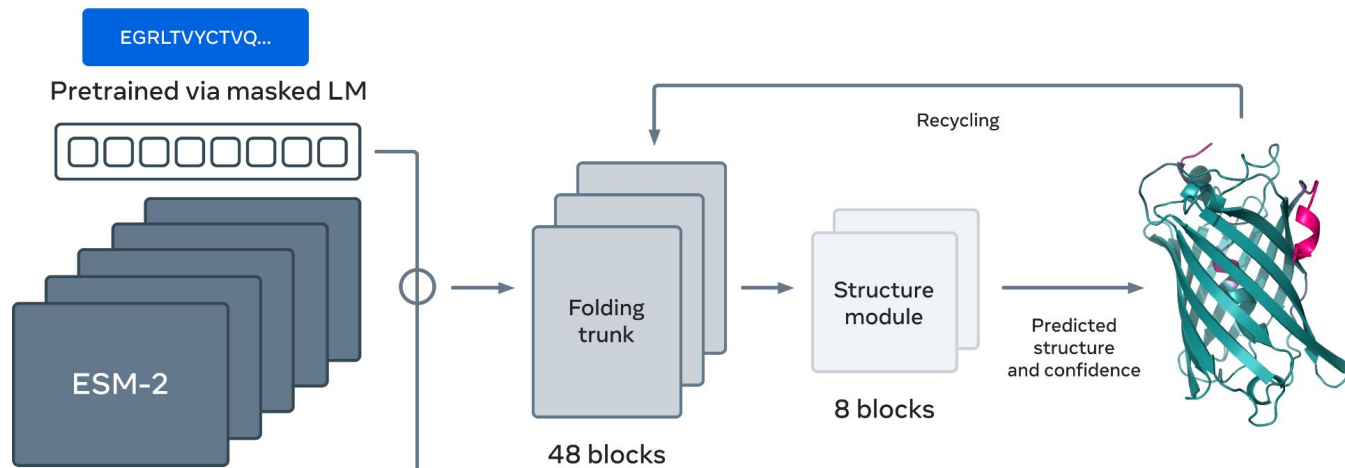
The masked-language loss $\mathcal{L}_{\text{MLM}}$ sums negative log probabilities P of correct amino acids x_i at masked positions i in $\mathcal{M}$, given the unmasked sequence $x_{\setminus \mathcal{M}}$.

The classic BERT pLM: ESM



- Trained on ~70 million sequences to fill in the “masked”/hidden amino acids
- By doing this simple task well, pLMs learn patterns governing structure, function, interaction, biophysical, and chemical properties of input sequences!

Enough to predict protein structure + other properties



So AMP-Diffusion changes four things:

1. Images x_0 become ESM-2 latents Z_0 .
2. A Transformer replaces the U-Net.
3. The network predicts $\hat{Z}_0$.
4. The ESM-2 language-model head converts generated latents into amino acids.

Setup is pretty much the same

The setup mirrors the image DDPM, with protein sequences, ESM-2, and latent-space dimensions replacing the image-specific settings.

```
import math
from pathlib import Path

import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

# Configuration
SEQUENCE_FILE = Path("amp_sequences.txt")
ESM2_NAME = "facebook/esm2_t6_8M_UR50D"
MAX_LENGTH = 64
GENERATED_LENGTH = 30
K = 1000
BETA_START = 1e-4
BETA_END = 2e-2
BATCH_SIZE = 32
ENCODING_BATCH_SIZE = 32
EPOCHS = 50
LEARNING_RATE = 1e-4
NUM_LAYERS = 6
NUM_HEADS = 8
NUM_SAMPLES = 8
AMINO_ACIDS = "ACDEFGHIKLMNPQRSTVWY"

DEVICE = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)
```

Forward corruption

The DDPM noise schedule remains unchanged, but forward corruption now adds Gaussian noise to each valid ESM-2 latent position while masking the padded positions.

```
# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
)

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, z_shape):
    # Select one schedule value per sequence and add dimensions for broadcasting.
    return values[k].view(k.shape[0], *[1] * (len(z_shape) - 1))

def q_sample(z0, k, attention_mask, noise=None):
    if noise is None:
        noise = torch.randn_like(z0) # Sample epsilon with the same shape as Z_0.

    signal_scale = extract(torch.sqrt(alpha_bars), k, z0.shape)
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, z0.shape)
    mask = attention_mask.unsqueeze(-1).to(z0.dtype) # Exclude padded token positions.

    zk = signal_scale * z0 + noise_scale * noise # Directly sample Z_k from Z_0.
    return zk * mask # Keep padded positions equal to zero.
```

Forward corruption

Computes the one-step retention

$$\alpha_k = 1 - \beta_k.$$

```
# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
)

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, z_shape):
    # Select one schedule value per sequence and add dimensions for broadcasting.
    return values[k].view(k.shape[0], *[1] * (len(z_shape) - 1))

def q_sample(z0, k, attention_mask, noise=None):
    if noise is None:
        noise = torch.randn_like(z0) # Sample epsilon with the same shape as Z_0.

    signal_scale = extract(torch.sqrt(alpha_bars), k, z0.shape)
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, z0.shape)
    mask = attention_mask.unsqueeze(-1).to(z0.dtype) # Exclude padded token positions.

    zk = signal_scale * z0 + noise_scale * noise # Directly sample Z_k from Z_0.
    return zk * mask # Keep padded positions equal to zero.
```

Forward corruption

Computes the one-step retention

$$\alpha_k = 1 - \beta_k.$$

At diffusion step k , the one-step signal retention α_k equals one minus the added Gaussian noise variance β_k , using the same schedule for the latent coordinates.

```
# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
)

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, z_shape):
    # Select one schedule value per sequence and add dimensions for broadcasting.
    return values[k].view(k.shape[0], *[1] * (len(z_shape) - 1))

def q_sample(z0, k, attention_mask, noise=None):
    if noise is None:
        noise = torch.randn_like(z0) # Sample epsilon with the same shape as Z_0.

    signal_scale = extract(torch.sqrt(alpha_bars), k, z0.shape)
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, z0.shape)
    mask = attention_mask.unsqueeze(-1).to(z0.dtype) # Exclude padded token positions.

    zk = signal_scale * z0 + noise_scale * noise # Directly sample Z_k from Z_0.
    return zk * mask # Keep padded positions equal to zero.
```

Forward corruption

Computes cumulative retention $\bar{\alpha}_k$

```
# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
)

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, z_shape):
    # Select one schedule value per sequence and add dimensions for broadcasting.
    return values[k].view(k.shape[0], *([1] * (len(z_shape) - 1)))

def q_sample(z0, k, attention_mask, noise=None):
    if noise is None:
        noise = torch.randn_like(z0) # Sample epsilon with the same shape as Z_0.

    signal_scale = extract(torch.sqrt(alpha_bars), k, z0.shape)
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, z0.shape)
    mask = attention_mask.unsqueeze(-1).to(z0.dtype) # Exclude padded token positions.

    zk = signal_scale * z0 + noise_scale * noise # Directly sample Z_k from Z_0.
    return zk * mask # Keep padded positions equal to zero.
```

Forward corruption

Computes cumulative retention $\bar{\alpha}_k$

The cumulative signal retention $\bar{\alpha}_k$ equals the product of the one-step retention factors from step one through step k , determining the clean signal remaining in the noisy latent.

```
# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
)

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, z_shape):
    # Select one schedule value per sequence and add dimensions for broadcasting.
    return values[k].view(k.shape[0], *[1] * (len(z_shape) - 1))

def q_sample(z0, k, attention_mask, noise=None):
    if noise is None:
        noise = torch.randn_like(z0) # Sample epsilon with the same shape as Z_0.

    signal_scale = extract(torch.sqrt(alpha_bars), k, z0.shape)
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, z0.shape)
    mask = attention_mask.unsqueeze(-1).to(z0.dtype) # Exclude padded token positions.

    zk = signal_scale * z0 + noise_scale * noise # Directly sample Z_k from Z_0.
    return zk * mask # Keep padded positions equal to zero.
```

Forward corruption

Compute the reverse variance $\tilde{\beta}_k$.

```
# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
)

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)

posterior_variances[0] = 0.0

def extract(values, k, z_shape):
    # Select one schedule value per sequence and add dimensions for broadcasting.
    return values[k].view(k.shape[0], *[1] * (len(z_shape) - 1))

def q_sample(z0, k, attention_mask, noise=None):
    if noise is None:
        noise = torch.randn_like(z0) # Sample epsilon with the same shape as Z_0.

    signal_scale = extract(torch.sqrt(alpha_bars), k, z0.shape)
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, z0.shape)
    mask = attention_mask.unsqueeze(-1).to(z0.dtype) # Exclude padded token positions.

    zk = signal_scale * z0 + noise_scale * noise # Directly sample Z_k from Z_0.
    return zk * mask # Keep padded positions equal to zero.
```

Forward corruption

Select the correct coefficient for each sequence's sampled step k .

```
# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
)

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, z_shape):
    # Select one schedule value per sequence and add dimensions for broadcasting.
    return values[k].view(k.shape[0], *([1] * (len(z_shape) - 1)))

def q_sample(z0, k, attention_mask, noise=None):
    if noise is None:
        noise = torch.randn_like(z0) # Sample epsilon with the same shape as Z_0.

    signal_scale = extract(torch.sqrt(alpha_bars), k, z0.shape)
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, z0.shape)
    mask = attention_mask.unsqueeze(-1).to(z0.dtype) # Exclude padded token positions.

    zk = signal_scale * z0 + noise_scale * noise # Directly sample Z_k from Z_0.
    return zk * mask # Keep padded positions equal to zero.
```

Forward corruption

Constructs

$$Z_k = \sqrt{\bar{\alpha}_k} Z_0 + \sqrt{1 - \bar{\alpha}_k} \epsilon.$$

```
# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
)

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, z_shape):
    # Select one schedule value per sequence and add dimensions for broadcasting.
    return values[k].view(k.shape[0], *[1] * (len(z_shape) - 1))

def q_sample(z0, k, attention_mask, noise=None):
    if noise is None:
        noise = torch.randn_like(z0) # Sample epsilon with the same shape as Z_0.

    signal_scale = extract(torch.sqrt(alpha_bars), k, z0.shape)
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, z0.shape)
    mask = attention_mask.unsqueeze(-1).to(z0.dtype) # Exclude padded token positions.

    zk = signal_scale * z0 + noise_scale * noise # Directly sample Z_k from Z_0.
    return zk * mask # Keep padded positions equal to zero.
```

Forward corruption

Constructs

$$Z_k = \sqrt{\bar{\alpha}_k} Z_0 + \sqrt{1 - \bar{\alpha}_k} \epsilon.$$

The noisy latent Z_k equals the clean latent Z_0 scaled by the square root of cumulative retention $\bar{\alpha}_k$, plus standard Gaussian noise ϵ scaled by the square root of one minus that retention.

```
# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
)

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, z_shape):
    # Select one schedule value per sequence and add dimensions for broadcasting.
    return values[k].view(k.shape[0], *[1] * (len(z_shape) - 1))

def q_sample(z0, k, attention_mask, noise=None):
    if noise is None:
        noise = torch.randn_like(z0) # Sample epsilon with the same shape as Z_0.

    signal_scale = extract(torch.sqrt(alpha_bars), k, z0.shape)
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, z0.shape)
    mask = attention_mask.unsqueeze(-1).to(z0.dtype) # Exclude padded token positions.

    zk = signal_scale * z0 + noise_scale * noise # Directly sample Z_k from Z_0.
    return zk * mask # Keep padded positions equal to zero.
```

Forward corruption

Here, we keep padded sequence positions equal to zero, with M being the attention mask.

$$Z_k = M \odot [\sqrt{\bar{\alpha}_k} Z_0 + \sqrt{1 - \bar{\alpha}_k} \varepsilon]$$

```
# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
)

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, z_shape):
    # Select one schedule value per sequence and add dimensions for broadcasting.
    return values[k].view(k.shape[0], *[1] * (len(z_shape) - 1))

def q_sample(z0, k, attention_mask, noise=None):
    if noise is None:
        noise = torch.randn_like(z0) # Sample epsilon with the same shape as Z_0.

    signal_scale = extract(torch.sqrt(alpha_bars), k, z0.shape)
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, z0.shape)
    mask = attention_mask.unsqueeze(-1).to(z0.dtype) # Exclude padded token positions.

    zk = signal_scale * z0 + noise_scale * noise # Directly sample Z_k from Z_0.
    return zk * mask # Keep padded positions equal to zero.
```

Forward corruption

Here, we keep padded sequence positions equal to zero, with M being the attention mask.

$$Z_k = M \odot [\sqrt{\bar{\alpha}_k} Z_0 + \sqrt{1 - \bar{\alpha}_k} \varepsilon]$$

Multiply the Gaussian corruption of clean latent Z_0 elementwise ($\odot$) by mask M to obtain noisy latent Z_k , keeping valid tokens and zeroing padding; $\bar{\alpha}_k$ is cumulative retention and ε is standard Gaussian noise.

```
# Forward diffusion schedule: beta_k, alpha_k, and alpha_bar_k
betas = torch.cat( # Store beta_0 = 0, followed by beta_1 through beta_K.
    [
        torch.zeros(1, device=DEVICE),
        torch.linspace(BETA_START, BETA_END, K, device=DEVICE),
    ]
)

alphas = 1.0 - betas # alpha_k = 1 - beta_k.
alpha_bars = torch.cumprod(alphas, dim=0) # alpha_bar_k = product of alpha_1,...,alpha_k.
alpha_bars_previous = torch.cat(
    [torch.ones(1, device=DEVICE), alpha_bars[:-1]]
)

posterior_variances = (
    betas
    * (1.0 - alpha_bars_previous)
    / (1.0 - alpha_bars).clamp(min=1e-20)
)
posterior_variances[0] = 0.0

def extract(values, k, z_shape):
    # Select one schedule value per sequence and add dimensions for broadcasting.
    return values[k].view(k.shape[0], *[1] * (len(z_shape) - 1))

def q_sample(z0, k, attention_mask, noise=None):
    if noise is None:
        noise = torch.randn_like(z0) # Sample epsilon with the same shape as Z_0.

    signal_scale = extract(torch.sqrt(alpha_bars), k, z0.shape)
    noise_scale = extract(torch.sqrt(1.0 - alpha_bars), k, z0.shape)
    mask = attention_mask.unsqueeze(-1).to(z0.dtype) # Exclude padded token positions.

    zk = signal_scale * z0 + noise_scale * noise # Directly sample Z_k from Z_0.
    return zk * mask # Keep padded positions equal to zero.
```

Then the Transformer latent denoiser and loss

```
class ESM2LatentDenoiser(nn.Module):
    def __init__(self, latent_dim, max_length, num_layers=6, num_heads=8):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(latent_dim),
            nn.Linear(latent_dim, latent_dim * 4),
            nn.SiLU(),
            nn.Linear(latent_dim * 4, latent_dim),
        )

        self.position_embedding = nn.Parameter(
            torch.randn(1, max_length, latent_dim) * 0.02
        )

        layer = nn.TransformerEncoderLayer(
            d_model=latent_dim,
            nhead=num_heads,
            dim_feedforward=4 * latent_dim,
            activation="gelu",
            batch_first=True,
            norm_first=True,
        )

        self.transformer = nn.TransformerEncoder(
            layer,
            num_layers=num_layers,
            norm=nn.LayerNorm(latent_dim),
        )

        self.output_projection = nn.Linear(latent_dim, latent_dim)

    def forward(self, zk, k, attention_mask):
        time_embedding = self.time_mlp(k).unsqueeze(1)  # Broadcast k across residue positions.
        h = zk + time_embedding + self.position_embedding[:, :, zk.shape[1]]

        h = self.transformer(
            h,
            src_key_padding_mask=~attention_mask.bool(),
        )

        predicted_z0 = self.output_projection(h)  # Predict the clean ESM-2 latent Z_0.
        return predicted_z0 * attention_mask.unsqueeze(-1).to(zk.dtype)

    def masked_clean_latent_mse(predicted_z0, true_z0, attention_mask):
        mask = attention_mask.unsqueeze(-1).to(true_z0.dtype)  # Select real token positions.
        squared_error = (predicted_z0 - true_z0).pow(2) * mask  # Elementwise Z_0 error.
        denominator = mask.sum() * true_z0.shape[-1]  # Number of valid latent coordinates.
        return squared_error.sum() / denominator.clamp(min=1.0)
```

Then the Transformer latent denoiser and loss

Uses the same sinusoidal step embedding as the image DDPM.

```
class ESM2LatentDenoiser(nn.Module):
    def __init__(self, latent_dim, max_length, num_layers=6, num_heads=8):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(latent_dim),
            nn.Linear(latent_dim, latent_dim * 4),
            nn.SiLU(),
            nn.Linear(latent_dim * 4, latent_dim),
        )

        self.position_embedding = nn.Parameter(
            torch.randn(1, max_length, latent_dim) * 0.02
        )

        layer = nn.TransformerEncoderLayer(
            d_model=latent_dim,
            nhead=num_heads,
            dim_feedforward=4 * latent_dim,
            activation="gelu",
            batch_first=True,
            norm_first=True,
        )

        self.transformer = nn.TransformerEncoder(
            layer,
            num_layers=num_layers,
            norm=nn.LayerNorm(latent_dim),
        )

        self.output_projection = nn.Linear(latent_dim, latent_dim)

    def forward(self, zk, k, attention_mask):
        time_embedding = self.time_mlp(k).unsqueeze(1)  # Broadcast k across residue positions.
        h = zk + time_embedding + self.position_embedding[:, :, zk.shape[1]]

        h = self.transformer(
            h,
            src_key_padding_mask=~attention_mask.bool(),
        )

        predicted_z0 = self.output_projection(h)  # Predict the clean ESM-2 latent Z_0.
        return predicted_z0 * attention_mask.unsqueeze(-1).to(zk.dtype)

    def masked_clean_latent_mse(predicted_z0, true_z0, attention_mask):
        mask = attention_mask.unsqueeze(-1).to(true_z0.dtype)  # Select real token positions.
        squared_error = (predicted_z0 - true_z0).pow(2) * mask  # Elementwise Z_0 error.
        denominator = mask.sum() * true_z0.shape[-1]  # Number of valid latent coordinates.
        return squared_error.sum() / denominator.clamp(min=1.0)
```

Then the Transformer latent denoiser and loss

Adds residue-position information because order matters in a protein sequence.

```
class ESM2LatentDenoiser(nn.Module):
    def __init__(self, latent_dim, max_length, num_layers=6, num_heads=8):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(latent_dim),
            nn.Linear(latent_dim, latent_dim * 4),
            nn.SiLU(),
            nn.Linear(latent_dim * 4, latent_dim),
        )

        self.position_embedding = nn.Parameter(
            torch.randn(1, max_length, latent_dim) * 0.02
        )

        layer = nn.TransformerEncoderLayer(
            d_model=latent_dim,
            nhead=num_heads,
            dim_feedforward=4 * latent_dim,
            activation="gelu",
            batch_first=True,
            norm_first=True,
        )

        self.transformer = nn.TransformerEncoder(
            layer,
            num_layers=num_layers,
            norm=nn.LayerNorm(latent_dim),
        )

        self.output_projection = nn.Linear(latent_dim, latent_dim)

    def forward(self, zk, k, attention_mask):
        time_embedding = self.time_mlp(k).unsqueeze(1)  # Broadcast k across residue positions.
        h = zk + time_embedding + self.position_embedding[:, :, zk.shape[1]]

        h = self.transformer(
            h,
            src_key_padding_mask=~attention_mask.bool(),
        )

        predicted_z0 = self.output_projection(h)  # Predict the clean ESM-2 latent Z_0.
        return predicted_z0 * attention_mask.unsqueeze(-1).to(zk.dtype)

    def masked_clean_latent_mse(predicted_z0, true_z0, attention_mask):
        mask = attention_mask.unsqueeze(-1).to(true_z0.dtype)  # Select real token positions.
        squared_error = (predicted_z0 - true_z0).pow(2) * mask  # Elementwise Z_0 error.
        denominator = mask.sum() * true_z0.shape[-1]  # Number of valid latent coordinates.
        return squared_error.sum() / denominator.clamp(min=1.0)
```

Then the Transformer latent denoiser and loss

This creates one self-attention layer operating over the latent sequence.

```
class ESM2LatentDenoiser(nn.Module):
    def __init__(self, latent_dim, max_length, num_layers=6, num_heads=8):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(latent_dim),
            nn.Linear(latent_dim, latent_dim * 4),
            nn.SiLU(),
            nn.Linear(latent_dim * 4, latent_dim),
        )

        self.position_embedding = nn.Parameter(
            torch.randn(1, max_length, latent_dim) * 0.02
        )

        layer = nn.TransformerEncoderLayer(
            d_model=latent_dim,
            nhead=num_heads,
            dim_feedforward=4 * latent_dim,
            activation="gelu",
            batch_first=True,
            norm_first=True,
        )

        self.transformer = nn.TransformerEncoder(
            layer,
            num_layers=num_layers,
            norm=nn.LayerNorm(latent_dim),
        )

        self.output_projection = nn.Linear(latent_dim, latent_dim)

    def forward(self, zk, k, attention_mask):
        time_embedding = self.time_mlp(k).unsqueeze(1)  # Broadcast k across residue positions.
        h = zk + time_embedding + self.position_embedding[:, :, zk.shape[1]]

        h = self.transformer(
            h,
            src_key_padding_mask=~attention_mask.bool(),
        )

        predicted_z0 = self.output_projection(h)  # Predict the clean ESM-2 latent Z_0.
        return predicted_z0 * attention_mask.unsqueeze(-1).to(zk.dtype)

    def masked_clean_latent_mse(predicted_z0, true_z0, attention_mask):
        mask = attention_mask.unsqueeze(-1).to(true_z0.dtype)  # Select real token positions.
        squared_error = (predicted_z0 - true_z0).pow(2) * mask  # Elementwise Z_0 error.
        denominator = mask.sum() * true_z0.shape[-1]  # Number of valid latent coordinates.
        return squared_error.sum() / denominator.clamp(min=1.0)
```

Then the Transformer latent denoiser and loss

We will stack six Transformer layers.

```
class ESM2LatentDenoiser(nn.Module):
    def __init__(self, latent_dim, max_length, num_layers=6, num_heads=8):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(latent_dim),
            nn.Linear(latent_dim, latent_dim * 4),
            nn.SiLU(),
            nn.Linear(latent_dim * 4, latent_dim),
        )

        self.position_embedding = nn.Parameter(
            torch.randn(1, max_length, latent_dim) * 0.02
        )

        layer = nn.TransformerEncoderLayer(
            d_model=latent_dim,
            nhead=num_heads,
            dim_feedforward=4 * latent_dim,
            activation="gelu",
            batch_first=True,
            norm_first=True,
        )

        self.transformer = nn.TransformerEncoder(
            layer,
            num_layers=num_layers,
            norm=nn.LayerNorm(latent_dim),
        )

        self.output_projection = nn.Linear(latent_dim, latent_dim)

    def forward(self, zk, k, attention_mask):
        time_embedding = self.time_mlp(k).unsqueeze(1)  # Broadcast k across residue positions.
        h = zk + time_embedding + self.position_embedding[:, :, zk.shape[1]]

        h = self.transformer(
            h,
            src_key_padding_mask=~attention_mask.bool(),
        )

        predicted_z0 = self.output_projection(h)  # Predict the clean ESM-2 latent Z_0.
        return predicted_z0 * attention_mask.unsqueeze(-1).to(zk.dtype)

    def masked_clean_latent_mse(predicted_z0, true_z0, attention_mask):
        mask = attention_mask.unsqueeze(-1).to(true_z0.dtype)  # Select real token positions.
        squared_error = (predicted_z0 - true_z0).pow(2) * mask  # Elementwise Z_0 error.
        denominator = mask.sum() * true_z0.shape[-1]  # Number of valid latent coordinates.
        return squared_error.sum() / denominator.clamp(min=1.0)
```

Then the Transformer latent denoiser and loss

Combine:

noisy latent + diffusion step + residue position

```
class ESM2LatentDenoiser(nn.Module):
    def __init__(self, latent_dim, max_length, num_layers=6, num_heads=8):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(latent_dim),
            nn.Linear(latent_dim, latent_dim * 4),
            nn.SiLU(),
            nn.Linear(latent_dim * 4, latent_dim),
        )

        self.position_embedding = nn.Parameter(
            torch.randn(1, max_length, latent_dim) * 0.02
        )

        layer = nn.TransformerEncoderLayer(
            d_model=latent_dim,
            nhead=num_heads,
            dim_feedforward=4 * latent_dim,
            activation="gelu",
            batch_first=True,
            norm_first=True,
        )

        self.transformer = nn.TransformerEncoder(
            layer,
            num_layers=num_layers,
            norm=nn.LayerNorm(latent_dim),
        )

        self.output_projection = nn.Linear(latent_dim, latent_dim)

    def forward(self, zk, k, attention_mask):
        time_embedding = SinusoidalTimeEmbedding(zk)
        h = zk + time_embedding + self.position_embedding[:, :, zk.shape[1]]

        h = self.transformer(
            h,
            src_key_padding_mask=~attention_mask.bool(),
        )

        predicted_z0 = self.output_projection(h) # Predict the clean ESM-2 latent Z_0.
        return predicted_z0 * attention_mask.unsqueeze(-1).to(zk.dtype)

    def masked_clean_latent_mse(predicted_z0, true_z0, attention_mask):
        mask = attention_mask.unsqueeze(-1).to(true_z0.dtype) # Select real token positions.
        squared_error = (predicted_z0 - true_z0).pow(2) * mask # Elementwise Z_0 error.
        denominator = mask.sum() * true_z0.shape[-1] # Number of valid latent coordinates.
        return squared_error.sum() / denominator.clamp(min=1.0)
```

Then the Transformer latent denoiser and loss

Combine:

noisy latent + diffusion step + residue position

Add the noisy residue embeddings, the diffusion-step embedding shared across positions, and a learned embedding for each token position before applying the Transformer.

```
class ESM2LatentDenoiser(nn.Module):
    def __init__(self, latent_dim, max_length, num_layers=6, num_heads=8):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(latent_dim),
            nn.Linear(latent_dim, latent_dim * 4),
            nn.SiLU(),
            nn.Linear(latent_dim * 4, latent_dim),
        )

        self.position_embedding = nn.Parameter(
            torch.randn(1, max_length, latent_dim) * 0.02
        )

        layer = nn.TransformerEncoderLayer(
            d_model=latent_dim,
            nhead=num_heads,
            dim_feedforward=4 * latent_dim,
            activation="gelu",
            batch_first=True,
            norm_first=True,
        )

        self.transformer = nn.TransformerEncoder(
            layer,
            num_layers=num_layers,
            norm=nn.LayerNorm(latent_dim),
        )

        self.output_projection = nn.Linear(latent_dim, latent_dim)

    def forward(self, zk, k, attention_mask):
        # zk: [batch, seq_len, latent_dim]
        # k: [batch, seq_len, latent_dim]
        # attention_mask: [batch, seq_len, seq_len]

        h = zk + time_embedding + self.position_embedding[:, :, zk.shape[1]]

        h = self.transformer(
            h,
            src_key_padding_mask=~attention_mask.bool(),
        )

        predicted_z0 = self.output_projection(h) # Predict the clean ESM-2 latent Z_0.
        return predicted_z0 * attention_mask.unsqueeze(-1).to(zk.dtype)

    def masked_clean_latent_mse(predicted_z0, true_z0, attention_mask):
        mask = attention_mask.unsqueeze(-1).to(true_z0.dtype) # Select real token positions.
        squared_error = (predicted_z0 - true_z0).pow(2) * mask # Elementwise Z_0 error.
        denominator = mask.sum() * true_z0.shape[-1] # Number of valid latent coordinates.
        return squared_error.sum() / denominator.clamp(min=1.0)
```

Then the Transformer latent denoiser and loss

We need to prevent self-attention from using padded positions.

:

```
class ESM2LatentDenoiser(nn.Module):
    def __init__(self, latent_dim, max_length, num_layers=6, num_heads=8):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(latent_dim),
            nn.Linear(latent_dim, latent_dim * 4),
            nn.SiLU(),
            nn.Linear(latent_dim * 4, latent_dim),
        )

        self.position_embedding = nn.Parameter(
            torch.randn(1, max_length, latent_dim) * 0.02
        )

        layer = nn.TransformerEncoderLayer(
            d_model=latent_dim,
            nhead=num_heads,
            dim_feedforward=4 * latent_dim,
            activation="gelu",
            batch_first=True,
            norm_first=True,
        )

        self.transformer = nn.TransformerEncoder(
            layer,
            num_layers=num_layers,
            norm=nn.LayerNorm(latent_dim),
        )

        self.output_projection = nn.Linear(latent_dim, latent_dim)

    def forward(self, zk, k, attention_mask):
        time_embedding = self.time_mlp(k).unsqueeze(1)  # Broadcast k across residue positions.
        h = zk + time_embedding + self.position_embedding[:, :, zk.shape[1]]

        h = self.transformer(
            h,
            src_key_padding_mask=~attention_mask.bool(),
        )

        predicted_z0 = self.output_projection(h)  # Predict the clean ESM-2 latent Z_0.
        return predicted_z0 * attention_mask.unsqueeze(-1).to(zk.dtype)

    def masked_clean_latent_mse(predicted_z0, true_z0, attention_mask):
        mask = attention_mask.unsqueeze(-1).to(true_z0.dtype)  # Select real token positions.
        squared_error = (predicted_z0 - true_z0).pow(2) * mask  # Elementwise Z_0 error.
        denominator = mask.sum() * true_z0.shape[-1]  # Number of valid latent coordinates.
        return squared_error.sum() / denominator.clamp(min=1.0)
```

Then the Transformer latent denoiser and loss

We need to prevent self-attention from using padded positions.

:

The attention_mask is one for valid tokens and zero for padding. The code converts it to Boolean and applies ~ to reverse the convention: True in src_key_padding_mask means ignore that position.

```
class ESM2LatentDenoiser(nn.Module):
    def __init__(self, latent_dim, max_length, num_layers=6, num_heads=8):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(latent_dim),
            nn.Linear(latent_dim, latent_dim * 4),
            nn.SiLU(),
            nn.Linear(latent_dim * 4, latent_dim),
        )

        self.position_embedding = nn.Parameter(
            torch.randn(1, max_length, latent_dim) * 0.02
        )

        layer = nn.TransformerEncoderLayer(
            d_model=latent_dim,
            nhead=num_heads,
            dim_feedforward=4 * latent_dim,
            activation="gelu",
            batch_first=True,
            norm_first=True,
        )

        self.transformer = nn.TransformerEncoder(
            layer,
            num_layers=num_layers,
            norm=nn.LayerNorm(latent_dim),
        )

        self.output_projection = nn.Linear(latent_dim, latent_dim)

    def forward(self, zk, k, attention_mask):
        time_embedding = self.time_mlp(k).unsqueeze(1)  # Broadcast k across residue positions.
        h = zk + time_embedding + self.position_embedding[:, :, zk.shape[1]]

        h = self.transformer(
            h,
            src_key_padding_mask=~attention_mask.bool(),
        )

        predicted_z0 = self.output_projection(h)  # Predict the clean ESM-2 latent Z_0.
        return predicted_z0 * attention_mask.unsqueeze(-1).to(zk.dtype)

    def masked_clean_latent_mse(predicted_z0, true_z0, attention_mask):
        mask = attention_mask.unsqueeze(-1).to(true_z0.dtype)  # Select real token positions.
        squared_error = (predicted_z0 - true_z0).pow(2) * mask  # Elementwise Z_0 error.
        denominator = mask.sum() * true_z0.shape[-1]  # Number of valid latent coordinates.
        return squared_error.sum() / denominator.clamp(min=1.0)
```

Then the Transformer latent denoiser and loss

We need to prevent self-attention from using padded positions.

:

Each valid token can attend to every valid position in the sequence, while padded positions receive no attention weight.

```
class ESM2LatentDenoiser(nn.Module):
    def __init__(self, latent_dim, max_length, num_layers=6, num_heads=8):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(latent_dim),
            nn.Linear(latent_dim, latent_dim * 4),
            nn.SiLU(),
            nn.Linear(latent_dim * 4, latent_dim),
        )

        self.position_embedding = nn.Parameter(
            torch.randn(1, max_length, latent_dim) * 0.02
        )

        layer = nn.TransformerEncoderLayer(
            d_model=latent_dim,
            nhead=num_heads,
            dim_feedforward=4 * latent_dim,
            activation="gelu",
            batch_first=True,
            norm_first=True,
        )

        self.transformer = nn.TransformerEncoder(
            layer,
            num_layers=num_layers,
            norm=nn.LayerNorm(latent_dim),
        )

        self.output_projection = nn.Linear(latent_dim, latent_dim)

    def forward(self, zk, k, attention_mask):
        time_embedding = self.time_mlp(k).unsqueeze(1)  # Broadcast k across residue positions.
        h = zk + time_embedding + self.position_embedding[:, :, zk.shape[1]]

        h = self.transformer(
            h,
            src_key_padding_mask=~attention_mask.bool(),
        )

        predicted_z0 = self.output_projection(h)  # Predict the clean ESM-2 latent Z_0.
        return predicted_z0 * attention_mask.unsqueeze(-1).to(zk.dtype)

    def masked_clean_latent_mse(predicted_z0, true_z0, attention_mask):
        mask = attention_mask.unsqueeze(-1).to(true_z0.dtype)  # Select real token positions.
        squared_error = (predicted_z0 - true_z0).pow(2) * mask  # Elementwise Z_0 error.
        denominator = mask.sum() * true_z0.shape[-1]  # Number of valid latent coordinates.
        return squared_error.sum() / denominator.clamp(min=1.0)
```

For the loss

We only compute the clean-latent error only at valid sequence positions.

$$\mathcal{L}(\theta) = \frac{\sum_{b,\ell,d} M_{b\ell} \left(\hat{Z}_{0,b\ell d} - Z_{0,b\ell d} \right)^2}{D \sum_{b,\ell} M_{b\ell}}.$$

```
class ESM2LatentDenoiser(nn.Module):
    def __init__(self, latent_dim, max_length, num_layers=6, num_heads=8):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(latent_dim),
            nn.Linear(latent_dim, latent_dim * 4),
            nn.SiLU(),
            nn.Linear(latent_dim * 4, latent_dim),
        )

        self.position_embedding = nn.Parameter(
            torch.randn(1, max_length, latent_dim) * 0.02
        )

        layer = nn.TransformerEncoderLayer(
            d_model=latent_dim,
            nhead=num_heads,
            dim_feedforward=4 * latent_dim,
            activation="gelu",
            batch_first=True,
            norm_first=True,
        )

        self.transformer = nn.TransformerEncoder(
            layer,
            num_layers=num_layers,
            norm=nn.LayerNorm(latent_dim),
        )

        self.output_projection = nn.Linear(latent_dim, latent_dim)

    def forward(self, zk, k, attention_mask):
        time_embedding = self.time_mlp(k).unsqueeze(1) # Broadcast k across residue positions.
        h = zk + time_embedding + self.position_embedding[:, :, zk.shape[1]]

        h = self.transformer(
            h,
            src_key_padding_mask=~attention_mask.bool(),
        )

        predicted_z0 = self.output_projection(h) # Predict the clean ESM-2 latent Z_0.
        return predicted_z0 * attention_mask.unsqueeze(-1).to(zk.dtype)

    def masked_clean_latent_mse(predicted_z0, true_z0, attention_mask):
        mask = attention_mask.unsqueeze(-1).to(true_z0.dtype) # Select real token positions.
        squared_error = (predicted_z0 - true_z0).pow(2) * mask # Elementwise Z_0 error.
        denominator = mask.sum() * true_z0.shape[-1] # Number of valid latent coordinates.
        return squared_error.sum() / denominator.clamp(min=1.0)
```

For the loss

We only compute the clean-latent error only at valid sequence positions.

$$\mathcal{L}(\theta) = \frac{\sum_{b,\ell,d} M_{bl} \left(\hat{Z}_{0,bld} - Z_{0,bld} \right)^2}{D \sum_{b,\ell} M_{bl}}.$$

The loss $\mathcal{L}(\theta)$ averages squared errors between predicted clean latents $\hat{Z}_0$ and targets Z_0 across valid tokens selected by mask M and D embedding coordinates, training network parameters θ .

```
class ESM2LatentDenoiser(nn.Module):
    def __init__(self, latent_dim, max_length, num_layers=6, num_heads=8):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(latent_dim),
            nn.Linear(latent_dim, latent_dim * 4),
            nn.SiLU(),
            nn.Linear(latent_dim * 4, latent_dim),
        )

        self.position_embedding = nn.Parameter(
            torch.randn(1, max_length, latent_dim) * 0.02
        )

        layer = nn.TransformerEncoderLayer(
            d_model=latent_dim,
            nhead=num_heads,
            dim_feedforward=4 * latent_dim,
            activation="gelu",
            batch_first=True,
            norm_first=True,
        )

        self.transformer = nn.TransformerEncoder(
            layer,
            num_layers=num_layers,
            norm=nn.LayerNorm(latent_dim),
        )

        self.output_projection = nn.Linear(latent_dim, latent_dim)

    def forward(self, zk, k, attention_mask):
        time_embedding = self.time_mlp(k).unsqueeze(1)  # Broadcast k across residue positions.
        h = zk + time_embedding + self.position_embedding[:, :, zk.shape[1]]

        h = self.transformer(
            h,
            src_key_padding_mask=~attention_mask.bool(),
        )

        predicted_z0 = self.output_projection(h)  # Predict the clean ESM-2 latent Z_0.
        return predicted_z0 * attention_mask.unsqueeze(-1).to(zk.dtype)

def masked_clean_latent_mse(predicted_z0, true_z0, attention_mask):
    mask = attention_mask.unsqueeze(-1).to(true_z0.dtype)  # Select real token positions.
    squared_error = (predicted_z0 - true_z0).pow(2) * mask  # Elementwise Z_0 error.
    denominator = mask.sum() * true_z0.shape[-1]  # Number of valid latent coordinates.
    return squared_error.sum() / denominator.clamp(min=1.0)
```

For the loss

We only compute the clean-latent error only at valid sequence positions.

$$\mathcal{L}(\theta) = \frac{\sum_{b,\ell,d} M_{b\ell} \left(\hat{Z}_{0,b\ell d} - Z_{0,b\ell d} \right)^2}{D \sum_{b,\ell} M_{b\ell}}.$$

The indices b , ℓ , and d select the sequence, token position, and embedding coordinate; mask $M_{b\ell}$ is one for valid tokens and zero for padding.

```
class ESM2LatentDenoiser(nn.Module):
    def __init__(self, latent_dim, max_length, num_layers=6, num_heads=8):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(latent_dim),
            nn.Linear(latent_dim, latent_dim * 4),
            nn.SiLU(),
            nn.Linear(latent_dim * 4, latent_dim),
        )

        self.position_embedding = nn.Parameter(
            torch.randn(1, max_length, latent_dim) * 0.02
        )

        layer = nn.TransformerEncoderLayer(
            d_model=latent_dim,
            nhead=num_heads,
            dim_feedforward=4 * latent_dim,
            activation="gelu",
            batch_first=True,
            norm_first=True,
        )

        self.transformer = nn.TransformerEncoder(
            layer,
            num_layers=num_layers,
            norm=nn.LayerNorm(latent_dim),
        )

        self.output_projection = nn.Linear(latent_dim, latent_dim)

    def forward(self, zk, k, attention_mask):
        time_embedding = self.time_mlp(k).unsqueeze(1)  # Broadcast k across residue positions.
        h = zk + time_embedding + self.position_embedding[:, :, zk.shape[1]]

        h = self.transformer(
            h,
            src_key_padding_mask=~attention_mask.bool(),
        )

        predicted_z0 = self.output_projection(h)  # Predict the clean ESM-2 latent Z_0.
        return predicted_z0 * attention_mask.unsqueeze(-1).to(zk.dtype)

    def masked_clean_latent_mse(predicted_z0, true_z0, attention_mask):
        mask = attention_mask.unsqueeze(-1).to(true_z0.dtype)  # Select real token positions.
        squared_error = (predicted_z0 - true_z0).pow(2) * mask  # Elementwise Z_0 error.
        denominator = mask.sum() * true_z0.shape[-1]  # Number of valid latent coordinates.
        return squared_error.sum() / denominator.clamp(min=1.0)
```

For the loss

We only compute the clean-latent error only at valid sequence positions.

$$\mathcal{L}(\theta) = \frac{\sum_{b,\ell,d} M_{b\ell} \left(\hat{Z}_{0,b\ell d} - Z_{0,b\ell d} \right)^2}{D \sum_{b,\ell} M_{b\ell}}.$$

Padding contributes neither error nor weight, so extra padding does not dilute the loss.

```
class ESM2LatentDenoiser(nn.Module):
    def __init__(self, latent_dim, max_length, num_layers=6, num_heads=8):
        super().__init__()

        self.time_mlp = nn.Sequential(
            SinusoidalTimeEmbedding(latent_dim),
            nn.Linear(latent_dim, latent_dim * 4),
            nn.SiLU(),
            nn.Linear(latent_dim * 4, latent_dim),
        )

        self.position_embedding = nn.Parameter(
            torch.randn(1, max_length, latent_dim) * 0.02
        )

        layer = nn.TransformerEncoderLayer(
            d_model=latent_dim,
            nhead=num_heads,
            dim_feedforward=4 * latent_dim,
            activation="gelu",
            batch_first=True,
            norm_first=True,
        )

        self.transformer = nn.TransformerEncoder(
            layer,
            num_layers=num_layers,
            norm=nn.LayerNorm(latent_dim),
        )

        self.output_projection = nn.Linear(latent_dim, latent_dim)

    def forward(self, zk, k, attention_mask):
        time_embedding = self.time_mlp(k).unsqueeze(1)  # Broadcast k across residue positions.
        h = zk + time_embedding + self.position_embedding[:, : zk.shape[1]]

        h = self.transformer(
            h,
            src_key_padding_mask=~attention_mask.bool(),
        )

        predicted_z0 = self.output_projection(h)  # Predict the clean ESM-2 latent Z_0.
        return predicted_z0 * attention_mask.unsqueeze(-1).to(zk.dtype)

def masked_clean_latent_mse(predicted_z0, true_z0, attention_mask):
    mask = attention_mask.unsqueeze(-1).to(true_z0.dtype)  # Select real token positions.
    squared_error = (predicted_z0 - true_z0).pow(2) * mask  # Elementwise Z_0 error.
    denominator = mask.sum() * true_z0.shape[-1]  # Number of valid latent coordinates.
    return squared_error.sum() / denominator.clamp(min=1.0)
```

Reverse latent sampling

Convert $\hat{Z}_0$ into a noise estimate and reuse the DDPM reverse update.

```
@torch.no_grad()
def p_sample(model, zk, k, attention_mask):
    predicted_z0 = model(zk, k, attention_mask) # Estimate the clean latent Z_0.

    alpha_bar_k = extract(alpha_bars, k, zk.shape)
    predicted_noise = ( # Convert Z_0-prediction into epsilon-prediction.
        | zk - torch.sqrt(alpha_bar_k) * predicted_z0
        | ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    beta_k = extract(betas, k, zk.shape)
    alpha_k = extract(alphas, k, zk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute mu_theta(Z_k, k).
        | zk
        | - beta_k
        | * predicted_noise
        | / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
        | )

    posterior_variance_k = extract(posterior_variances, k, zk.shape)
    reverse_noise = torch.randn_like(zk) # Sample fresh reverse-process noise.
    add_noise = (k > 1).float().view(k.shape[0], 1, 1) # Use no noise at k = 1.

    previous_z = reverse_mean + add_noise * torch.sqrt(
        | posterior_variance_k
        | ) * reverse_noise

    return previous_z * attention_mask.unsqueeze(-1).to(zk.dtype)

@torch.no_grad()
def sample_latents(model, latent_dim, num_samples, sequence_length):
    model.eval() # Disable training-time behavior during generation.

    if sequence_length + 2 > MAX_LENGTH:
        | raise ValueError("GENERATED_LENGTH + 2 must be no larger than MAX_LENGTH")

    attention_mask = torch.zeros(
        | num_samples,
        | MAX_LENGTH,
        | device=DEVICE,
        | dtype=torch.bool,
        | )

    attention_mask[:, : sequence_length + 2] = True # Include start and end tokens.

    zk = torch.randn(num_samples, MAX_LENGTH, latent_dim, device=DEVICE) # Start from Z_K.
    zk = zk * attention_mask.unsqueeze(-1)

    for step in range(K, 0, -1):
        | k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        | zk = p_sample(model, zk, k, attention_mask) # Sample Z_{k-1} from Z_k.

    return zk
```

Reverse latent sampling

Predict the clean latent $\hat{Z}_0$.

```
@torch.no_grad()
def sample(model, zk, k, attention_mask):
    predicted_z0 = model(zk, k, attention_mask) # Estimate the clean latent Z_0.

    alpha_bar_k = extract(alpha_bars, k, zk.shape)
    predicted_noise = ( # Convert Z_0-prediction into epsilon-prediction.
        zk - torch.sqrt(alpha_bar_k) * predicted_z0
    ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    beta_k = extract(betas, k, zk.shape)
    alpha_k = extract(alphas, k, zk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute mu_theta(Z_k, k).
        zk
        - beta_k
        * predicted_noise
    ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    posterior_variance_k = extract(posterior_variances, k, zk.shape)
    reverse_noise = torch.randn_like(zk) # Sample fresh reverse-process noise.
    add_noise = (k > 1).float().view(k.shape[0], 1, 1) # Use no noise at k = 1.

    previous_z = reverse_mean + add_noise * torch.sqrt(
        posterior_variance_k
    ) * reverse_noise

    return previous_z * attention_mask.unsqueeze(-1).to(zk.dtype)

@torch.no_grad()
def sample_latents(model, latent_dim, num_samples, sequence_length):
    model.eval() # Disable training-time behavior during generation.

    if sequence_length + 2 > MAX_LENGTH:
        raise ValueError("GENERATED_LENGTH + 2 must be no larger than MAX_LENGTH")

    attention_mask = torch.zeros(
        num_samples,
        MAX_LENGTH,
        device=DEVICE,
        dtype=torch.bool,
    )

    attention_mask[:, : sequence_length + 2] = True # Include start and end tokens.

    zk = torch.randn(num_samples, MAX_LENGTH, latent_dim, device=DEVICE) # Start from Z_K.
    zk = zk * attention_mask.unsqueeze(-1)

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        zk = p_sample(model, zk, k, attention_mask) # Sample Z_{k-1} from Z_k.

    return zk
```

Reverse latent sampling

Rearranges the forward equation to recover $\hat{\epsilon}$:

$$\hat{\epsilon} = \frac{Z_k - \sqrt{\bar{\alpha}_k} \hat{Z}_0}{\sqrt{1 - \bar{\alpha}_k}}.$$

```
@torch.no_grad()
def p_sample(model, zk, k, attention_mask):
    predicted_z0 = model(zk, k, attention_mask) # Estimate the clean latent Z_0.

    alpha_bar_k = extract(alpha_bars, k, zk.shape)
    predicted_noise = ( # Convert Z_0-prediction into epsilon-prediction.
        zk - torch.sqrt(alpha_bar_k) * predicted_z0
    ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    beta_k = extract(betas, k, zk.shape)
    alpha_k = extract(alphas, k, zk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute mu_theta(Z_k, k).
        zk
        - beta_k
        * predicted_noise
    ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    posterior_variance_k = extract(posterior_variances, k, zk.shape)
    reverse_noise = torch.randn_like(zk) # Sample fresh reverse-process noise.
    add_noise = (k > 1).float().view(k.shape[0], 1, 1) # Use no noise at k = 1.

    previous_z = reverse_mean + add_noise * torch.sqrt(
        posterior_variance_k
    ) * reverse_noise

    return previous_z * attention_mask.unsqueeze(-1).to(zk.dtype)

@torch.no_grad()
def sample_latents(model, latent_dim, num_samples, sequence_length):
    model.eval() # Disable training-time behavior during generation.

    if sequence_length + 2 > MAX_LENGTH:
        raise ValueError("GENERATED_LENGTH + 2 must be no larger than MAX_LENGTH")

    attention_mask = torch.zeros(
        num_samples,
        MAX_LENGTH,
        device=DEVICE,
        dtype=torch.bool,
    )
    attention_mask[:, : sequence_length + 2] = True # Include start and end tokens.

    zk = torch.randn(num_samples, MAX_LENGTH, latent_dim, device=DEVICE) # Start from Z_K.
    zk = zk * attention_mask.unsqueeze(-1)

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        zk = p_sample(model, zk, k, attention_mask) # Sample Z_{k-1} from Z_k.

    return zk
```

Reverse latent sampling

Rearranges the forward equation to recover $\hat{\epsilon}$:

$$\hat{\epsilon} = \frac{Z_k - \sqrt{\bar{\alpha}_k} \hat{Z}_0}{\sqrt{1 - \bar{\alpha}_k}}.$$

The inferred noise $\hat{\epsilon}$ equals noisy latent Z_k minus predicted clean latent $\hat{Z}_0$ scaled by the square root of cumulative retention $\bar{\alpha}_k$, all divided by the square root of one minus that retention.

```
@torch.no_grad()
def p_sample(model, zk, k, attention_mask):
    predicted_z0 = model(zk, k, attention_mask) # Estimate the clean latent Z_0.

    alpha_bar_k = extract(alpha_bars, k, zk.shape)
    predicted_noise = ( # Convert Z_0-prediction into epsilon-prediction.
        zk - torch.sqrt(alpha_bar_k) * predicted_z0
    ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    beta_k = extract(betas, k, zk.shape)
    alpha_k = extract(alphas, k, zk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute mu_theta(Z_k, k).
        zk
        - beta_k
        * predicted_noise
    ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    posterior_variance_k = extract(posterior_variances, k, zk.shape)
    reverse_noise = torch.randn_like(zk) # Sample fresh reverse-process noise.
    add_noise = (k > 1).float().view(k.shape[0], 1, 1) # Use no noise at k = 1.

    previous_z = reverse_mean + add_noise * torch.sqrt(
        posterior_variance_k
    ) * reverse_noise

    return previous_z * attention_mask.unsqueeze(-1).to(zk.dtype)

@torch.no_grad()
def sample_latents(model, latent_dim, num_samples, sequence_length):
    model.eval() # Disable training-time behavior during generation.

    if sequence_length + 2 > MAX_LENGTH:
        raise ValueError("GENERATED_LENGTH + 2 must be no larger than MAX_LENGTH")

    attention_mask = torch.zeros(
        num_samples,
        MAX_LENGTH,
        device=DEVICE,
        dtype=torch.bool,
    )

    attention_mask[:, : sequence_length + 2] = True # Include start and end tokens.

    zk = torch.randn(num_samples, MAX_LENGTH, latent_dim, device=DEVICE) # Start from Z_K.
    zk = zk * attention_mask.unsqueeze(-1)

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        zk = p_sample(model, zk, k, attention_mask) # Sample Z_{k-1} from Z_k.

    return zk
```

Reverse latent sampling

Rearranges the forward equation to recover $\hat{\epsilon}$:

$$\hat{\epsilon} = \frac{Z_k - \sqrt{\bar{\alpha}_k} \hat{Z}_0}{\sqrt{1 - \bar{\alpha}_k}}.$$

A clean-state prediction is enough to calculate the noise estimate required by the DDPM sampler.

```
@torch.no_grad()
def p_sample(model, zk, k, attention_mask):
    predicted_z0 = model(zk, k, attention_mask) # Estimate the clean latent Z_0.

    alpha_bar_k = extract(alpha_bars, k, zk.shape)
    predicted_noise = ( # Convert Z_0-prediction into epsilon-prediction.
        zk - torch.sqrt(alpha_bar_k) * predicted_z0
    ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    beta_k = extract(betas, k, zk.shape)
    alpha_k = extract(alphas, k, zk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute mu_theta(Z_k, k).
        zk
        - beta_k
        * predicted_noise
    ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    posterior_variance_k = extract(posterior_variances, k, zk.shape)
    reverse_noise = torch.randn_like(zk) # Sample fresh reverse-process noise.
    add_noise = (k > 1).float().view(k.shape[0], 1, 1) # Use no noise at k = 1.

    previous_z = reverse_mean + add_noise * torch.sqrt(
        posterior_variance_k
    ) * reverse_noise

    return previous_z * attention_mask.unsqueeze(-1).to(zk.dtype)

@torch.no_grad()
def sample_latents(model, latent_dim, num_samples, sequence_length):
    model.eval() # Disable training-time behavior during generation.

    if sequence_length + 2 > MAX_LENGTH:
        raise ValueError("GENERATED_LENGTH + 2 must be no larger than MAX_LENGTH")

    attention_mask = torch.zeros(
        num_samples,
        MAX_LENGTH,
        device=DEVICE,
        dtype=torch.bool,
    )
    attention_mask[:, : sequence_length + 2] = True # Include start and end tokens.

    zk = torch.randn(num_samples, MAX_LENGTH, latent_dim, device=DEVICE) # Start from Z_K.
    zk = zk * attention_mask.unsqueeze(-1)

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        zk = p_sample(model, zk, k, attention_mask) # Sample Z_{k-1} from Z_k.

    return zk
```

Reverse latent sampling

Uses the standard DDPM reverse mean:

$$\mu_{\theta}(Z_k, k) = \frac{1}{\sqrt{\alpha_k}} \left[Z_k - \frac{\beta_k}{\sqrt{1 - \bar{\alpha}_k}} \hat{\epsilon} \right]$$

```
@torch.no_grad()
def p_sample(model, zk, k, attention_mask):
    predicted_z0 = model(zk, k, attention_mask) # Estimate the clean latent Z_0.

    alpha_bar_k = extract(alpha_bars, k, zk.shape)
    predicted_noise = ( # Convert Z_0-prediction into epsilon-prediction.
        | zk - torch.sqrt(alpha_bar_k) * predicted_z0
        ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    beta_k = extract(betas, k, zk.shape)
    alpha_k = extract(alphas, k, zk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute mu_theta(Z_k, k).
        | zk
        | - beta_k
        | * predicted_noise
        | / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
        )

    posterior_variance_k = extract(posterior_variances, k, zk.shape)
    reverse_noise = torch.randn_like(zk) # Sample fresh reverse-process noise.
    add_noise = (k > 1).float().view(k.shape[0], 1, 1) # Use no noise at k = 1.

    previous_z = reverse_mean + add_noise * torch.sqrt(
        | posterior_variance_k
        ) * reverse_noise

    return previous_z * attention_mask.unsqueeze(-1).to(zk.dtype)

@torch.no_grad()
def sample_latents(model, latent_dim, num_samples, sequence_length):
    model.eval() # Disable training-time behavior during generation.

    if sequence_length + 2 > MAX_LENGTH:
        raise ValueError("GENERATED_LENGTH + 2 must be no larger than MAX_LENGTH")

    attention_mask = torch.zeros(
        | num_samples,
        | MAX_LENGTH,
        | device=DEVICE,
        | dtype=torch.bool,
        )
    attention_mask[:, : sequence_length + 2] = True # Include start and end tokens.

    zk = torch.randn(num_samples, MAX_LENGTH, latent_dim, device=DEVICE) # Start from Z_K.
    zk = zk * attention_mask.unsqueeze(-1)

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        zk = p_sample(model, zk, k, attention_mask) # Sample Z_{k-1} from Z_k.

    return zk
```

Reverse latent sampling

Uses the standard DDPM reverse mean:

$$\mu_{\theta}(Z_k, k) = \frac{1}{\sqrt{\alpha_k}} \left[Z_k - \frac{\beta_k}{\sqrt{1 - \bar{\alpha}_k}} \hat{\epsilon} \right]$$

The reverse mean μ_{θ} equals noisy latent Z_k minus inferred noise $\hat{\epsilon}$ weighted by step variance β_k over the square root of one minus cumulative retention $\bar{\alpha}_k$, then divided by the square root of one-step retention α_k .

```
@torch.no_grad()
def p_sample(model, zk, k, attention_mask):
    predicted_z0 = model(zk, k, attention_mask) # Estimate the clean latent Z_0.

    alpha_bar_k = extract(alpha_bars, k, zk.shape)
    predicted_noise = ( # Convert Z_0-prediction into epsilon-prediction.
        | zk - torch.sqrt(alpha_bar_k) * predicted_z0
        ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    beta_k = extract(betas, k, zk.shape)
    alpha_k = extract(alphas, k, zk.shape)
```

```
reverse_mean = torch.rsqrt(alpha_k) * ( # Compute mu_theta(Z_k, k).
    | zk
    | - beta_k
    | * predicted_noise
    | / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
)
```

```
posterior_variance_k = extract(posterior_variances, k, zk.shape)
reverse_noise = torch.randn_like(zk) # Sample fresh reverse-process noise.
add_noise = (k > 1).float().view(k.shape[0], 1, 1) # Use no noise at k = 1.

previous_z = reverse_mean + add_noise * torch.sqrt(
    | posterior_variance_k
) * reverse_noise
```

```
return previous_z * attention_mask.unsqueeze(-1).to(zk.dtype)
```

```
@torch.no_grad()
def sample_latents(model, latent_dim, num_samples, sequence_length):
    model.eval() # Disable training-time behavior during generation.

    if sequence_length + 2 > MAX_LENGTH:
        raise ValueError("GENERATED_LENGTH + 2 must be no larger than MAX_LENGTH")

    attention_mask = torch.zeros(
        | num_samples,
        | MAX_LENGTH,
        | device=DEVICE,
        | dtype=torch.bool,
    )

    attention_mask[:, : sequence_length + 2] = True # Include start and end tokens.

    zk = torch.randn(num_samples, MAX_LENGTH, latent_dim, device=DEVICE) # Start from Z_K.
    zk = zk * attention_mask.unsqueeze(-1)

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        zk = p_sample(model, zk, k, attention_mask) # Sample Z_{k-1} from Z_k.

    return zk
```

Reverse latent sampling

Samples the preceding latent:

$$Z_{k-1} = \mu_{\theta}(Z_k, k) + \sqrt{\tilde{\beta}_k} \xi_k$$

```
@torch.no_grad()
def p_sample(model, zk, k, attention_mask):
    predicted_z0 = model(zk, k, attention_mask) # Estimate the clean latent Z_0.

    alpha_bar_k = extract(alpha_bars, k, zk.shape)
    predicted_noise = ( # Convert Z_0-prediction into epsilon-prediction.
        zk - torch.sqrt(alpha_bar_k) * predicted_z0
    ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    beta_k = extract(betas, k, zk.shape)
    alpha_k = extract(alphas, k, zk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute mu_theta(Z_k, k).
        zk
        - beta_k
        * predicted_noise
    ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    posterior_variance_k = extract(posterior_variances, k, zk.shape)
    reverse_noise = torch.randn_like(zk) # Sample fresh reverse-process noise.
    add_noise = (k > 1).float().view(k.shape[0], 1, 1) # Use no noise at k = 1.

    previous_z = reverse_mean + add_noise * torch.sqrt(
        posterior_variance_k
    ) * reverse_noise

    return previous_z * attention_mask.unsqueeze(-1).to(zk.dtype)

@torch.no_grad()
def sample_latents(model, latent_dim, num_samples, sequence_length):
    model.eval() # Disable training-time behavior during generation.

    if sequence_length + 2 > MAX_LENGTH:
        raise ValueError("GENERATED_LENGTH + 2 must be no larger than MAX_LENGTH")

    attention_mask = torch.zeros(
        num_samples,
        MAX_LENGTH,
        device=DEVICE,
        dtype=torch.bool,
    )
    attention_mask[:, : sequence_length + 2] = True # Include start and end tokens.

    zk = torch.randn(num_samples, MAX_LENGTH, latent_dim, device=DEVICE) # Start from Z_K.
    zk = zk * attention_mask.unsqueeze(-1)

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        zk = p_sample(model, zk, k, attention_mask) # Sample Z_{k-1} from Z_k.

    return zk
```

Reverse latent sampling

Samples the preceding latent:

$$Z_{k-1} = \mu_{\theta}(Z_k, k) + \sqrt{\tilde{\beta}_k} \xi_k$$

The preceding latent Z_{k-1} equals the predicted reverse mean $\mu_{\theta}(Z_k, k)$ for current latent Z_k at step k , plus fresh standard Gaussian noise ξ_k scaled by the square root of reverse variance $\tilde{\beta}_k$.

```
@torch.no_grad()
def p_sample(model, zk, k, attention_mask):
    predicted_z0 = model(zk, k, attention_mask) # Estimate the clean latent Z_0.

    alpha_bar_k = extract(alpha_bars, k, zk.shape)
    predicted_noise = ( # Convert Z_0-prediction into epsilon-prediction.
        zk - torch.sqrt(alpha_bar_k) * predicted_z0
    ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    beta_k = extract(betas, k, zk.shape)
    alpha_k = extract(alphas, k, zk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute mu_theta(Z_k, k).
        zk
        - beta_k
        * predicted_noise
    ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )
```

```
posterior_variance_k = extract(posterior_variances, k, zk.shape)
reverse_noise = torch.randn_like(zk) # Sample fresh reverse-process noise.
add_noise = (k > 1).float().view(k.shape[0], 1, 1) # Use no noise at k = 1.

previous_z = reverse_mean + add_noise * torch.sqrt(
    posterior_variance_k
) * reverse_noise
```

```
return previous_z * attention_mask.unsqueeze(-1).to(zk.dtype)
```

```
@torch.no_grad()
def sample_latents(model, latent_dim, num_samples, sequence_length):
    model.eval() # Disable training-time behavior during generation.

    if sequence_length + 2 > MAX_LENGTH:
        raise ValueError("GENERATED_LENGTH + 2 must be no larger than MAX_LENGTH")

    attention_mask = torch.zeros(
        num_samples,
        MAX_LENGTH,
        device=DEVICE,
        dtype=torch.bool,
    )

    attention_mask[:, : sequence_length + 2] = True # Include start and end tokens.

    zk = torch.randn(num_samples, MAX_LENGTH, latent_dim, device=DEVICE) # Start from Z_K.
    zk = zk * attention_mask.unsqueeze(-1)

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        zk = p_sample(model, zk, k, attention_mask) # Sample Z_{k-1} from Z_k.

    return zk
```

Reverse latent sampling

While keeping padding positions equal to zero throughout sampling.

```
@torch.no_grad()
def p_sample(model, zk, k, attention_mask):
    predicted_z0 = model(zk, k, attention_mask) # Estimate the clean latent Z_0.

    alpha_bar_k = extract(alpha_bars, k, zk.shape)
    predicted_noise = ( # Convert Z_0-prediction into epsilon-prediction.
        | zk - torch.sqrt(alpha_bar_k) * predicted_z0
        | ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    beta_k = extract(betas, k, zk.shape)
    alpha_k = extract(alphas, k, zk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute mu_theta(Z_k, k).
        | zk
        | - beta_k
        | * predicted_noise
        | / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )
```

```
    posterior_variance_k = extract(posterior_variances, k, zk.shape)
    reverse_noise = torch.randn_like(zk) # Sample fresh reverse-process noise.
    add_noise = (k > 1).float().view(k.shape[0], 1, 1) # Use no noise at k = 1.

    previous_z = reverse_mean + add_noise * torch.sqrt(
        | posterior_variance_k
    ) * reverse_noise

    return previous_z * attention_mask.unsqueeze(-1).to(zk.dtype)
```

```
@torch.no_grad()
def sample_latents(model, latent_dim, num_samples, sequence_length):
    model.eval() # Disable training-time behavior during generation.

    if sequence_length + 2 > MAX_LENGTH:
        raise ValueError("GENERATED_LENGTH + 2 must be no larger than MAX_LENGTH")

    attention_mask = torch.zeros(
        | num_samples,
        | MAX_LENGTH,
        | device=DEVICE,
        | dtype=torch.bool,
    )

    attention_mask[:, : sequence_length + 2] = True # Include start and end tokens.

    zk = torch.randn(num_samples, MAX_LENGTH, latent_dim, device=DEVICE) # Start from Z_K.
    zk = zk * attention_mask.unsqueeze(-1)

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        zk = p_sample(model, zk, k, attention_mask) # Sample Z_{k-1} from Z_k.

    return zk
```

Reverse latent sampling

Starts with Gaussian latent noise

$$Z_K \sim \mathcal{N}(0, I)$$

```
@torch.no_grad()
def p_sample(model, zk, k, attention_mask):
    predicted_z0 = model(zk, k, attention_mask) # Estimate the clean latent Z_0.

    alpha_bar_k = extract(alpha_bars, k, zk.shape)
    predicted_noise = ( # Convert Z_0-prediction into epsilon-prediction.
        | zk - torch.sqrt(alpha_bar_k) * predicted_z0
        ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    beta_k = extract(betas, k, zk.shape)
    alpha_k = extract(alphas, k, zk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute mu_theta(Z_k, k).
        | zk
        | - beta_k
        | * predicted_noise
        | / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, zk.shape)
    reverse_noise = torch.randn_like(zk) # Sample fresh reverse-process noise.
    add_noise = (k > 1).float().view(k.shape[0], 1, 1) # Use no noise at k = 1.

    previous_z = reverse_mean + add_noise * torch.sqrt(
        | posterior_variance_k
    ) * reverse_noise

    return previous_z * attention_mask.unsqueeze(-1).to(zk.dtype)

@torch.no_grad()
def sample_latents(model, latent_dim, num_samples, sequence_length):
    model.eval() # Disable training-time behavior during generation.

    if sequence_length + 2 > MAX_LENGTH:
        raise ValueError("GENERATED_LENGTH + 2 must be no larger than MAX_LENGTH")

    attention_mask = torch.zeros(
        | num_samples,
        | MAX_LENGTH,
        | device=DEVICE,
        | dtype=torch.bool,
    )
    attention_mask[:, : sequence_length + 2] = True # Include start and end tokens.

    zk = torch.randn(num_samples, MAX_LENGTH, latent_dim, device=DEVICE) # Start from Z_K.
    zk = zk * attention_mask.unsqueeze(-1)

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        zk = p_sample(model, zk, k, attention_mask) # Sample Z_{k-1} from Z_k.

    return zk
```

Reverse latent sampling

Starts with Gaussian latent noise

$$Z_K \sim \mathcal{N}(0, I)$$

The initial latent Z_K , at final noise index K , has zero mean and identity covariance I , giving independent unit-variance Gaussian noise at every active latent coordinate.

```
@torch.no_grad()
def p_sample(model, zk, k, attention_mask):
    predicted_z0 = model(zk, k, attention_mask) # Estimate the clean latent Z_0.

    alpha_bar_k = extract(alpha_bars, k, zk.shape)
    predicted_noise = ( # Convert Z_0-prediction into epsilon-prediction.
        | zk - torch.sqrt(alpha_bar_k) * predicted_z0
        ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    beta_k = extract(betas, k, zk.shape)
    alpha_k = extract(alphas, k, zk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute mu_theta(Z_k, k).
        | zk
        | - beta_k
        | * predicted_noise
        ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    posterior_variance_k = extract(posterior_variances, k, zk.shape)
    reverse_noise = torch.randn_like(zk) # Sample fresh reverse-process noise.
    add_noise = (k > 1).float().view(k.shape[0], 1, 1) # Use no noise at k = 1.

    previous_z = reverse_mean + add_noise * torch.sqrt(
        | posterior_variance_k
    ) * reverse_noise

    return previous_z * attention_mask.unsqueeze(-1).to(zk.dtype)

@torch.no_grad()
def sample_latents(model, latent_dim, num_samples, sequence_length):
    model.eval() # Disable training-time behavior during generation.

    if sequence_length + 2 > MAX_LENGTH:
        raise ValueError("GENERATED_LENGTH + 2 must be no larger than MAX_LENGTH")

    attention_mask = torch.zeros(
        | num_samples,
        | MAX_LENGTH,
        | device=DEVICE,
        | dtype=torch.bool,
    )

    attention_mask[:, : sequence_length + 2] = True # Include start and end tokens.

    zk = torch.randn(num_samples, MAX_LENGTH, latent_dim, device=DEVICE) # Start from Z_K.
    zk = zk * attention_mask.unsqueeze(-1)

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        zk = p_sample(model, zk, k, attention_mask) # Sample Z_{k-1} from Z_k.

    return zk
```

Reverse latent sampling

Run

$$Z_K \rightarrow Z_{K-1} \rightarrow \cdots \rightarrow Z_0$$

```
@torch.no_grad()
def p_sample(model, zk, k, attention_mask):
    predicted_z0 = model(zk, k, attention_mask) # Estimate the clean latent Z_0.

    alpha_bar_k = extract(alpha_bars, k, zk.shape)
    predicted_noise = ( # Convert Z_0-prediction into epsilon-prediction.
        | zk - torch.sqrt(alpha_bar_k) * predicted_z0
        ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    beta_k = extract(betas, k, zk.shape)
    alpha_k = extract(alphas, k, zk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute mu_theta(Z_k, k).
        | zk
        | - beta_k
        | * predicted_noise
        | / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, zk.shape)
    reverse_noise = torch.randn_like(zk) # Sample fresh reverse-process noise.
    add_noise = (k > 1).float().view(k.shape[0], 1, 1) # Use no noise at k = 1.

    previous_z = reverse_mean + add_noise * torch.sqrt(
        | posterior_variance_k
    ) * reverse_noise

    return previous_z * attention_mask.unsqueeze(-1).to(zk.dtype)

@torch.no_grad()
def sample_latents(model, latent_dim, num_samples, sequence_length):
    model.eval() # Disable training-time behavior during generation.

    if sequence_length + 2 > MAX_LENGTH:
        raise ValueError("GENERATED_LENGTH + 2 must be no larger than MAX_LENGTH")

    attention_mask = torch.zeros(
        | num_samples,
        | MAX_LENGTH,
        | device=DEVICE,
        | dtype=torch.bool,
    )

    attention_mask[:, : sequence_length + 2] = True # Include start and end tokens.

    zk = torch.randn(num_samples, MAX_LENGTH, latent_dim, device=DEVICE) # Start from Z_K.
    zk = zk * attention_mask.unsqueeze(-1)

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        zk = p_sample(model, zk, k, attention_mask) # Sample Z_{k-1} from Z_k.

    return zk
```

Reverse latent sampling

Run

$$Z_K \rightarrow Z_{K-1} \rightarrow \dots \rightarrow Z_0$$

Starting from Gaussian latent Z_K at noise index K , repeatedly sample the preceding latent until reaching generated clean latent Z_0 .

```
@torch.no_grad()
def p_sample(model, zk, k, attention_mask):
    predicted_z0 = model(zk, k, attention_mask) # Estimate the clean latent Z_0.

    alpha_bar_k = extract(alpha_bars, k, zk.shape)
    predicted_noise = ( # Convert Z_0-prediction into epsilon-prediction.
        | zk - torch.sqrt(alpha_bar_k) * predicted_z0
        ) / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)

    beta_k = extract(betas, k, zk.shape)
    alpha_k = extract(alphas, k, zk.shape)

    reverse_mean = torch.rsqrt(alpha_k) * ( # Compute mu_theta(Z_k, k).
        | zk
        | - beta_k
        | * predicted_noise
        | / torch.sqrt(1.0 - alpha_bar_k).clamp(min=1e-8)
    )

    posterior_variance_k = extract(posterior_variances, k, zk.shape)
    reverse_noise = torch.randn_like(zk) # Sample fresh reverse-process noise.
    add_noise = (k > 1).float().view(k.shape[0], 1, 1) # Use no noise at k = 1.

    previous_z = reverse_mean + add_noise * torch.sqrt(
        | posterior_variance_k
    ) * reverse_noise

    return previous_z * attention_mask.unsqueeze(-1).to(zk.dtype)

@torch.no_grad()
def sample_latents(model, latent_dim, num_samples, sequence_length):
    model.eval() # Disable training-time behavior during generation.

    if sequence_length + 2 > MAX_LENGTH:
        raise ValueError("GENERATED_LENGTH + 2 must be no larger than MAX_LENGTH")

    attention_mask = torch.zeros(
        | num_samples,
        | MAX_LENGTH,
        | device=DEVICE,
        | dtype=torch.bool,
    )

    attention_mask[:, : sequence_length + 2] = True # Include start and end tokens.

    zk = torch.randn(num_samples, MAX_LENGTH, latent_dim, device=DEVICE) # Start from Z_K.
    zk = zk * attention_mask.unsqueeze(-1)

    for step in range(K, 0, -1):
        k = torch.full((num_samples,), step, device=DEVICE, dtype=torch.long)
        zk = p_sample(model, zk, k, attention_mask) # Sample Z_{k-1} from Z_k.

    return zk
```

Now encode and decode latents

```
@torch.no_grad()
def encode_sequences(sequences, tokenizer, esm2):
    latent_batches = []
    mask_batches = []

    for start in range(0, len(sequences), ENCODING_BATCH_SIZE):
        sequence_batch = sequences[start : start + ENCODING_BATCH_SIZE]

        tokenized = tokenizer(
            sequence_batch,
            return_tensors="pt",
            padding="max_length",
            truncation=True,
            max_length=MAX_LENGTH,
        )

        input_ids = tokenized["input_ids"].to(DEVICE)
        attention_mask = tokenized["attention_mask"].to(DEVICE).bool()

        # Run the frozen ESM-2 encoder and collect its final hidden states Z_0.
        outputs = esm2.esm(
            input_ids=input_ids,
            attention_mask=attention_mask,
            return_dict=True,
        )

        latent_batches.append(outputs.last_hidden_state.cpu())
        mask_batches.append(attention_mask.cpu())

    return torch.cat(latent_batches), torch.cat(mask_batches)

@torch.no_grad()
def decode_latents(standardized_latents, latent_mean, latent_std, tokenizer, esm2):
    clean_latents = standardized_latents * latent_std + latent_mean # Undo standardization.
    logits = esm2.lm_head(clean_latents) # Convert each latent position to token logits.

    amino_acid_ids = torch.tensor(
        [tokenizer.convert_tokens_to_ids(amino_acid) for amino_acid in AMINO_ACIDS],
        device=DEVICE,
    )

    residue_logits = logits[:, 1 : GENERATED_LENGTH + 1]
    amino_acid_logits = residue_logits.index_select(-1, amino_acid_ids)
    predicted_indices = amino_acid_logits.argmax(dim=-1) # Choose the most likely amino acid.

    return [
        "".join(AMINO_ACIDS[index] for index in row.tolist())
        for row in predicted_indices
    ]
```

Now encode and decode latents

Convert amino-acid sequences into ESM-2 token IDs and attention masks.

```
@torch.no_grad()
def encode_sequences(sequences, tokenizer, esm2):
    latent_batches = []
    mask_batches = []

    for start in range(0, len(sequences), ENCODING_BATCH_SIZE):
        sequence_batch = sequences[start : start + ENCODING_BATCH_SIZE]

        tokenized = tokenizer(
            sequence_batch,
            return_tensors="pt",
            padding="max_length",
            truncation=True,
            max_length=MAX_LENGTH,
        )

        input_ids = tokenized["input_ids"].to(DEVICE)
        attention_mask = tokenized["attention_mask"].to(DEVICE).bool()

        # Run the frozen ESM-2 encoder and collect its final hidden states Z_0.
        outputs = esm2.esm(
            input_ids=input_ids,
            attention_mask=attention_mask,
            return_dict=True,
        )

        latent_batches.append(outputs.last_hidden_state.cpu())
        mask_batches.append(attention_mask.cpu())

    return torch.cat(latent_batches), torch.cat(mask_batches)
```

```
@torch.no_grad()
def decode_latents(standardized_latents, latent_mean, latent_std, tokenizer, esm2):
    clean_latents = standardized_latents * latent_std + latent_mean # Undo standardization.
    logits = esm2.lm_head(clean_latents) # Convert each latent position to token logits.

    amino_acid_ids = torch.tensor(
        [tokenizer.convert_tokens_to_ids(amino_acid) for amino_acid in AMINO_ACIDS],
        device=DEVICE,
    )

    residue_logits = logits[:, 1 : GENERATED_LENGTH + 1]
    amino_acid_logits = residue_logits.index_select(-1, amino_acid_ids)
    predicted_indices = amino_acid_logits.argmax(dim=-1) # Choose the most likely amino acid.

    return [
        "".join(AMINO_ACIDS[index] for index in row.tolist())
        for row in predicted_indices
    ]
```

Now encode and decode latents

Run the frozen ESM-2 encoder.

```
@torch.no_grad()
def encode_sequences(sequences, tokenizer, esm2):
    latent_batches = []
    mask_batches = []

    for start in range(0, len(sequences), ENCODING_BATCH_SIZE):
        sequence_batch = sequences[start : start + ENCODING_BATCH_SIZE]

        tokenized = tokenizer(
            sequence_batch,
            return_tensors="pt",
            padding="max_length",
            truncation=True,
            max_length=MAX_LENGTH,
        )

        input_ids = tokenized["input_ids"].to(DEVICE)
        attention_mask = tokenized["attention_mask"].to(DEVICE).bool()

        # Run the frozen ESM-2 encoder and collect its final hidden states Z_0.
        outputs = esm2.esm(
            input_ids=input_ids,
            attention_mask=attention_mask,
            return_dict=True,
        )

        latent_batches.append(outputs.last_hidden_state.cpu())
        mask_batches.append(attention_mask.cpu())

    return torch.cat(latent_batches), torch.cat(mask_batches)

@torch.no_grad()
def decode_latents(standardized_latents, latent_mean, latent_std, tokenizer, esm2):
    clean_latents = standardized_latents * latent_std + latent_mean # Undo standardization.
    logits = esm2.lm_head(clean_latents) # Convert each latent position to token logits.

    amino_acid_ids = torch.tensor(
        [tokenizer.convert_tokens_to_ids(amino_acid) for amino_acid in AMINO_ACIDS],
        device=DEVICE,
    )

    residue_logits = logits[:, 1 : GENERATED_LENGTH + 1]
    amino_acid_logits = residue_logits.index_select(-1, amino_acid_ids)
    predicted_indices = amino_acid_logits.argmax(dim=-1) # Choose the most likely amino acid.

    return [
        "".join(AMINO_ACIDS[index] for index in row.tolist())
        for row in predicted_indices
    ]
```

Now encode and decode latents

Extract the clean latent representation:

$$Z_0 \in \mathbb{R}^{B \times L \times D}$$

Each residue receives one D-dimensional contextual embedding.

```
@torch.no_grad()
def encode_sequences(sequences, tokenizer, esm2):
    latent_batches = []
    mask_batches = []

    for start in range(0, len(sequences), ENCODING_BATCH_SIZE):
        sequence_batch = sequences[start : start + ENCODING_BATCH_SIZE]

        tokenized = tokenizer(
            sequence_batch,
            return_tensors="pt",
            padding="max_length",
            truncation=True,
            max_length=MAX_LENGTH,
        )

        input_ids = tokenized["input_ids"].to(DEVICE)
        attention_mask = tokenized["attention_mask"].to(DEVICE).bool()

        # Run the frozen ESM-2 encoder and collect its final hidden states Z_0.
        outputs = esm2.esm(
            input_ids=input_ids,
            attention_mask=attention_mask,
            return_dict=True,
        )

        latent_batches.append(outputs.last_hidden_state.cpu())
        mask_batches.append(attention_mask.cpu())

    return torch.cat(latent_batches), torch.cat(mask_batches)
```

```
@torch.no_grad()
def decode_latents(standardized_latents, latent_mean, latent_std, tokenizer, esm2):
    clean_latents = standardized_latents * latent_std + latent_mean # Undo standardization.
    logits = esm2.lm_head(clean_latents) # Convert each latent position to token logits.

    amino_acid_ids = torch.tensor(
        [tokenizer.convert_tokens_to_ids(amino_acid) for amino_acid in AMINO_ACIDS],
        device=DEVICE,
    )

    residue_logits = logits[:, 1 : GENERATED_LENGTH + 1]
    amino_acid_logits = residue_logits.index_select(-1, amino_acid_ids)
    predicted_indices = amino_acid_logits.argmax(dim=-1) # Choose the most likely amino acid.

    return [
        "".join(AMINO_ACIDS[index] for index in row.tolist())
        for row in predicted_indices
    ]
```

Now encode and decode latents

Extract the clean latent representation:

$$Z_0 \in \mathbb{R}^{B \times L \times D}$$

Each residue receives one D-dimensional contextual embedding.

The clean latent tensor Z_0 contains B sequences, L token positions per sequence, and D embedding coordinates at each position.

```
@torch.no_grad()
def encode_sequences(sequences, tokenizer, esm2):
    latent_batches = []
    mask_batches = []

    for start in range(0, len(sequences), ENCODING_BATCH_SIZE):
        sequence_batch = sequences[start : start + ENCODING_BATCH_SIZE]

        tokenized = tokenizer(
            sequence_batch,
            return_tensors="pt",
            padding="max_length",
            truncation=True,
            max_length=MAX_LENGTH,
        )

        input_ids = tokenized["input_ids"].to(DEVICE)
        attention_mask = tokenized["attention_mask"].to(DEVICE).bool()

        # Run the frozen ESM-2 encoder and collect its final hidden states Z_0.
        outputs = esm2.esm(
            input_ids=input_ids,
            attention_mask=attention_mask,
            return_dict=True,
        )

        latent_batches.append(outputs.last_hidden_state.cpu())
        mask_batches.append(attention_mask.cpu())

    return torch.cat(latent_batches), torch.cat(mask_batches)
```

```
@torch.no_grad()
def decode_latents(standardized_latents, latent_mean, latent_std, tokenizer, esm2):
    clean_latents = standardized_latents * latent_std + latent_mean # Undo standardization.
    logits = esm2.lm_head(clean_latents) # Convert each latent position to token logits.

    amino_acid_ids = torch.tensor(
        [tokenizer.convert_tokens_to_ids(amino_acid) for amino_acid in AMINO_ACIDS],
        device=DEVICE,
    )

    residue_logits = logits[:, 1 : GENERATED_LENGTH + 1]
    amino_acid_logits = residue_logits.index_select(-1, amino_acid_ids)
    predicted_indices = amino_acid_logits.argmax(dim=-1) # Choose the most likely amino acid.

    return [
        "".join(AMINO_ACIDS[index] for index in row.tolist())
        for row in predicted_indices
    ]
```

Decoding

Reverses latent standardization:

$$Z_0 = \sigma \odot Z_0^{\text{standardized}} + \mu$$

```
@torch.no_grad()
def encode_sequences(sequences, tokenizer, esm2):
    latent_batches = []
    mask_batches = []

    for start in range(0, len(sequences), ENCODING_BATCH_SIZE):
        sequence_batch = sequences[start : start + ENCODING_BATCH_SIZE]

        tokenized = tokenizer(
            sequence_batch,
            return_tensors="pt",
            padding="max_length",
            truncation=True,
            max_length=MAX_LENGTH,
        )

        input_ids = tokenized["input_ids"].to(DEVICE)
        attention_mask = tokenized["attention_mask"].to(DEVICE).bool()

        # Run the frozen ESM-2 encoder and collect its final hidden states Z_0.
        outputs = esm2.esm(
            input_ids=input_ids,
            attention_mask=attention_mask,
            return_dict=True,
        )

        latent_batches.append(outputs.last_hidden_state.cpu())
        mask_batches.append(attention_mask.cpu())

    return torch.cat(latent_batches), torch.cat(mask_batches)

@torch.no_grad()
def decode_latents(standardized_latents, latent_mean, latent_std, tokenizer, esm2):
    clean_latents = standardized_latents * latent_std + latent_mean # Undo standardization.
    logits = esm2.lm_head(clean_latents) # Convert each latent position to token logits.

    amino_acid_ids = torch.tensor(
        [tokenizer.convert_tokens_to_ids(amino_acid) for amino_acid in AMINO_ACIDS],
        device=DEVICE,
    )

    residue_logits = logits[:, 1 : GENERATED_LENGTH + 1]
    amino_acid_logits = residue_logits.index_select(-1, amino_acid_ids)
    predicted_indices = amino_acid_logits.argmax(dim=-1) # Choose the most likely amino acid.

    return [
        "".join(AMINO_ACIDS[index] for index in row.tolist())
        for row in predicted_indices
    ]
```

Decoding

Reverses latent standardization:

$$Z_0 = \sigma \odot Z_0^{\text{standardized}} + \mu$$

Recover the unstandardized latent Z_0 by multiplying standardized latent $Z_0^{\text{standardized}}$ element by element by coordinate-wise standard deviations σ , then adding coordinate-wise means μ .

```
@torch.no_grad()
def encode_sequences(sequences, tokenizer, esm2):
    latent_batches = []
    mask_batches = []

    for start in range(0, len(sequences), ENCODING_BATCH_SIZE):
        sequence_batch = sequences[start : start + ENCODING_BATCH_SIZE]

        tokenized = tokenizer(
            sequence_batch,
            return_tensors="pt",
            padding="max_length",
            truncation=True,
            max_length=MAX_LENGTH,
        )

        input_ids = tokenized["input_ids"].to(DEVICE)
        attention_mask = tokenized["attention_mask"].to(DEVICE).bool()

        # Run the frozen ESM-2 encoder and collect its final hidden states Z_0.
        outputs = esm2.esm(
            input_ids=input_ids,
            attention_mask=attention_mask,
            return_dict=True,
        )

        latent_batches.append(outputs.last_hidden_state.cpu())
        mask_batches.append(attention_mask.cpu())

    return torch.cat(latent_batches), torch.cat(mask_batches)

@torch.no_grad()
def decode_latents(standardized_latents, latent_mean, latent_std, tokenizer, esm2):
    clean_latents = standardized_latents * latent_std + latent_mean # Undo standardization.
    logits = esm2.lm_head(clean_latents) # Convert each latent position to token logits.

    amino_acid_ids = torch.tensor(
        [tokenizer.convert_tokens_to_ids(amino_acid) for amino_acid in AMINO_ACIDS],
        device=DEVICE,
    )

    residue_logits = logits[:, 1 : GENERATED_LENGTH + 1]
    amino_acid_logits = residue_logits.index_select(-1, amino_acid_ids)
    predicted_indices = amino_acid_logits.argmax(dim=-1) # Choose the most likely amino acid.

    return [
        "".join(AMINO_ACIDS[index] for index in row.tolist())
        for row in predicted_indices
    ]
```

Decoding

Map each generated latent vector to amino-acid token logits.

```
@torch.no_grad()
def encode_sequences(sequences, tokenizer, esm2):
    latent_batches = []
    mask_batches = []

    for start in range(0, len(sequences), ENCODING_BATCH_SIZE):
        sequence_batch = sequences[start : start + ENCODING_BATCH_SIZE]

        tokenized = tokenizer(
            sequence_batch,
            return_tensors="pt",
            padding="max_length",
            truncation=True,
            max_length=MAX_LENGTH,
        )

        input_ids = tokenized["input_ids"].to(DEVICE)
        attention_mask = tokenized["attention_mask"].to(DEVICE).bool()

        # Run the frozen ESM-2 encoder and collect its final hidden states Z_0.
        outputs = esm2.esm(
            input_ids=input_ids,
            attention_mask=attention_mask,
            return_dict=True,
        )

        latent_batches.append(outputs.last_hidden_state.cpu())
        mask_batches.append(attention_mask.cpu())

    return torch.cat(latent_batches), torch.cat(mask_batches)

@torch.no_grad()
def decode_latents(standardized_latents, latent_mean, latent_std, tokenizer, esm2):
    clean_latents = standardized_latents * latent_std + latent_mean # Undo standardization.
    logits = esm2.lm_head(clean_latents) # Convert each latent position to token logits.

    amino_acid_ids = torch.tensor(
        [tokenizer.convert_tokens_to_ids(amino_acid) for amino_acid in AMINO_ACIDS],
        device=DEVICE,
    )

    residue_logits = logits[:, 1 : GENERATED_LENGTH + 1]
    amino_acid_logits = residue_logits.index_select(-1, amino_acid_ids)
    predicted_indices = amino_acid_logits.argmax(dim=-1) # Choose the most likely amino acid.

    return [
        "".join(AMINO_ACIDS[index] for index in row.tolist())
        for row in predicted_indices
    ]
```

Decoding

Restrict the output vocabulary to the 20 standard amino acids.

```
@torch.no_grad()
def encode_sequences(sequences, tokenizer, esm2):
    latent_batches = []
    mask_batches = []

    for start in range(0, len(sequences), ENCODING_BATCH_SIZE):
        sequence_batch = sequences[start : start + ENCODING_BATCH_SIZE]

        tokenized = tokenizer(
            sequence_batch,
            return_tensors="pt",
            padding="max_length",
            truncation=True,
            max_length=MAX_LENGTH,
        )

        input_ids = tokenized["input_ids"].to(DEVICE)
        attention_mask = tokenized["attention_mask"].to(DEVICE).bool()

        # Run the frozen ESM-2 encoder and collect its final hidden states Z_0.
        outputs = esm2.esm(
            input_ids=input_ids,
            attention_mask=attention_mask,
            return_dict=True,
        )

        latent_batches.append(outputs.last_hidden_state.cpu())
        mask_batches.append(attention_mask.cpu())

    return torch.cat(latent_batches), torch.cat(mask_batches)

@torch.no_grad()
def decode_latents(standardized_latents, latent_mean, latent_std, tokenizer, esm2):
    clean_latents = standardized_latents * latent_std + latent_mean # Undo standardization.
    logits = esm2.lm_head(clean_latents) # Convert each latent position to token logits.

    amino_acid_ids = torch.tensor(
        [tokenizer.convert_tokens_to_ids(amino_acid) for amino_acid in AMINO_ACIDS],
        device=DEVICE,
    )

    residue_logits = logits[:, 1 : GENERATED_LENGTH + 1]
    amino_acid_logits = residue_logits.index_select(-1, amino_acid_ids)
    predicted_indices = amino_acid_logits.argmax(dim=-1) # Choose the most likely amino acid.

    return [
        "".join(AMINO_ACIDS[index] for index in row.tolist())
        for row in predicted_indices
    ]
```

Decoding

Select the highest-scoring amino acid at each residue position.

```
@torch.no_grad()
def encode_sequences(sequences, tokenizer, esm2):
    latent_batches = []
    mask_batches = []

    for start in range(0, len(sequences), ENCODING_BATCH_SIZE):
        sequence_batch = sequences[start : start + ENCODING_BATCH_SIZE]

        tokenized = tokenizer(
            sequence_batch,
            return_tensors="pt",
            padding="max_length",
            truncation=True,
            max_length=MAX_LENGTH,
        )

        input_ids = tokenized["input_ids"].to(DEVICE)
        attention_mask = tokenized["attention_mask"].to(DEVICE).bool()

        # Run the frozen ESM-2 encoder and collect its final hidden states Z_0.
        outputs = esm2.esm(
            input_ids=input_ids,
            attention_mask=attention_mask,
            return_dict=True,
        )

        latent_batches.append(outputs.last_hidden_state.cpu())
        mask_batches.append(attention_mask.cpu())

    return torch.cat(latent_batches), torch.cat(mask_batches)

@torch.no_grad()
def decode_latents(standardized_latents, latent_mean, latent_std, tokenizer, esm2):
    clean_latents = standardized_latents * latent_std + latent_mean # Undo standardization.
    logits = esm2.lm_head(clean_latents) # Convert each latent position to token logits.

    amino_acid_ids = torch.tensor(
        [tokenizer.convert_tokens_to_ids(amino_acid) for amino_acid in AMINO_ACIDS],
        device=DEVICE,
    )

    residue_logits = logits[:, 1 : GENERATED_LENGTH + 1]
    amino_acid_logits = residue_logits.index_select(-1, amino_acid_ids)
    predicted_indices = amino_acid_logits.argmax(dim=-1) # Choose the most likely amino acid.

    return [
        "".join(AMINO_ACIDS[index] for index in row.tolist())
        for row in predicted_indices
    ]
```

Decoding

Select the highest-scoring amino acid at each residue position.

The result is a generated peptide sequence.

```
@torch.no_grad()
def encode_sequences(sequences, tokenizer, esm2):
    latent_batches = []
    mask_batches = []

    for start in range(0, len(sequences), ENCODING_BATCH_SIZE):
        sequence_batch = sequences[start : start + ENCODING_BATCH_SIZE]

        tokenized = tokenizer(
            sequence_batch,
            return_tensors="pt",
            padding="max_length",
            truncation=True,
            max_length=MAX_LENGTH,
        )

        input_ids = tokenized["input_ids"].to(DEVICE)
        attention_mask = tokenized["attention_mask"].to(DEVICE).bool()

        # Run the frozen ESM-2 encoder and collect its final hidden states Z_0.
        outputs = esm2.esm(
            input_ids=input_ids,
            attention_mask=attention_mask,
            return_dict=True,
        )

        latent_batches.append(outputs.last_hidden_state.cpu())
        mask_batches.append(attention_mask.cpu())

    return torch.cat(latent_batches), torch.cat(mask_batches)

@torch.no_grad()
def decode_latents(standardized_latents, latent_mean, latent_std, tokenizer, esm2):
    clean_latents = standardized_latents * latent_std + latent_mean # Undo standardization.
    logits = esm2.lm_head(clean_latents) # Convert each latent position to token logits.

    amino_acid_ids = torch.tensor(
        [tokenizer.convert_tokens_to_ids(amino_acid) for amino_acid in AMINO_ACIDS],
        device=DEVICE,
    )

    residue_logits = logits[:, 1 : GENERATED_LENGTH + 1]
    amino_acid_logits = residue_logits.index_select(-1, amino_acid_ids)
    predicted_indices = amino_acid_logits.argmax(dim=-1) # Choose the most likely amino acid.

    return [
        "".join(AMINO_ACIDS[index] for index in row.tolist())
        for row in predicted_indices
    ]
```

Now to main

```
def main():
    from transformers import AutoTokenizer, EsmForMaskedLM

    torch.manual_seed(7)

    sequences = [
        line.strip().upper()
        for line in SEQUENCE_FILE.read_text().splitlines()
        if line.strip() and not line.startswith(">")
    ]

    if not sequences:
        raise ValueError("amp_sequences.txt must contain at least one sequence")

    invalid_sequences = [
        sequence
        for sequence in sequences
        if not set(sequence).issubset(set(AMINO_ACIDS))
    ]

    if invalid_sequences:
        raise ValueError("Every input line must contain only the 20 standard amino acids")

    tokenizer = AutoTokenizer.from_pretrained(ESM2_NAME)
    esm2 = EsmForMaskedLM.from_pretrained(ESM2_NAME).to(DEVICE)
    esm2.eval()
    esm2.requires_grad_(False)

    z0, attention_mask = encode_sequences(sequences, tokenizer, esm2) # Frozen ESM-2 latents.
    z0 = z0.to(DEVICE)
    attention_mask = attention_mask.to(DEVICE)

    valid_z0 = z0[attention_mask]
    latent_mean = valid_z0.mean(dim=0) # Per-dimension latent mean.
    latent_std = valid_z0.std(dim=0).clamp(min=1e-6) # Per-dimension latent scale.

    z0 = (z0 - latent_mean) / latent_std # Standardize before applying diffusion.
    z0 = z0 * attention_mask.unsqueeze(-1)

    dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

    latent_dim = z0.shape[-1]
    model = ESM2LatentDenoiser(
        latent_dim=latent_dim,
        max_length=MAX_LENGTH,
        num_layers=NUM_LAYERS,
        num_heads=NUM_HEADS,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
```

Now to main

Load the ESM-2 tokenizer.

```
def main():
    from transformers import AutoTokenizer, EsmForMaskedLM

    torch.manual_seed(7)

    sequences = [
        line.strip().upper()
        for line in SEQUENCE_FILE.read_text().splitlines()
        if line.strip() and not line.startswith(">")
    ]

    if not sequences:
        raise ValueError("amp_sequences.txt must contain at least one sequence")

    invalid_sequences = [
        sequence
        for sequence in sequences
        if not set(sequence).issubset(set(AMINO_ACIDS))
    ]

    if invalid_sequences:
        raise ValueError("Every input line must contain only the 20 standard amino acids")

    tokenizer = AutoTokenizer.from_pretrained(ESM2_NAME)
    esm2 = EsmForMaskedLM.from_pretrained(ESM2_NAME).to(DEVICE)
    esm2.eval()
    esm2.requires_grad_(False)

    z0, attention_mask = encode_sequences(sequences, tokenizer, esm2) # Frozen ESM-2 latents.
    z0 = z0.to(DEVICE)
    attention_mask = attention_mask.to(DEVICE)

    valid_z0 = z0[attention_mask]
    latent_mean = valid_z0.mean(dim=0) # Per-dimension latent mean.
    latent_std = valid_z0.std(dim=0).clamp(min=1e-6) # Per-dimension latent scale.

    z0 = (z0 - latent_mean) / latent_std # Standardize before applying diffusion.
    z0 = z0 * attention_mask.unsqueeze(-1)

    dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

    latent_dim = z0.shape[-1]
    model = ESM2LatentDenoiser(
        latent_dim=latent_dim,
        max_length=MAX_LENGTH,
        num_layers=NUM_LAYERS,
        num_heads=NUM_HEADS,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
```

Now to main

Load the ESM-2 encoder and language-model head.

```
def main():
    from transformers import AutoTokenizer, EsmForMaskedLM

    torch.manual_seed(7)

    sequences = [
        line.strip().upper()
        for line in SEQUENCE_FILE.read_text().splitlines()
        if line.strip() and not line.startswith(">")
    ]

    if not sequences:
        raise ValueError("amp_sequences.txt must contain at least one sequence")

    invalid_sequences = [
        sequence
        for sequence in sequences
        if not set(sequence).issubset(set(AMINO_ACIDS))
    ]

    if invalid_sequences:
        raise ValueError("Every input line must contain only the 20 standard amino acids")

    tokenizer = AutoTokenizer.from_pretrained(ESM2_NAME)
    esm2 = EsmForMaskedLM.from_pretrained(ESM2_NAME).to(DEVICE)
    esm2.eval()
    esm2.requires_grad_(False)

    z0, attention_mask = encode_sequences(sequences, tokenizer, esm2) # Frozen ESM-2 latents.
    z0 = z0.to(DEVICE)
    attention_mask = attention_mask.to(DEVICE)

    valid_z0 = z0[attention_mask]
    latent_mean = valid_z0.mean(dim=0) # Per-dimension latent mean.
    latent_std = valid_z0.std(dim=0).clamp(min=1e-6) # Per-dimension latent scale.

    z0 = (z0 - latent_mean) / latent_std # Standardize before applying diffusion.
    z0 = z0 * attention_mask.unsqueeze(-1)

    dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

    latent_dim = z0.shape[-1]
    model = ESM2LatentDenoiser(
        latent_dim=latent_dim,
        max_length=MAX_LENGTH,
        num_layers=NUM_LAYERS,
        num_heads=NUM_HEADS,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
```

Now to main

Freeze ESM-2. Training updates only the diffusion denoiser.

```
def main():
    from transformers import AutoTokenizer, EsmForMaskedLM

    torch.manual_seed(7)

    sequences = [
        line.strip().upper()
        for line in SEQUENCE_FILE.read_text().splitlines()
        if line.strip() and not line.startswith(">")
    ]

    if not sequences:
        raise ValueError("amp_sequences.txt must contain at least one sequence")

    invalid_sequences = [
        sequence
        for sequence in sequences
        if not set(sequence).issubset(set(AMINO_ACIDS))
    ]

    if invalid_sequences:
        raise ValueError("Every input line must contain only the 20 standard amino acids")

    tokenizer = AutoTokenizer.from_pretrained(ESM2_NAME)
    esm2 = EsmForMaskedLM.from_pretrained(ESM2_NAME).to(DEVICE)
    esm2.eval()
    esm2.requires_grad_(False)

    z0, attention_mask = encode_sequences(sequences, tokenizer, esm2) # Frozen ESM-2 latents.
    z0 = z0.to(DEVICE)
    attention_mask = attention_mask.to(DEVICE)

    valid_z0 = z0[attention_mask]
    latent_mean = valid_z0.mean(dim=0) # Per-dimension latent mean.
    latent_std = valid_z0.std(dim=0).clamp(min=1e-6) # Per-dimension latent scale.

    z0 = (z0 - latent_mean) / latent_std # Standardize before applying diffusion.
    z0 = z0 * attention_mask.unsqueeze(-1)

    dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

    latent_dim = z0.shape[-1]
    model = ESM2LatentDenoiser(
        latent_dim=latent_dim,
        max_length=MAX_LENGTH,
        num_layers=NUM_LAYERS,
        num_heads=NUM_HEADS,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
```

Standardize the latent space

Compute the mean μ and standard deviation σ of each latent dimension.

```
def main():
    from transformers import AutoTokenizer, EsmForMaskedLM

    torch.manual_seed(7)

    sequences = [
        line.strip().upper()
        for line in SEQUENCE_FILE.read_text().splitlines()
        if line.strip() and not line.startswith(">")
    ]

    if not sequences:
        raise ValueError("amp_sequences.txt must contain at least one sequence")

    invalid_sequences = [
        sequence
        for sequence in sequences
        if not set(sequence).issubset(set(AMINO_ACIDS))
    ]

    if invalid_sequences:
        raise ValueError("Every input line must contain only the 20 standard amino acids")

    tokenizer = AutoTokenizer.from_pretrained(ESM2_NAME)
    esm2 = EsmForMaskedLM.from_pretrained(ESM2_NAME).to(DEVICE)
    esm2.eval()
    esm2.requires_grad_(False)

    z0, attention_mask = encode_sequences(sequences, tokenizer, esm2) # Frozen ESM-2 latents.
    z0 = z0.to(DEVICE)
    attention_mask = attention_mask.to(DEVICE)

    valid_z0 = z0[attention_mask]

    latent_mean = valid_z0.mean(dim=0) # Per-dimension latent mean.
    latent_std = valid_z0.std(dim=0).clamp(min=1e-6) # Per-dimension latent scale.

    z0 = (z0 - latent_mean) / latent_std # Standardize before applying diffusion.
    z0 = z0 * attention_mask.unsqueeze(-1)

    dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

    latent_dim = z0.shape[-1]
    model = ESM2LatentDenoiser(
        latent_dim=latent_dim,
        max_length=MAX_LENGTH,
        num_layers=NUM_LAYERS,
        num_heads=NUM_HEADS,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
```

Standardize the latent space

Standardizes the clean latents:

$$Z_0^{\text{standardized}} = \frac{Z_0 - \mu}{\sigma}$$

```
def main():
    from transformers import AutoTokenizer, EsmForMaskedLM

    torch.manual_seed(7)

    sequences = [
        line.strip().upper()
        for line in SEQUENCE_FILE.read_text().splitlines()
        if line.strip() and not line.startswith(">")
    ]

    if not sequences:
        raise ValueError("amp_sequences.txt must contain at least one sequence")

    invalid_sequences = [
        sequence
        for sequence in sequences
        if not set(sequence).issubset(set(AMINO_ACIDS))
    ]

    if invalid_sequences:
        raise ValueError("Every input line must contain only the 20 standard amino acids")

    tokenizer = AutoTokenizer.from_pretrained(ESM2_NAME)
    esm2 = EsmForMaskedLM.from_pretrained(ESM2_NAME).to(DEVICE)
    esm2.eval()
    esm2.requires_grad_(False)

    z0, attention_mask = encode_sequences(sequences, tokenizer, esm2) # Frozen ESM-2 latents.
    z0 = z0.to(DEVICE)
    attention_mask = attention_mask.to(DEVICE)

    valid_z0 = z0[attention_mask]
    latent_mean = valid_z0.mean(dim=0) # Per-dimension latent mean.
    latent_std = valid_z0.std(dim=0).clamp(min=1e-6) # Per-dimension latent scale.

    z0 = (z0 - latent_mean) / latent_std # Standardize before applying diffusion.
    z0 = z0 * attention_mask.unsqueeze(-1)

    dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

    latent_dim = z0.shape[-1]
    model = ESM2LatentDenoiser(
        latent_dim=latent_dim,
        max_length=MAX_LENGTH,
        num_layers=NUM_LAYERS,
        num_heads=NUM_HEADS,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
```

Standardize the latent space

Standardizes the clean latents:

$$Z_0^{\text{standardized}} = \frac{Z_0 - \mu}{\sigma}$$

The standardized clean latent $Z_0^{\text{standardized}}$ equals original latent Z_0 minus each coordinate's training mean μ , divided by that coordinate's training standard deviation σ .

```
def main():
    from transformers import AutoTokenizer, EsmForMaskedLM

    torch.manual_seed(7)

    sequences = [
        line.strip().upper()
        for line in SEQUENCE_FILE.read_text().splitlines()
        if line.strip() and not line.startswith(">")
    ]

    if not sequences:
        raise ValueError("amp_sequences.txt must contain at least one sequence")

    invalid_sequences = [
        sequence
        for sequence in sequences
        if not set(sequence).issubset(set(AMINO_ACIDS))
    ]

    if invalid_sequences:
        raise ValueError("Every input line must contain only the 20 standard amino acids")

    tokenizer = AutoTokenizer.from_pretrained(ESM2_NAME)
    esm2 = EsmForMaskedLM.from_pretrained(ESM2_NAME).to(DEVICE)
    esm2.eval()
    esm2.requires_grad_(False)

    z0, attention_mask = encode_sequences(sequences, tokenizer, esm2) # Frozen ESM-2 latents.
    z0 = z0.to(DEVICE)
    attention_mask = attention_mask.to(DEVICE)

    valid_z0 = z0[attention_mask]
    latent_mean = valid_z0.mean(dim=0) # Per-dimension latent mean.
    latent_std = valid_z0.std(dim=0).clamp(min=1e-6) # Per-dimension latent scale.

    z0 = (z0 - latent_mean) / latent_std # Standardize before applying diffusion.
    z0 = z0 * attention_mask.unsqueeze(-1)

    dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

    latent_dim = z0.shape[-1]
    model = ESM2LatentDenoiser(
        latent_dim=latent_dim,
        max_length=MAX_LENGTH,
        num_layers=NUM_LAYERS,
        num_heads=NUM_HEADS,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
```

Standardize the latent space

Standardizes the clean latents:

$$Z_0^{\text{standardized}} = \frac{Z_0 - \mu}{\sigma}$$

Standardizing each latent coordinate brings its training mean near zero and variance near one.

```
def main():
    from transformers import AutoTokenizer, EsmForMaskedLM

    torch.manual_seed(7)

    sequences = [
        line.strip().upper()
        for line in SEQUENCE_FILE.read_text().splitlines()
        if line.strip() and not line.startswith(">")
    ]

    if not sequences:
        raise ValueError("amp_sequences.txt must contain at least one sequence")

    invalid_sequences = [
        sequence
        for sequence in sequences
        if not set(sequence).issubset(set(AMINO_ACIDS))
    ]

    if invalid_sequences:
        raise ValueError("Every input line must contain only the 20 standard amino acids")

    tokenizer = AutoTokenizer.from_pretrained(ESM2_NAME)
    esm2 = EsmForMaskedLM.from_pretrained(ESM2_NAME).to(DEVICE)
    esm2.eval()
    esm2.requires_grad_(False)

    z0, attention_mask = encode_sequences(sequences, tokenizer, esm2) # Frozen ESM-2 latents.
    z0 = z0.to(DEVICE)
    attention_mask = attention_mask.to(DEVICE)

    valid_z0 = z0[attention_mask]
    latent_mean = valid_z0.mean(dim=0) # Per-dimension latent mean.
    latent_std = valid_z0.std(dim=0).clamp(min=1e-6) # Per-dimension latent scale.

    z0 = (z0 - latent_mean) / latent_std # Standardize before applying diffusion.
    z0 = z0 * attention_mask.unsqueeze(-1)

    dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

    latent_dim = z0.shape[-1]
    model = ESM2LatentDenoiser(
        latent_dim=latent_dim,
        max_length=MAX_LENGTH,
        num_layers=NUM_LAYERS,
        num_heads=NUM_HEADS,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
```

Standardize the latent space

Standardizes the clean latents:

$$Z_0^{\text{standardized}} = \frac{Z_0 - \mu}{\sigma}$$

This makes the Gaussian terminal distribution more appropriate for latent diffusion.

```
def main():
    from transformers import AutoTokenizer, EsmForMaskedLM

    torch.manual_seed(7)

    sequences = [
        line.strip().upper()
        for line in SEQUENCE_FILE.read_text().splitlines()
        if line.strip() and not line.startswith(">")
    ]

    if not sequences:
        raise ValueError("amp_sequences.txt must contain at least one sequence")

    invalid_sequences = [
        sequence
        for sequence in sequences
        if not set(sequence).issubset(set(AMINO_ACIDS))
    ]

    if invalid_sequences:
        raise ValueError("Every input line must contain only the 20 standard amino acids")

    tokenizer = AutoTokenizer.from_pretrained(ESM2_NAME)
    esm2 = EsmForMaskedLM.from_pretrained(ESM2_NAME).to(DEVICE)
    esm2.eval()
    esm2.requires_grad_(False)

    z0, attention_mask = encode_sequences(sequences, tokenizer, esm2) # Frozen ESM-2 latents.
    z0 = z0.to(DEVICE)
    attention_mask = attention_mask.to(DEVICE)

    valid_z0 = z0[attention_mask]
    latent_mean = valid_z0.mean(dim=0) # Per-dimension latent mean.
    latent_std = valid_z0.std(dim=0).clamp(min=1e-6) # Per-dimension latent scale.

    z0 = (z0 - latent_mean) / latent_std # Standardize before applying diffusion.
    z0 = z0 * attention_mask.unsqueeze(-1)

    dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

    latent_dim = z0.shape[-1]
    model = ESM2LatentDenoiser(
        latent_dim=latent_dim,
        max_length=MAX_LENGTH,
        num_layers=NUM_LAYERS,
        num_heads=NUM_HEADS,
    ).to(DEVICE)
    optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

    for epoch in range(1, EPOCHS + 1):
```

Train the denoiser

```
dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

latent_dim = z0.shape[-1]
model = ESM2LatentDenoiser(
    latent_dim=latent_dim,
    max_length=MAX_LENGTH,
    num_layers=NUM_LAYERS,
    num_heads=NUM_HEADS,
).to(DEVICE)
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

for epoch in range(1, EPOCHS + 1):
    model.train()
    epoch_loss = 0.0

    for true_z0, batch_mask in loader:
        true_z0 = true_z0.to(DEVICE)
        batch_mask = batch_mask.to(DEVICE)
        k = torch.randint(1, K + 1, (true_z0.shape[0],), device=DEVICE) # One k per sequence.

        zk = q_sample(true_z0, k, batch_mask) # Corrupt the clean latent at step k.
        predicted_z0 = model(zk, k, batch_mask) # Predict the clean latent.
        loss = masked_clean_latent_mse(predicted_z0, true_z0, batch_mask)

        optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
        loss.backward() # Differentiate the clean-latent MSE.
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # Stabilize training.
        optimizer.step() # Update theta.

    epoch_loss += loss.item()

    print(f"epoch={epoch:03d} clean_latent_mse={epoch_loss / len(loader):.6f}")

torch.save(model.state_dict(), "ampldiffusion_esm2.pt")

generated_latents = sample_latents(
    model,
    latent_dim=latent_dim,
    num_samples=NUM_SAMPLES,
    sequence_length=GENERATED_LENGTH,
)

generated_sequences = decode_latents(
    generated_latents,
    latent_mean,
    latent_std,
    tokenizer,
    esm2,
)

for index, sequence in enumerate(generated_sequences, start=1):
    print(f">generated_{index}")
    print(sequence)
```

Train the denoiser

Samples one diffusion step per sequence.

```
dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

latent_dim = z0.shape[-1]
model = ESM2LatentDenoiser(
    latent_dim=latent_dim,
    max_length=MAX_LENGTH,
    num_layers=NUM_LAYERS,
    num_heads=NUM_HEADS,
).to(DEVICE)
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

for epoch in range(1, EPOCHS + 1):
    model.train()
    epoch_loss = 0.0

    for true_z0, batch_mask in loader:
        true_z0 = true_z0.to(DEVICE)
        batch_mask = batch_mask.to(DEVICE)

        k = torch.randint(1, K + 1, (true_z0.shape[0],), device=DEVICE) # One k per sequence.

        zk = q_sample(true_z0, k, batch_mask) # Corrupt the clean latent at step k.
        predicted_z0 = model(zk, k, batch_mask) # Predict the clean latent.
        loss = masked_clean_latent_mse(predicted_z0, true_z0, batch_mask)

        optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
        loss.backward() # Differentiate the clean-latent MSE.
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # Stabilize training.
        optimizer.step() # Update theta.

        epoch_loss += loss.item()

    print(f"epoch={epoch:03d} clean_latent_mse={epoch_loss / len(loader):.6f}")

torch.save(model.state_dict(), "ampdiffusion_esm2.pt")

generated_latents = sample_latents(
    model,
    latent_dim=latent_dim,
    num_samples=NUM_SAMPLES,
    sequence_length=GENERATED_LENGTH,
)

generated_sequences = decode_latents(
    generated_latents,
    latent_mean,
    latent_std,
    tokenizer,
    esm2,
)

for index, sequence in enumerate(generated_sequences, start=1):
    print(f">generated_{index}")
    print(sequence)
```

Train the denoiser

Corrupt the clean ESM-2 latent.

```
dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

latent_dim = z0.shape[-1]
model = ESM2LatentDenoiser(
    latent_dim=latent_dim,
    max_length=MAX_LENGTH,
    num_layers=NUM_LAYERS,
    num_heads=NUM_HEADS,
).to(DEVICE)
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

for epoch in range(1, EPOCHS + 1):
    model.train()
    epoch_loss = 0.0

    for true_z0, batch_mask in loader:
        true_z0 = true_z0.to(DEVICE)
        batch_mask = batch_mask.to(DEVICE)
        k = torch.randint(1, K + 1, (true_z0.shape[0],), device=DEVICE) # One k per sequence.

        zk = q_sample(true_z0, k, batch_mask) # Corrupt the clean latent at step k.
        predicted_z0 = model(zk, k, batch_mask) # Predict the clean latent.
        loss = masked_clean_latent_mse(predicted_z0, true_z0, batch_mask)

        optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
        loss.backward() # Differentiate the clean-latent MSE.
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # Stabilize training.
        optimizer.step() # Update theta.

        epoch_loss += loss.item()

    print(f"epoch={epoch:03d} clean_latent_mse={epoch_loss / len(loader):.6f}")

torch.save(model.state_dict(), "ampdiffusion_esm2.pt")

generated_latents = sample_latents(
    model,
    latent_dim=latent_dim,
    num_samples=NUM_SAMPLES,
    sequence_length=GENERATED_LENGTH,
)

generated_sequences = decode_latents(
    generated_latents,
    latent_mean,
    latent_std,
    tokenizer,
    esm2,
)

for index, sequence in enumerate(generated_sequences, start=1):
    print(f">generated_{index}")
    print(sequence)
```

Train the denoiser

Predict $\hat{Z}_0$.

```
dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

latent_dim = z0.shape[-1]
model = ESM2LatentDenoiser(
    latent_dim=latent_dim,
    max_length=MAX_LENGTH,
    num_layers=NUM_LAYERS,
    num_heads=NUM_HEADS,
).to(DEVICE)
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

for epoch in range(1, EPOCHS + 1):
    model.train()
    epoch_loss = 0.0

    for true_z0, batch_mask in loader:
        true_z0 = true_z0.to(DEVICE)
        batch_mask = batch_mask.to(DEVICE)
        k = torch.randint(1, K + 1, (true_z0.shape[0],), device=DEVICE) # One k per sequence.

        # Sample k clean latents from the true latents
        zk = true_z0[torch.arange(true_z0.shape[0]), k]

        predicted_z0 = model(zk, k, batch_mask) # Predict the clean latent.
        loss = masked_clean_latent_mse(predicted_z0, true_z0, batch_mask)

        optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
        loss.backward() # Differentiate the clean-latent MSE.
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # Stabilize training.
        optimizer.step() # Update theta.

        epoch_loss += loss.item()

    print(f"epoch={epoch:03d} clean_latent_mse={epoch_loss / len(loader):.6f}")

torch.save(model.state_dict(), "ampdiffusion_esm2.pt")

generated_latents = sample_latents(
    model,
    latent_dim=latent_dim,
    num_samples=NUM_SAMPLES,
    sequence_length=GENERATED_LENGTH,
)

generated_sequences = decode_latents(
    generated_latents,
    latent_mean,
    latent_std,
    tokenizer,
    esm2,
)

for index, sequence in enumerate(generated_sequences, start=1):
    print(f">generated_{index}")
    print(sequence)
```

Train the denoiser

Compares $\hat{Z}_0$ with Z_0 only at valid residue positions.

```
dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

latent_dim = z0.shape[-1]
model = ESM2LatentDenoiser(
    latent_dim=latent_dim,
    max_length=MAX_LENGTH,
    num_layers=NUM_LAYERS,
    num_heads=NUM_HEADS,
).to(DEVICE)
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

for epoch in range(1, EPOCHS + 1):
    model.train()
    epoch_loss = 0.0

    for true_z0, batch_mask in loader:
        true_z0 = true_z0.to(DEVICE)
        batch_mask = batch_mask.to(DEVICE)
        k = torch.randint(1, K + 1, (true_z0.shape[0],), device=DEVICE) # One k per sequence.

        zk = q_sample(true_z0, k, batch_mask) # Corrupt the clean latent at step k.
        predicted_z0 = model(zk, k, batch_mask) # Predict the clean latent.
        loss = masked_clean_latent_mse(predicted_z0, true_z0, batch_mask)

        optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
        loss.backward() # Differentiate the clean-latent MSE.
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # Stabilize training.
        optimizer.step() # Update theta.

    epoch_loss += loss.item()

    print(f"epoch={epoch:03d} clean_latent_mse={epoch_loss / len(loader):.6f}")

torch.save(model.state_dict(), "ampdiffusion_esm2.pt")

generated_latents = sample_latents(
    model,
    latent_dim=latent_dim,
    num_samples=NUM_SAMPLES,
    sequence_length=GENERATED_LENGTH,
)

generated_sequences = decode_latents(
    generated_latents,
    latent_mean,
    latent_std,
    tokenizer,
    esm2,
)

for index, sequence in enumerate(generated_sequences, start=1):
    print(f">generated_{index}")
    print(sequence)
```

Train the denoiser

Update the Transformer parameters θ .

```
dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

latent_dim = z0.shape[-1]
model = ESM2LatentDenoiser(
    latent_dim=latent_dim,
    max_length=MAX_LENGTH,
    num_layers=NUM_LAYERS,
    num_heads=NUM_HEADS,
).to(DEVICE)
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

for epoch in range(1, EPOCHS + 1):
    model.train()
    epoch_loss = 0.0

    for true_z0, batch_mask in loader:
        true_z0 = true_z0.to(DEVICE)
        batch_mask = batch_mask.to(DEVICE)
        k = torch.randint(1, K + 1, (true_z0.shape[0],), device=DEVICE) # One k per sequence.

        zk = q_sample(true_z0, k, batch_mask) # Corrupt the clean latent at step k.
        predicted_z0 = model(zk, k, batch_mask) # Predict the clean latent.
        loss = masked_clean_latent_mse(predicted_z0, true_z0, batch_mask)

        optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
        loss.backward() # Differentiate the clean-latent MSE.
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # Stabilize training.
        optimizer.step() # Update theta.

    epoch_loss += loss.item()

    print(f"epoch={epoch:03d} clean_latent_mse={epoch_loss / len(loader):.6f}")

torch.save(model.state_dict(), "ampdiffusion_esm2.pt")

generated_latents = sample_latents(
    model,
    latent_dim=latent_dim,
    num_samples=NUM_SAMPLES,
    sequence_length=GENERATED_LENGTH,
)

generated_sequences = decode_latents(
    generated_latents,
    latent_mean,
    latent_std,
    tokenizer,
    esm2,
)

for index, sequence in enumerate(generated_sequences, start=1):
    print(f">generated_{index}")
    print(sequence)
```

Generate peptides

Generates standardized clean latents from Gaussian noise.

```
dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

latent_dim = z0.shape[-1]
model = ESM2LatentDenoiser(
    latent_dim=latent_dim,
    max_length=MAX_LENGTH,
    num_layers=NUM_LAYERS,
    num_heads=NUM_HEADS,
).to(DEVICE)
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

for epoch in range(1, EPOCHS + 1):
    model.train()
    epoch_loss = 0.0

    for true_z0, batch_mask in loader:
        true_z0 = true_z0.to(DEVICE)
        batch_mask = batch_mask.to(DEVICE)
        k = torch.randint(1, K + 1, (true_z0.shape[0],), device=DEVICE) # One k per sequence.

        zk = q_sample(true_z0, k, batch_mask) # Corrupt the clean latent at step k.
        predicted_z0 = model(zk, k, batch_mask) # Predict the clean latent.
        loss = masked_clean_latent_mse(predicted_z0, true_z0, batch_mask)

        optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
        loss.backward() # Differentiate the clean-latent MSE.
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # Stabilize training.
        optimizer.step() # Update theta.

    epoch_loss += loss.item()

    print(f"epoch={epoch:03d} clean_latent_mse={epoch_loss / len(loader):.6f}")

torch.save(model.state_dict(), "ampdiffusion_esm2.pt")

generated_latents = sample_latents(
    model,
    latent_dim=latent_dim,
    num_samples=NUM_SAMPLES,
    sequence_length=GENERATED_LENGTH,
)

generated_sequences = decode_latents(
    generated_latents,
    latent_mean,
    latent_std,
    tokenizer,
    esm2,
)

for index, sequence in enumerate(generated_sequences, start=1):
    print(f">generated_{index}")
    print(sequence)
```

Generate peptides

The final output is

$$Z_K \rightarrow Z_0 \rightarrow \text{amino-acid sequence}$$

```
dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

latent_dim = z0.shape[-1]
model = ESM2LatentDenoiser(
    latent_dim=latent_dim,
    max_length=MAX_LENGTH,
    num_layers=NUM_LAYERS,
    num_heads=NUM_HEADS,
).to(DEVICE)
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

for epoch in range(1, EPOCHS + 1):
    model.train()
    epoch_loss = 0.0

    for true_z0, batch_mask in loader:
        true_z0 = true_z0.to(DEVICE)
        batch_mask = batch_mask.to(DEVICE)
        k = torch.randint(1, K + 1, (true_z0.shape[0],), device=DEVICE) # One k per sequence.

        zk = q_sample(true_z0, k, batch_mask) # Corrupt the clean latent at step k.
        predicted_z0 = model(zk, k, batch_mask) # Predict the clean latent.
        loss = masked_clean_latent_mse(predicted_z0, true_z0, batch_mask)

        optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
        loss.backward() # Differentiate the clean-latent MSE.
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # Stabilize training.
        optimizer.step() # Update theta.

    epoch_loss += loss.item()

    print(f"epoch={epoch:03d} clean_latent_mse={epoch_loss / len(loader):.6f}")

torch.save(model.state_dict(), "ampdiffusion_esm2.pt")

generated_latents = sample_latents(
    model,
    latent_dim=latent_dim,
    num_samples=NUM_SAMPLES,
    sequence_length=GENERATED_LENGTH,
)

generated_sequences = decode_latents(
    generated_latents,
    latent_mean,
    latent_std,
    tokenizer,
    esm2,
)

for index, sequence in enumerate(generated_sequences, start=1):
    print(f">generated_{index}")
    print(sequence)
```

Generate peptides

The final output is

$$Z_K \rightarrow Z_0 \rightarrow \text{amino-acid sequence}$$

First reverse the Gaussian latent Z_K , at final noise index K , into a generated clean latent Z_0 ; then undo standardization and decode its token positions into an amino-acid sequence.

```
dataset = TensorDataset(z0.cpu(), attention_mask.cpu())
loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

latent_dim = z0.shape[-1]
model = ESM2LatentDenoiser(
    latent_dim=latent_dim,
    max_length=MAX_LENGTH,
    num_layers=NUM_LAYERS,
    num_heads=NUM_HEADS,
).to(DEVICE)
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)

for epoch in range(1, EPOCHS + 1):
    model.train()
    epoch_loss = 0.0

    for true_z0, batch_mask in loader:
        true_z0 = true_z0.to(DEVICE)
        batch_mask = batch_mask.to(DEVICE)
        k = torch.randint(1, K + 1, (true_z0.shape[0],), device=DEVICE) # One k per sequence.

        zk = q_sample(true_z0, k, batch_mask) # Corrupt the clean latent at step k.
        predicted_z0 = model(zk, k, batch_mask) # Predict the clean latent.
        loss = masked_clean_latent_mse(predicted_z0, true_z0, batch_mask)

        optimizer.zero_grad(set_to_none=True) # Clear the previous gradients.
        loss.backward() # Differentiate the clean-latent MSE.
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # Stabilize training.
        optimizer.step() # Update theta.

    epoch_loss += loss.item()

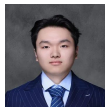
    print(f"epoch={epoch:03d} clean_latent_mse={epoch_loss / len(loader):.6f}")

torch.save(model.state_dict(), "ampdiffusion_esm2.pt")

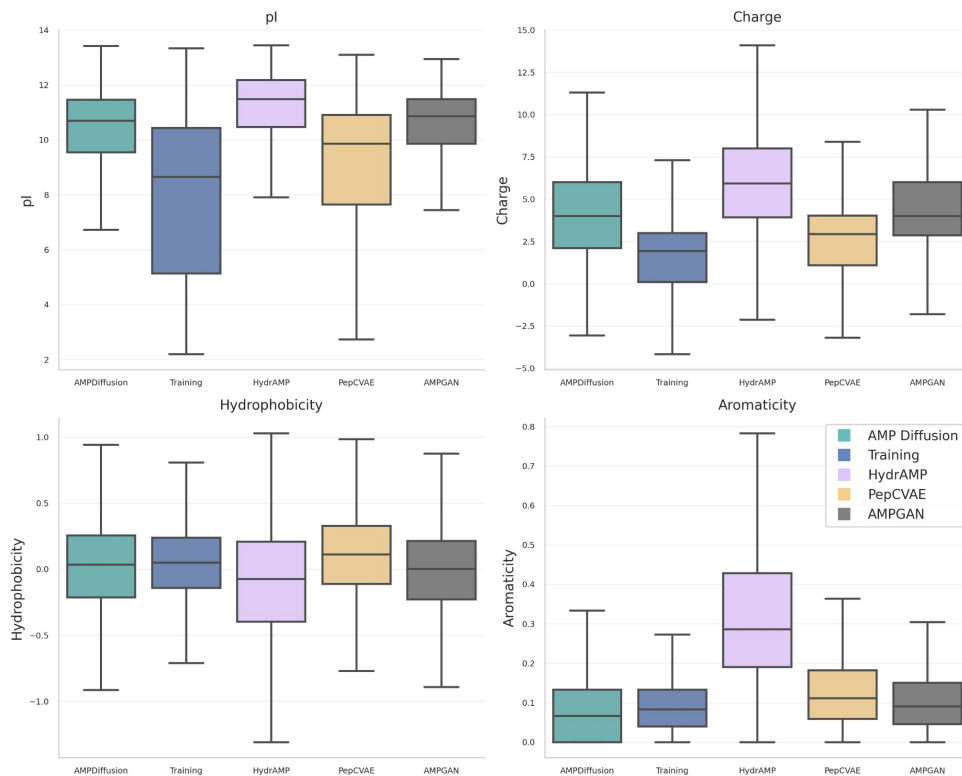
generated_latents = sample_latents(
    model,
    latent_dim=latent_dim,
    num_samples=NUM_SAMPLES,
    sequence_length=GENERATED_LENGTH,
)

generated_sequences = decode_latents(
    generated_latents,
    latent_mean,
    latent_std,
    tokenizer,
    esm2,
)

for index, sequence in enumerate(generated_sequences, start=1):
    print(f">generated_{index}")
    print(sequence)
```

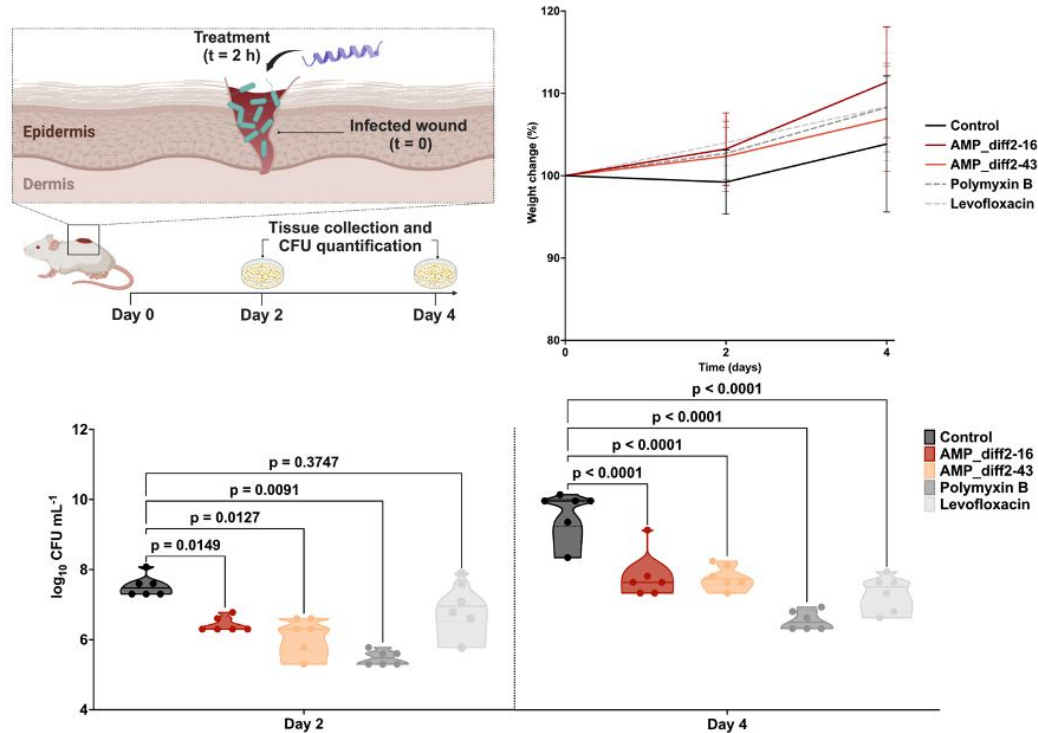


Model-derived AMPs are not only physicochemically valid,





Model-derived AMPs are not only physicochemically valid, but they also have broad anti-infective activities in animal models!



These code operations use a finite sequence of steps indexed by k .

These code operations use a finite sequence of steps indexed by k .

We will now show that, as each step becomes smaller, the same shrink-and-noise update becomes the continuous variance-preserving SDE I showed you earlier.

Suppose one discrete step spans a small continuous interval Δt .

Suppose one discrete step spans a small continuous interval Δt .

We choose

$$\beta_k \approx \beta(t_k) \Delta t.$$

Suppose one discrete step spans a small continuous interval Δt .

We choose

$$\beta_k \approx \beta(t_k) \Delta t.$$

Where, the discrete noise variance β_k is approximately the continuous noise rate $\beta(t_k)$ multiplied by the small time interval Δt .

Suppose one discrete step spans a small continuous interval Δt .

We choose

$$\beta_k \approx \beta(t_k) \Delta t$$

Because

$$\alpha_k = 1 - \beta_k$$

we have

$$\alpha_k \approx 1 - \beta(t_k) \Delta t$$

Suppose one discrete step spans a small continuous interval Δt .

We choose

$$\beta_k \approx \beta(t_k) \Delta t.$$

Because

$$\alpha_k = 1 - \beta_k$$

we have

$$\alpha_k \approx 1 - \beta(t_k) \Delta t$$

For small u ,

$$\sqrt{1 - u} \approx 1 - \frac{u}{2}$$

Suppose one discrete step spans a small continuous interval Δt .

We choose

$$\beta_k \approx \beta(t_k) \Delta t$$

Because

$$\alpha_k = 1 - \beta_k$$

we have

$$\alpha_k \approx 1 - \beta(t_k) \Delta t$$

For small u ,

$$\sqrt{1 - u} \approx 1 - \frac{u}{2}$$

Set $u = \beta(t_k) \Delta t$:

$$\sqrt{\alpha_k} \approx 1 - \frac{1}{2} \beta(t_k) \Delta t$$

Suppose one discrete step spans a small continuous interval Δt .

We choose

$$\beta_k \approx \beta(t_k) \Delta t.$$

Because

$$\alpha_k = 1 - \beta_k$$

we have

$$\alpha_k \approx 1 - \beta(t_k) \Delta t$$

For small u ,

$$\sqrt{1 - u} \approx 1 - \frac{u}{2}$$

Set $u = \beta(t_k) \Delta t$:

$$\sqrt{\alpha_k} \approx 1 - \frac{1}{2} \beta(t_k) \Delta t$$

The DDPM forward update is

$$X_k = \sqrt{\alpha_k} X_{k-1} + \sqrt{\beta_k} \epsilon_k$$

Suppose one discrete step spans a small continuous interval Δt .

We choose

$$\beta_k \approx \beta(t_k) \Delta t.$$

Because

$$\alpha_k = 1 - \beta_k$$

we have

$$\alpha_k \approx 1 - \beta(t_k) \Delta t$$

For small u ,

$$\sqrt{1 - u} \approx 1 - \frac{u}{2}$$

Set $u = \beta(t_k) \Delta t$:

$$\sqrt{\alpha_k} \approx 1 - \frac{1}{2} \beta(t_k) \Delta t$$

The DDPM forward update is

$$X_k = \sqrt{\alpha_k} X_{k-1} + \sqrt{\beta_k} \epsilon_k$$

Substitute the small-step approximations:

$$X_k \approx \left[1 - \frac{1}{2}\beta(t_k)\Delta t \right] X_{k-1} + \sqrt{\beta(t_k)\Delta t} \varepsilon_k$$

Substitute the small-step approximations:

$$X_k \approx \left[1 - \frac{1}{2}\beta(t_k)\Delta t \right] X_{k-1} + \sqrt{\beta(t_k)\Delta t} \varepsilon_k$$

Subtract X_{k-1} from both sides:

$$X_k - X_{k-1} \approx -\frac{1}{2}\beta(t_k)X_{k-1}\Delta t + \sqrt{\beta(t_k)}\sqrt{\Delta t}\varepsilon_k.$$

Substitute the small-step approximations:

$$X_k \approx \left[1 - \frac{1}{2}\beta(t_k)\Delta t \right] X_{k-1} + \sqrt{\beta(t_k)\Delta t} \varepsilon_k$$

Subtract X_{k-1} from both sides:

$$X_k - X_{k-1} \approx -\frac{1}{2}\beta(t_k)X_{k-1}\Delta t + \sqrt{\beta(t_k)}\sqrt{\Delta t} \varepsilon_k.$$

So the change $X_k - X_{k-1}$ over a small interval Δt approximately equals the variance-preserving drift $-(1/2)\beta(t_k)X_{k-1}\Delta t$ plus a Gaussian increment ε_k scaled by $\sqrt{\beta(t_k)}\sqrt{\Delta t}$.

Substitute the small-step approximations:

$$X_k \approx \left[1 - \frac{1}{2}\beta(t_k)\Delta t \right] X_{k-1} + \sqrt{\beta(t_k)\Delta t} \varepsilon_k$$

Subtract X_{k-1} from both sides:

$$X_k - X_{k-1} \approx -\frac{1}{2}\beta(t_k)X_{k-1}\Delta t + \sqrt{\beta(t_k)}\sqrt{\Delta t} \varepsilon_k.$$

This is the Euler-Maruyama update for the variance-preserving SDE:

$$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)} dW_t$$

Where $\beta(t)$ is the continuous noise rate.

So the forward correspondence is

$$\text{VP SDE} \longrightarrow \text{DDPM forward chain}$$

when we discretize continuous time into finite steps.

So the forward correspondence is

$$\text{VP SDE} \longrightarrow \text{DDPM forward chain}$$

when we discretize continuous time into finite steps.

The reverse correspondence is

$$\text{reverse-time SDE} \longrightarrow \text{DDPM reverse chain}$$

So the forward correspondence is

$$\text{VP SDE} \longrightarrow \text{DDPM forward chain}$$

when we discretize continuous time into finite steps.

The reverse correspondence is

$$\text{reverse-time SDE} \longrightarrow \text{DDPM reverse chain}$$

when we numerically approximate the reverse stochastic process.

So the forward correspondence is

$$\text{VP SDE} \longrightarrow \text{DDPM forward chain}$$

when we discretize continuous time into finite steps.

The reverse correspondence is

$$\text{reverse-time SDE} \longrightarrow \text{DDPM reverse chain}$$

when we numerically approximate the reverse stochastic process.

The continuous VP SDE describes infinitely many infinitesimal corruption updates.

The DDPM forward chain groups those updates into K finite Gaussian transitions.

So the forward correspondence is

$$\text{VP SDE} \longrightarrow \text{DDPM forward chain}$$

when we discretize continuous time into finite steps.

The reverse correspondence is

$$\text{reverse-time SDE} \longrightarrow \text{DDPM reverse chain}$$

when we numerically approximate the reverse stochastic process.

The continuous VP SDE describes infinitely many infinitesimal corruption updates.

The DDPM forward chain groups those updates into K finite Gaussian transitions.

The continuous reverse-time SDE describes infinitesimal denoising updates.

So the forward correspondence is

$$\text{VP SDE} \longrightarrow \text{DDPM forward chain}$$

when we discretize continuous time into finite steps.

The reverse correspondence is

$$\text{reverse-time SDE} \longrightarrow \text{DDPM reverse chain}$$

when we numerically approximate the reverse stochastic process.

The continuous VP SDE describes infinitely many infinitesimal corruption updates.

The DDPM forward chain groups those updates into K finite Gaussian transitions.

The continuous reverse-time SDE describes infinitesimal denoising updates. DDPM therefore translates the continuous diffusion theory into explicit Gaussian updates that we can implement, train, and sample.

Okay, so the DDPM code generated each reverse state by combining a learned mean with Gaussian noise:

$$X_{k-1} = \mu_{\theta}(X_k, k) + \sigma_k Z_k$$

Okay, so the DDPM code generated each reverse state by combining a learned mean with Gaussian noise:

$$X_{k-1} = \mu_{\theta}(X_k, k) + \sigma_k Z_k$$

The reverse chain therefore follows a stochastic trajectory. Even after fixing X_k , different draws of Z_k can produce different preceding states.

Okay, so the DDPM code generated each reverse state by combining a learned mean with Gaussian noise:

$$X_{k-1} = \mu_{\theta}(X_k, k) + \sigma_k Z_k$$

The reverse chain therefore follows a stochastic trajectory. Even after fixing X_k , different draws of Z_k can produce different preceding states.

So this raises a question: can we remove the random increments and construct a deterministic trajectory that passes through the same marginal distributions p_t ?

Okay, so the DDPM code generated each reverse state by combining a learned mean with Gaussian noise:

$$X_{k-1} = \mu_{\theta}(X_k, k) + \sigma_k Z_k$$

The reverse chain therefore follows a stochastic trajectory. Even after fixing X_k , different draws of Z_k can produce different preceding states.

So this raises a question: can we remove the random increments and construct a deterministic trajectory that passes through the same marginal distributions p_t ?

Two Ways to Evolve a Probability Density

From Section 2, a deterministic velocity field v_t evolves its density according to the continuity equation:

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot [p_t(x)v_t(x)]$$

Two Ways to Evolve a Probability Density

From Section 2, a deterministic velocity field v_t evolves its density according to the continuity equation:

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot [p_t(x)v_t(x)]$$

Here, the rate of change of the density $p_t(x)$ equals the negative divergence of the probability flux $p_t(x)v_t(x)$, where $v_t(x)$ is the deterministic velocity assigned to state x at time t .

Two Ways to Evolve a Probability Density

From Section 2, a deterministic velocity field v_t evolves its density according to the continuity equation:

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot [p_t(x)v_t(x)]$$

A diffusion SDE instead evolves individual states through drift and random motion:

$$dX_t = f_t(X_t) dt + g(t) dW_t$$

Two Ways to Evolve a Probability Density

From Section 2, a deterministic velocity field v_t evolves its density according to the continuity equation:

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot [p_t(x)v_t(x)]$$

A diffusion SDE instead evolves individual states through drift and random motion:

$$dX_t = f_t(X_t) dt + g(t) dW_t$$

Its density follows the Fokker-Planck equation:

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot [f_t(x)p_t(x)] + \frac{1}{2}g^2(t)\Delta_x p_t(x)$$

Two Ways to Evolve a Probability Density

From Section 2, a deterministic velocity field v_t evolves its density according to the continuity equation:

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot [p_t(x)v_t(x)]$$

A diffusion SDE instead evolves individual states through drift and random motion:

$$dX_t = f_t(X_t) dt + g(t) dW_t$$

Its density follows the Fokker-Planck equation:

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot [f_t(x)p_t(x)] + \frac{1}{2}g^2(t)\Delta_x p_t(x)$$

Where the rate of change of $p_t(x)$ equals the transport produced by the drift $f_t(x)$, plus the density spreading produced by Brownian motion with diffusion scale $g(t)$.

Two Ways to Evolve a Probability Density

From Section 2, a deterministic velocity field v_t evolves its density according to the continuity equation:

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot [p_t(x)v_t(x)]$$

A diffusion SDE instead evolves individual states through drift and random motion:

$$dX_t = f_t(X_t) dt + g(t) dW_t$$

Its density follows the Fokker-Planck equation:

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot [f_t(x)p_t(x)] + \frac{1}{2}g^2(t)\Delta_x p_t(x)$$

So the continuity equation describes deterministic transport, while the Fokker-Planck equation includes both deterministic transport and stochastic spreading.

To connect them, we must express the spreading term as part of an effective velocity.

To connect them, we must express the spreading term as part of an effective velocity.

Recall the score:

$$s_t(x) = \nabla_x \log p_t(x)$$

To connect them, we must express the spreading term as part of an effective velocity.

Recall the score:

$$s_t(x) = \nabla_x \log p_t(x)$$

Because

$$\nabla_x p_t(x) = p_t(x) \nabla_x \log p_t(x) = p_t(x) s_t(x)$$

To connect them, we must express the spreading term as part of an effective velocity.

Recall the score:

$$s_t(x) = \nabla_x \log p_t(x)$$

Because $\nabla_x p_t(x) = p_t(x) \nabla_x \log p_t(x) = p_t(x) s_t(x)$

the Laplacian can be written as

$$\Delta_x p_t(x) = \nabla_x \cdot [p_t(x) s_t(x)]$$

To connect them, we must express the spreading term as part of an effective velocity.

Recall the score:

$$s_t(x) = \nabla_x \log p_t(x)$$

Because $\nabla_x p_t(x) = p_t(x) \nabla_x \log p_t(x) = p_t(x) s_t(x)$

the Laplacian can be written as

$$\Delta_x p_t(x) = \nabla_x \cdot [p_t(x) s_t(x)]$$

Substitute this identity into the Fokker-Planck equation:

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot [f_t(x) p_t(x)] + \frac{1}{2} g^2(t) \nabla_x \cdot [p_t(x) s_t(x)]$$

Combine both terms into one divergence

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot \left[p_t(x) \left(f_t(x) - \frac{1}{2} g^2(t) s_t(x) \right) \right]$$

Combine both terms into one divergence

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot \left[p_t(x) \left(f_t(x) - \frac{1}{2}g^2(t)s_t(x) \right) \right]$$

This has exactly the form of the continuity equation. Therefore, the corresponding deterministic velocity is

$$v_t(x) = f_t(x) - \frac{1}{2}g^2(t)s_t(x)$$

Combine both terms into one divergence

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot \left[p_t(x) \left(f_t(x) - \frac{1}{2} g^2(t) s_t(x) \right) \right]$$

This has exactly the form of the continuity equation. Therefore, the corresponding deterministic velocity is

$$v_t(x) = f_t(x) - \frac{1}{2} g^2(t) s_t(x)$$

Where, the probability-flow velocity $v_t(x)$ equals the original SDE drift $f_t(x)$ minus one-half of the squared diffusion scale $g^2(t)$ multiplied by the score $s_t(x)$.

Combine both terms into one divergence

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot \left[p_t(x) \left(f_t(x) - \frac{1}{2} g^2(t) s_t(x) \right) \right]$$

This has exactly the form of the continuity equation. Therefore, the corresponding deterministic velocity is

$$v_t(x) = f_t(x) - \frac{1}{2} g^2(t) s_t(x)$$

Brownian motion spreads probability in the SDE.

Combine both terms into one divergence

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot \left[p_t(x) \left(f_t(x) - \frac{1}{2} g^2(t) s_t(x) \right) \right]$$

This has exactly the form of the continuity equation. Therefore, the corresponding deterministic velocity is

$$v_t(x) = f_t(x) - \frac{1}{2} g^2(t) s_t(x)$$

Brownian motion spreads probability in the SDE. The probability-flow velocity reproduces the same density spreading by moving states deterministically according to the score.

Combine both terms into one divergence

$$\frac{\partial p_t(x)}{\partial t} = -\nabla_x \cdot \left[p_t(x) \left(f_t(x) - \frac{1}{2} g^2(t) s_t(x) \right) \right]$$

This has exactly the form of the continuity equation. Therefore, the corresponding deterministic velocity is

$$v_t(x) = f_t(x) - \frac{1}{2} g^2(t) s_t(x)$$

Brownian motion spreads probability in the SDE. The probability-flow velocity reproduces the same density spreading by moving states deterministically according to the score.

This velocity leads directly to the probability-flow ODE.

The Probability-Flow ODE

The Probability-Flow ODE

The deterministic process is

$$\frac{dX_t}{dt} = f_t(X_t) - \frac{1}{2}g^2(t)\nabla_x \log p_t(X_t)$$

The Probability-Flow ODE

The deterministic process is

$$\frac{dX_t}{dt} = f_t(X_t) - \frac{1}{2}g^2(t)\nabla_x \log p_t(X_t)$$

The instantaneous velocity of X_t equals the SDE drift $f_t(X_t)$ minus one-half of $g^2(t)$ multiplied by the score $\nabla_x \log p_t(X_t)$, which points toward states of increasing probability density.

The Probability-Flow ODE

The deterministic process is

$$\frac{dX_t}{dt} = f_t(X_t) - \frac{1}{2}g^2(t)\nabla_x \log p_t(X_t)$$

The instantaneous velocity of X_t equals the SDE drift $f_t(X_t)$ minus one-half of $g^2(t)$ multiplied by the score $\nabla_x \log p_t(X_t)$, which points toward states of increasing probability density.

If the score is exact and both processes begin from the same distribution, the SDE and probability-flow ODE produce the same marginal density p_t at every time.

The Probability-Flow ODE

The deterministic process is

$$\frac{dX_t}{dt} = f_t(X_t) - \frac{1}{2}g^2(t)\nabla_x \log p_t(X_t)$$

The instantaneous velocity of X_t equals the SDE drift $f_t(X_t)$ minus one-half of $g^2(t)$ multiplied by the score $\nabla_x \log p_t(X_t)$, which points toward states of increasing probability density.

If the score is exact and both processes begin from the same distribution, the SDE and probability-flow ODE produce the same marginal density p_t at every time.

Their individual trajectories remain different. The SDE repeatedly introduces random motion, while the ODE assigns one deterministic trajectory to each initial state.

Take Brownian Motion Without Random Paths

Consider the forward SDE $dX_t = dW_t$, $X_0 \sim \mathcal{N}(0, 1)$

Take Brownian Motion Without Random Paths

Consider the forward SDE $dX_t = dW_t$, $X_0 \sim \mathcal{N}(0, 1)$

Brownian motion increases the variance linearly:

$$p_t(x) = \mathcal{N}(0, 1 + t)$$

Take Brownian Motion Without Random Paths

Consider the forward SDE $dX_t = dW_t$, $X_0 \sim \mathcal{N}(0, 1)$

Brownian motion increases the variance linearly:

$$p_t(x) = \mathcal{N}(0, 1 + t)$$

The score is

$$s_t(x) = -\frac{x}{1 + t}$$

Take Brownian Motion Without Random Paths

Consider the forward SDE $dX_t = dW_t$, $X_0 \sim \mathcal{N}(0, 1)$

Brownian motion increases the variance linearly:

$$p_t(x) = \mathcal{N}(0, 1 + t)$$

The score is
$$s_t(x) = -\frac{x}{1 + t}$$

Because $f_t(x) = 0$ and $g(t) = 1$, the probability-flow velocity becomes

$$v_t(x) = -\frac{1}{2}s_t(x) = \frac{x}{2(1 + t)}$$

Take Brownian Motion Without Random Paths

Consider the forward SDE $dX_t = dW_t$, $X_0 \sim \mathcal{N}(0, 1)$

Brownian motion increases the variance linearly:

$$p_t(x) = \mathcal{N}(0, 1 + t)$$

The score is
$$s_t(x) = -\frac{x}{1 + t}$$

Because $f_t(x) = 0$ and $g(t) = 1$, the probability-flow velocity becomes

$$v_t(x) = -\frac{1}{2}s_t(x) = \frac{x}{2(1 + t)}$$

Therefore,

$$\frac{dX_t}{dt} = \frac{X_t}{2(1 + t)}$$

Take Brownian Motion Without Random Paths

Consider the forward SDE $dX_t = dW_t$, $X_0 \sim \mathcal{N}(0, 1)$

Brownian motion increases the variance linearly:

$$p_t(x) = \mathcal{N}(0, 1 + t)$$

The score is

$$s_t(x) = -\frac{x}{1 + t}$$

Because $f_t(x) = 0$ and $g(t) = 1$, the probability-flow velocity becomes

$$v_t(x) = -\frac{1}{2}s_t(x) = \frac{x}{2(1 + t)}$$

Therefore,

$$\frac{dX_t}{dt} = \frac{X_t}{2(1 + t)}$$

Solving the ODE gives

$$X_t = \sqrt{1 + t} X_0$$

Take Brownian Motion Without Random Paths

Consider the forward SDE $dX_t = dW_t$, $X_0 \sim \mathcal{N}(0, 1)$

Brownian motion increases the variance linearly:

$$p_t(x) = \mathcal{N}(0, 1 + t)$$

The score is

$$s_t(x) = -\frac{x}{1 + t}$$

Because $f_t(x) = 0$ and $g(t) = 1$, the probability-flow velocity becomes

$$v_t(x) = -\frac{1}{2}s_t(x) = \frac{x}{2(1 + t)}$$

Therefore,

$$\frac{dX_t}{dt} = \frac{X_t}{2(1 + t)}$$

Solving the ODE gives

$$X_t = \sqrt{1 + t} X_0$$

The state X_t equals the initial state X_0 scaled by $\sqrt{1 + t}$, so the deterministic trajectory expands the distribution at exactly the rate required to match Brownian motion.

For example, if $X_0 = 2$ and $t = 3$,

For example, if $X_0 = 2$ and $t = 3$,

$$X_3 = \sqrt{4}(2) = 4$$

For example, if $X_0 = 2$ and $t = 3$,

$$X_3 = \sqrt{4}(2) = 4$$

More importantly, if

$$X_0 \sim \mathcal{N}(0, 1)$$

For example, if $X_0 = 2$ and $t = 3$,

$$X_3 = \sqrt{4}(2) = 4$$

More importantly, if

$$X_0 \sim \mathcal{N}(0, 1)$$

then

$$X_3 = 2X_0 \sim \mathcal{N}(0, 4)$$

For example, if $X_0 = 2$ and $t = 3$,

$$X_3 = \sqrt{4}(2) = 4$$

More importantly, if

$$X_0 \sim \mathcal{N}(0, 1)$$

Then

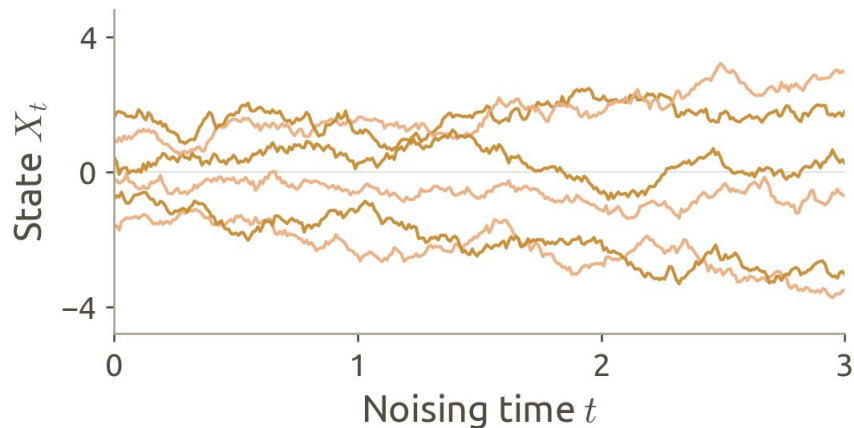
$$X_3 = 2X_0 \sim \mathcal{N}(0, 4)$$

Brownian motion also produces $\mathcal{N}(0,4)$ at $t = 3$. The individual paths differ, yet the marginal distributions agree.

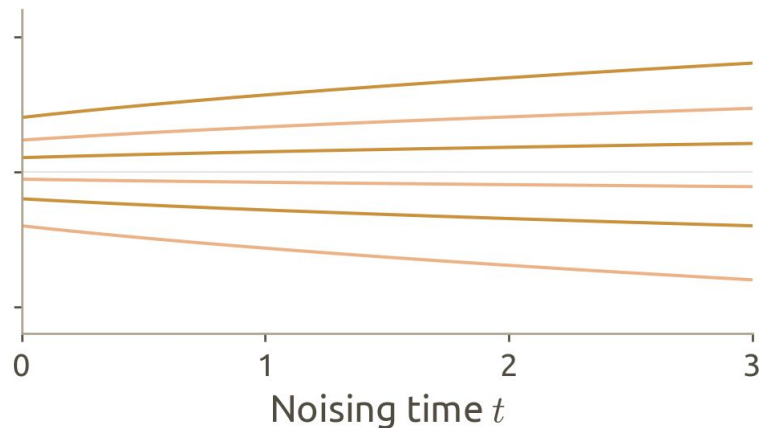
Follow several samples through the same density evolution

Start both processes from a standard Gaussian. The curves below show individual sample paths.

Brownian sample paths

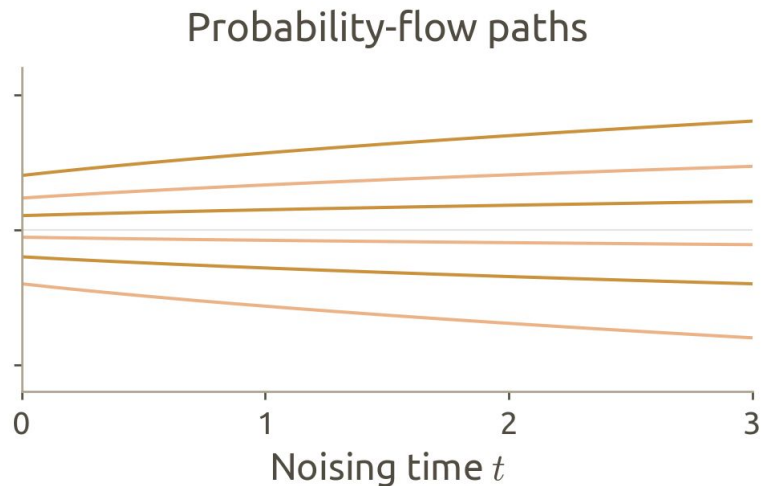
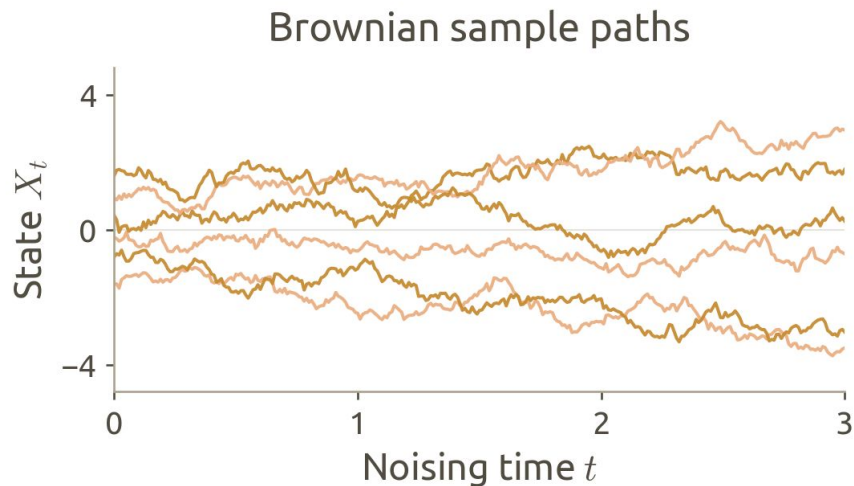


Probability-flow paths



Follow several samples through the same density evolution

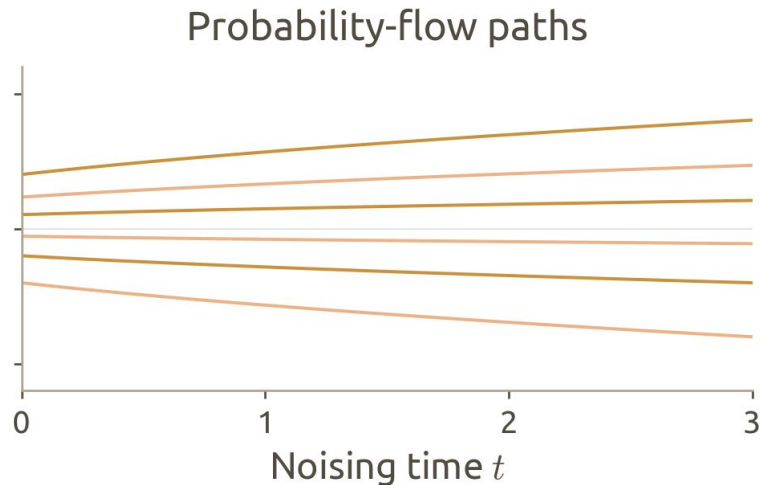
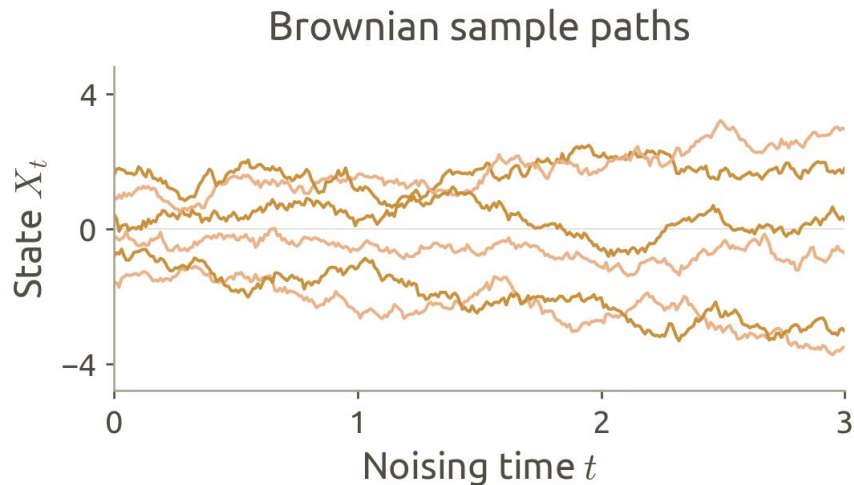
Start both processes from a standard Gaussian. The curves below show individual sample paths.



Under the ODE, each sample moves smoothly away from zero. Along a Brownian path, fresh random increments change the sample's position throughout the interval.

Follow several samples through the same density evolution

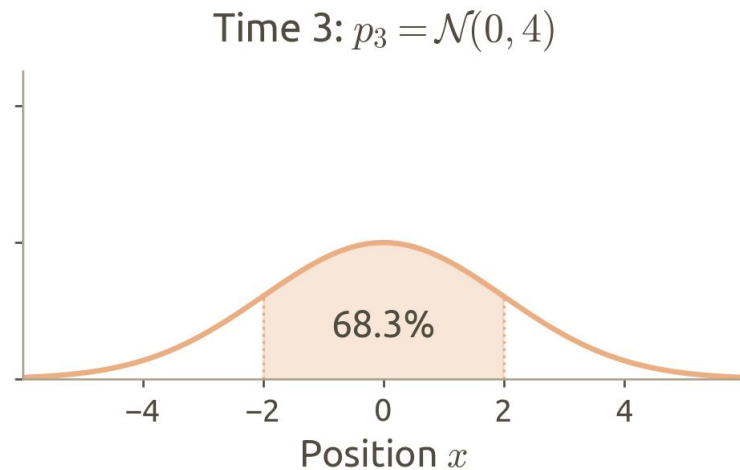
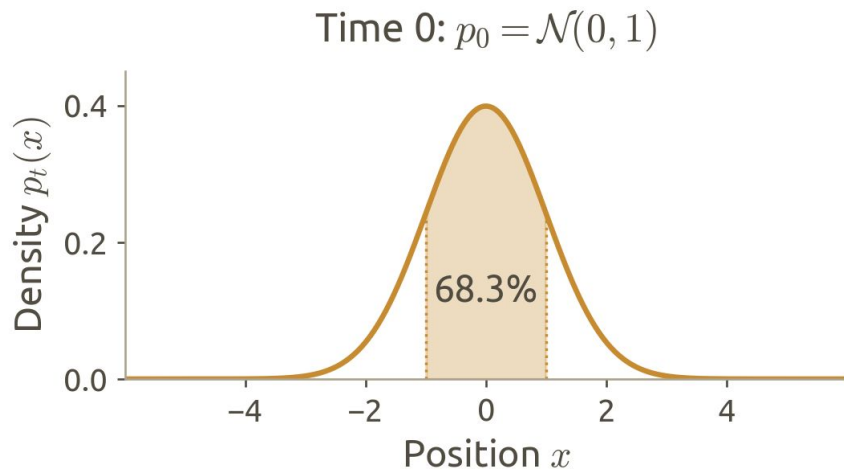
Start both processes from a standard Gaussian. The curves below show individual sample paths.



At any fixed time, both populations have the same Gaussian density when the score is exact.

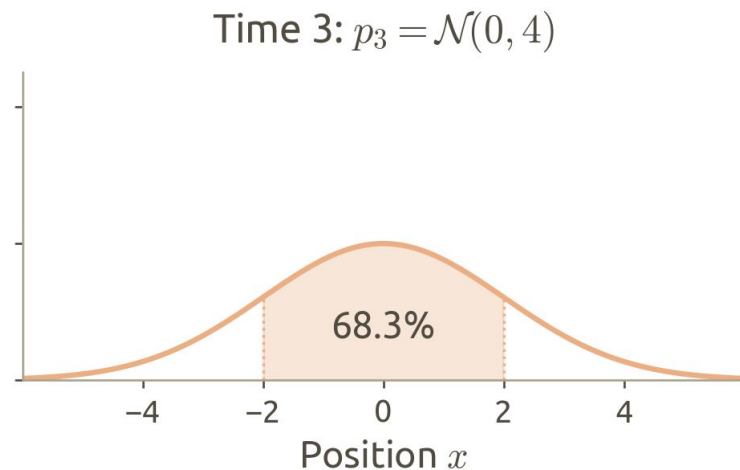
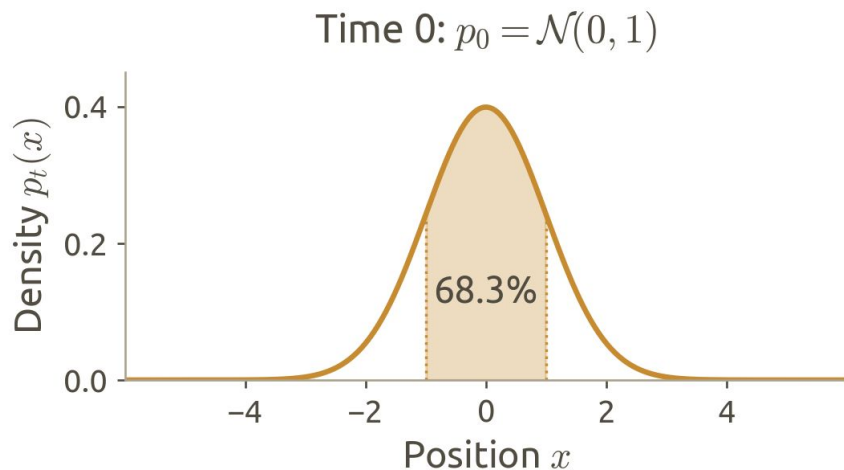
Watch the density widen as the samples move outward

Under the probability-flow ODE, every state doubles between time zero and time three.



Watch the density widen as the samples move outward

Under the probability-flow ODE, every state doubles between time zero and time three.



The density p_0 starts with variance one. At time three, density p_3 has variance four, and the interval containing the central 68.3% doubles in width.

Track the probability inside the moving interval

The initial state X_0 is standard Gaussian, and the ODE transports it to $X_3 = 2X_0$.

$$\Pr(-2 \leq X_3 \leq 2) = \Pr(-1 \leq X_0 \leq 1) \approx 0.6827$$

Track the probability inside the moving interval

The initial state X_0 is standard Gaussian, and the ODE transports it to $X_3 = 2X_0$.

$$\Pr(-2 \leq X_3 \leq 2) = \Pr(-1 \leq X_0 \leq 1) \approx 0.6827$$

The probability that the transported state X_3 lies between minus two and two equals the probability that the initial state X_0 lies between minus one and one, about 68.3%.

Track the probability inside the moving interval

The initial state X_0 is standard Gaussian, and the ODE transports it to $X_3 = 2X_0$.

$$\Pr(-2 \leq X_3 \leq 2) = \Pr(-1 \leq X_0 \leq 1) \approx 0.6827$$

The probability that the transported state X_3 lies between minus two and two equals the probability that the initial state X_0 lies between minus one and one, about 68.3%.

The interval moves with the samples, so it keeps the same probability even as it becomes wider.

Calculate the density after doubling the width

Keep the same map $X_3 = 2X_0$ and evaluate the density at the center, $x = 0$.

$$p_3(2x) = \frac{p_0(x)}{2}, \quad p_3(0) = \frac{0.3989}{2} \approx 0.1995$$

Calculate the density after doubling the width

Keep the same map $X_3 = 2X_0$ and evaluate the density at the center, $x = 0$.

$$p_3(2x) = \frac{p_0(x)}{2}, \quad p_3(0) = \frac{0.3989}{2} \approx 0.1995$$

The density p_3 at transported position $2x$ equals the initial density p_0 at x divided by two, so the peak at zero falls from about 0.3989 to 0.1995.

Calculate the density after doubling the width

Keep the same map $X_3 = 2X_0$ and evaluate the density at the center, $x = 0$.

$$p_3(2x) = \frac{p_0(x)}{2}, \quad p_3(0) = \frac{0.3989}{2} \approx 0.1995$$

The density p_3 at transported position $2x$ equals the initial density p_0 at x divided by two, so the peak at zero falls from about 0.3989 to 0.1995.

Each small interval doubles in width and keeps its probability. Its probability per unit length therefore halves.

What about Probability Flow for the Variance-Preserving SDE?

What about Probability Flow for the Variance-Preserving SDE?

Remember the forward VP SDE:

$$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)} dW_t$$

What about Probability Flow for the Variance-Preserving SDE?

Remember the forward VP SDE:

$$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)} dW_t$$

Therefore,

$$f_t(x) = -\frac{1}{2}\beta(t)x, \quad g^2(t) = \beta(t)$$

What about Probability Flow for the Variance-Preserving SDE?

Remember the forward VP SDE:

$$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)} dW_t$$

Therefore,

$$f_t(x) = -\frac{1}{2}\beta(t)x, \quad g^2(t) = \beta(t)$$

Substituting these terms into the probability-flow ODE gives

$$\boxed{\frac{dX_t}{dt} = -\frac{1}{2}\beta(t) [X_t + s_t(X_t)]}$$

A useful consistency check

A useful consistency check

At the terminal distribution,

$$p_t(x) = \mathcal{N}(0, I) \quad \implies \quad s_t(x) = -x$$

A useful consistency check

At the terminal distribution,

$$p_t(x) = \mathcal{N}(0, I) \quad \Longrightarrow \quad s_t(x) = -x$$

The probability-flow velocity then becomes

$$v_t(x) = -\frac{1}{2}\beta(t)[x - x] = 0$$

A useful consistency check

At the terminal distribution,

$$p_t(x) = \mathcal{N}(0, I) \quad \Longrightarrow \quad s_t(x) = -x$$

The probability-flow velocity then becomes

$$v_t(x) = -\frac{1}{2}\beta(t)[x - x] = 0$$

The standard Gaussian is stationary under the VP process.

A useful consistency check

At the terminal distribution,

$$p_t(x) = \mathcal{N}(0, I) \quad \implies \quad s_t(x) = -x$$

The probability-flow velocity then becomes

$$v_t(x) = -\frac{1}{2}\beta(t)[x - x] = 0$$

The standard Gaussian is stationary under the VP process.

Once the distribution reaches this Gaussian, the contraction and score correction cancel exactly.

Remember that our coded DDPM network did not explicitly predict the score.

It predicted the Gaussian noise ε that had been added to the clean state:

$$X_t = \alpha(t)X_0 + \sigma(t)\varepsilon$$